

Accelerating Multimedia Pipelines with Structural Specialization

by

Arjun Barrett

S.B. Electrical Engineering and Computer Science, Massachusetts Institute of Technology,
2026.

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND
COMPUTER SCIENCE

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

September 2026

© 2026 Arjun Barrett. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Arjun Barrett
Department of Electrical Engineering and Computer Science
August 14, 2026

Certified by: Saman Amarasinghe
Professor of Electrical Engineering and Computer Science, Thesis Supervisor

Accepted by: Katrina LaCurts
Chair, Master of Engineering Thesis Committee

Accelerating Multimedia Pipelines with Structural Specialization

by

Arjun Barrett

Submitted to the Department of Electrical Engineering and Computer Science
on August 14, 2026 in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND
COMPUTER SCIENCE

ABSTRACT

Specialization accelerates a program’s computations by exploiting tighter constraints on its inputs or environment. Most high-performance software makes use of specialization to only a limited degree, primarily at build-time to speed up code running on particular hardware architectures or input shapes. However, there are often opportunities to reduce the computational complexity of an algorithm by exploiting additional information about build-time or run-time parameters within the structure of a program. By reframing software performance engineering challenges as specialization problems, it becomes easier to reason about the remaining opportunities to speed up a given code block. I investigate the effectiveness of specialization-centric optimization in FFmpeg and VMAF, two popular open-source multimedia applications which already use specialization to a lesser extent and have received years of performance engineering effort. The approach drives significant speedups over existing open-source implementations in several video processing tasks. Most notably, I develop a patch to VMAF (a state-of-the-art video quality assessment program) that enables an end-to-end speedup of 40% on modern CPU hardware with output that matches the original implementation exactly on its integer feature paths, and differs only by floating-point reassociation elsewhere. These results demonstrate that specialization is a powerful performance engineering framework, and that it can be used to accelerate even software that has been heavily optimized using traditional techniques.

Thesis supervisor: Saman Amarasinghe

Title: Professor of Electrical Engineering and Computer Science

Acknowledgments

I would like to thank Professor Saman Amarasinghe for his guidance during my time as an undergraduate, and now as an M.Eng. student. Under his supervision, I've found myself diving into the most impactful and interesting research questions I've ever worked on, and I am grateful for the sense of direction he has given me throughout my research journey.

I am also deeply indebted to Dr. Ajay Brahmakshatriya for his constant support and mentorship over the last year. I have learned so much about performance engineering from our chats and meetings, and I would not have gotten to the finish line without his help. More than that, his advice has helped me manage both the professional and personal challenges I've encountered throughout my M.Eng., and I deeply appreciate his kindness.

Thank you to my friends for helping me thrive at MIT and beyond. They believed in me even when I did not, and I will forever be grateful that they have always been by my side over these last few years.

Most of all, thank you to my family for helping me find my own path. Mom and Dad - I will never be able to express how much your unwavering support means to me. Thank you for your love, and I hope you're proud of how far I've come.

Contents

<i>List of Figures</i>	11
<i>List of Tables</i>	13
<i>List of Code Blocks</i>	15
1 Introduction	17
1.1 The Need for Performance	17
1.2 Specialization and the Cost of Generality	18
1.3 Thesis Statement	19
1.4 Contributions	19
1.5 Thesis Organization	20
2 Background	21
2.1 From Hardware to Software	21
2.2 Software Specialization	22
2.2.1 Binding Time	23
2.2.2 Partial Evaluation	24
2.3 Multi-Stage Programming	25
2.3.1 BuildIt	25
2.4 SIMD and AVX-512	26
2.5 Multimedia Software	27
3 Related Work	29
3.1 Partial Evaluation and Automatic Specialization	29
3.2 Multi-Stage Programming and Staged DSLs	30
3.3 Domain-Specific Compilers for Image and Video Pipelines	30
3.4 Hand-Specialization in Production Multimedia	31
3.5 JIT Compilation in Media and Signal Pipelines	32
3.6 The Applicability Gap	32
4 A Framework for Structural Specialization	35
4.1 Structural Specialization	35
4.2 Specialization Axes	36
4.2.1 Shape Specialization	36
4.2.2 Value-Range Specialization	37

4.2.3	Algebraic Specialization	39
4.2.4	Environmental Specialization	39
4.3	Binding Time and Implementation Strategy	40
4.3.1	Static Specialization	40
4.3.2	Finite Runtime Families	41
4.3.3	Partial Specialization	41
4.3.4	Runtime Compilation	41
4.4	A Procedure for Finding Opportunities	42
4.5	Correctness Obligations	42
4.6	A Cost Model: When Specialization Pays	43
4.7	Managing the Combinatorial Explosion	43
4.8	Deployability Criteria	43
5	Case Study: FFmpeg	45
5.1	FFmpeg Filter Architecture	45
5.2	The Convolution Filter	46
5.3	The <code>geq</code> Filter	47
5.3.1	Applying the Procedure	48
5.3.2	Staging the Interpreter	48
5.3.3	Results	50
5.4	Limitations	53
6	Case Study: VMAF	55
6.1	<code>libvmaf</code> as an Adversarial Baseline	55
6.2	VIF	56
6.2.1	Cheaper widening and narrowing	57
6.2.2	Narrowed multipliers	60
6.3	CAMBI	63
6.3.1	AVX-512 vectorization	63
6.3.2	C-values optimization	64
6.3.3	<code>mode3</code> load reduction	65
6.3.4	Top- k selection	66
6.3.5	Boolean DP scan	67
6.4	Motion	70
6.4.1	Striped widening	70
6.4.2	MADD filter computation	71
6.5	ADM	72
6.5.1	Byte-wide multiplies in the DWT	72
6.6	Float Convolution	75
6.6.1	The existing vector path	75
6.6.2	Explicit vectorization	76
6.6.3	Staged code generation with <code>BuildIt</code>	79
6.6.4	Results	83
6.7	Composite Result and Upstreaming	85
6.7.1	Correctness	86

6.7.2	Dependencies	87
6.7.3	Incremental adoption	87
6.7.4	Fallback paths	87
6.7.5	Maintainability	87
7	Evaluation	89
7.1	Experimental Setup	89
7.2	Output Parity	92
7.3	Per-Kernel Results	92
7.3.1	Vectorization alone	95
7.3.2	Composed vectorization and specialization	96
7.4	End-to-End Results	98
7.4.1	Per-feature results	100
7.4.2	Realistic end-to-end with decoding	102
7.5	Baseline Strength	103
7.6	Assessing the Methodology	103
7.7	Limitations	104
7.7.1	Toolchain sensitivity	104
7.7.2	Abstract cost model	105
7.7.3	Maintainability	105
7.7.4	Corpus	106
8	Conclusion	107
8.1	Summary	107
8.2	Future Work	109
A	Speedup Estimation	111
A.1	Derivation	111
A.2	Confidence Intervals	112
A.3	Per-Kernel Speedup Measurements	113
A.4	Limitations	113
B	Complete Measurement Data	115
B.1	Scope	115
B.2	Reading the Tables	116
B.3	Tables	116
B.4	Input Breadth: Resolution, Content and Codec	121
B.5	FFmpeg Filter Specialization	125
	<i>References</i>	129

List of Figures

5.1	FFmpeg filter speedups	52
5.2	Amortizing the generation cost	53
6.1	Contiguous versus striped widening round-trips	59
6.2	The packed boolean prefix-sum scan	69
7.1	Per-kernel speedups	94
7.2	End-to-end speedup by resolution and host, v1 model	99
7.3	End-to-end speedup by resolution and host, v0 model	100
7.4	Wall-clock composition before and after	101

List of Tables

6.1	Specialization axes exercised per feature	57
6.2	Interleaved DWT tap pairs for MADDUBS	74
6.3	Conventional tuning against specialization	85
7.1	Measurement hosts	90
7.2	Per-kernel speedups and specialization axes	93
7.3	Decomposing vectorization from specialization	97
7.4	End-to-end results	99
7.5	FFmpeg end-to-end speedup, one thread against four	102
B.1	End-to-end speedup, all configurations	116
B.2	Baseline wall clock	117
B.3	Per-kernel speedup, 8-bit content	117
B.4	Per-kernel speedup, 10-bit content	118
B.5	Per-feature speedup, 8-bit content	118
B.6	Per-feature speedup, 10-bit content	119
B.7	Per-commit speedup, 4K 8-bit, v0	119
B.8	Per-commit speedup, 4K 8-bit, v1	119
B.9	Per-commit speedup, 4K 10-bit, v0	120
B.10	Per-commit speedup, 4K 10-bit, v1	120
B.11	Resolution sweep, 8-bit, v1	121
B.12	Resolution sweep, 10-bit, v1	122
B.13	Resolution sweep, 8-bit, v0	122
B.14	Resolution sweep, 10-bit, v0	123
B.15	Decode-inclusive speedup, v1	123
B.16	Decode-inclusive speedup, v0	123
B.17	Decode-inclusive speedup, four threads, v1	124
B.18	Decode-inclusive speedup, four threads, v0	124
B.19	Filter-only speedup	126
B.20	End-to-end filter speedup	127
B.21	Decode share and generation cost	127

List of Code Blocks

5.1	Staging the <code>geq</code> evaluator	49
5.2	The residual program for one <code>geq</code> expression	51
6.1	The upstream AVX2 vertical pass	76
6.2	The hand-written vertical pass, abridged	77
6.3	The hand-written horizontal pass, abridged	78
6.4	The middle-out structure in the generator, abridged	80
6.5	Two consecutive tap blocks of the generated middle-out kernel	81
6.6	The sequential structure, abridged	82
6.7	Three consecutive blocks of generated code for the 3-wide filter	82
6.8	The shuffle-favoring horizontal convolution generator, abridged	84
6.9	Generated horizontal pass using immediate-shift windows	84

Chapter 1

Introduction

1.1 The Need for Performance

The computational demands of a single person are orders of magnitude larger today than they were just a few years ago. The last decade has seen rapid advancements in artificial intelligence, video streaming, gaming, and countless other applications. We expect our computers to be able to perform increasingly difficult tasks each year. In the past, this demand was largely serviced by Moore’s law: improvements in semiconductor fabrication alone were enough to support exponential growth in the computational power of our devices. But as fabrication technology approaches the limit of how small a transistor can be, Moore’s law has slowed down. We must now meet ever-increasing computational demands with hardware that cannot keep up.

Hardware and software designers have developed many new approaches to fulfill these expectations. Modern hardware uses dedicated accelerators to decode video or run tensor arithmetic more efficiently than a general-purpose CPU could. Software developers exploit multicore, SIMD, and instruction-level parallelism to optimize their programs’ throughput, and they write code that takes advantage of the specific execution capabilities of modern microarchitectures. There is a common theme among these techniques: instead of relying on faster hardware, engineers shrink the gap between the computation they wish to run and the physical silicon they have to run it.

Today, both the hardware we use and the software we run are extremely well-optimized, and it is becoming more and more difficult to speed things up by optimizing the execution framework for a particular algorithm. To satisfy the needs of today’s applications, we must instead work in reverse. By reformulating a computational problem such that it better fits the capabilities of the hardware, we fundamentally reduce the amount of work needed to solve it.

This thesis focuses on one of the most universally applicable and effective approaches for reducing the amount of work our programs need to do: specialization. By exploiting additional information about the inputs to a particular computation, we can restructure the computation

in such a way that it becomes dramatically easier for our hardware to execute.

1.2 Specialization and the Cost of Generality

Regardless of how a program is represented or what kind of hardware it runs on, generality comes with a price. A routine that accepts arbitrary runtime input can only be represented by machine code covering every case that its caller could ever induce, and it pays the cost of the more expensive instructions required to handle even the rarest edge case regardless of whether that flexibility is actually used. CPUs try to mitigate this cost using smarter hardware, but no amount of branch prediction or move elimination can change the fact that there is fundamentally more work to do when executing a program that handles a wider range of inputs. A compiler cannot remove this overhead on the programmer’s behalf either because it is obligated to preserve the program’s behavior across the entire input domain. It cannot exploit a constraint that holds reliably in the deployed system but is nowhere expressed in the source.

This principle applies to more than just generality in a program’s runtime input. Programs that must be able to run on older hardware can often only target a limited subset of the features available in some ISA, forgoing many useful instructions for the sake of compatibility. In other words, binaries that are general in their supported hardware will pay a performance tax because they cannot take advantage of the unique capabilities of any particular microarchitecture. Even within a high-level program, using a function to generalize a code pattern can lead to slowdowns because a compiler cannot tune the function body for the constraints of any particular callsite when it is invoked from more than one location.

Specialization is the practice of reclaiming the cost of generality. A specialized implementation exploits a stronger set of assumptions about its inputs or environment than the general one is permitted to make, computing the same result with less work. Crucially, this is not approximation: the specialized routine is not trading accuracy for speed, but rather declining to perform work that was only ever necessary for inputs it will not receive.

Viewed this way, several techniques usually treated as unrelated turn out to be instances of one idea. SIMD vectorization specializes a computation for a particular hardware execution model. A fast path for power-of-two sizes in an FFT library specializes for input shape. Just-in-time compilation specializes an interpreter for the one program it has been asked to run. Interpreting different optimization techniques as variants of a single principle is useful because it provides a more rigorous framework for performance engineering. Instead of scanning a project for code patterns we recognize and know how to speed up, we can discover novel optimization opportunities by simply reasoning about the constraints on the program’s inputs, parameters, or environment that hold in practice but are not yet reflected in its implementation.

The difficulty is that this framing has been far more productive in the literature than in practice. Work on partial evaluation, multi-stage programming, and domain-specific compilers has demonstrated large speedups, but those results are most often established

against clean-slate kernels written for the purpose, against naive or scalar baselines, or by rewriting the computation into a new domain-specific language. Applying specialization-based optimization to existing production codebases requires conforming to stricter constraints on build complexity, maintainability, and correctness than a greenfield project. Each of these makes the result difficult to transfer.

Production codebases also impose constraints that this literature rarely models. A change that alters numerical output is unmergeable in a domain with conformance requirements. A technique that requires adopting a new language or build dependency will often be declined regardless of its speedup. Generated code must remain reviewable by maintainers who did not write the generator. These are not incidental engineering details; they are the reason many published specialization results have not reached the software that would benefit from them.

The resulting gap is the subject of this thesis. There is already abundant evidence that specialization helps accelerate unoptimized code [1, 2, 3, 4]. We will see that it remains an extremely effective tool in code which is already state-of-the-art.

1.3 Thesis Statement

I will demonstrate that structural specialization is an effective and practical framework for performance engineering, and that it surfaces substantial gains even in software that expert practitioners have already optimized heavily, under the correctness and deployability constraints that real projects impose.

Establishing this requires a strong baseline. Multimedia software is unusually well suited to the task: it is compute-dense and throughput-critical, its hot kernels are well delimited, its correctness expectations are strict enough to forbid accuracy-for-speed trades, and it is optimized to a degree that few other domains approach.

The primary vehicle is `libvmaf`, the reference implementation of Netflix’s VMAF video quality metric. VMAF is not the subject of this thesis; it is the evidence. It was chosen precisely because it is a hostile target: it contains thousands of lines of existing hand-written SIMD intrinsics, uses core kernels that already saturate execution units on current hardware, and is deployed in a context where sacrificing correctness for speed is not an option. If specialization yields a meaningful speedup here, it will be a testament to the power of specialization as an optimization tool for even the most challenging of codebases.

1.4 Contributions

- **A framework for structural specialization** (Chapter 4): a set of specialization axes, an account of the correctness obligations each axis incurs, a reproducible procedure for locating opportunities in an unfamiliar codebase, and a set of deployability criteria against which the results are judged.

- **A case study on a soft target** (Chapter 5): BuildIt-staged FFmpeg filters, including a `geq` implementation up to $11\times$ faster, which establishes the method as an easy win in less-optimized real-world software.
- **A strong result on a hard target** (Chapter 6, Chapter 7): a ten-patch series for `libvmaf` achieving a 40% end-to-end speedup on the latest VMAF model at 4K, with output that is bit-identical on the integer feature paths and differs only by floating-point reassociation elsewhere, obtained by exploiting value-range invariants in VIF, a boolean domain invariant in CAMBI, a bespoke coefficient-packing technique in ADM, staged generation of a family of width-specialized convolution kernels, and several other specialization techniques.
- **An empirical observation about where headroom remains** (Chapter 8): in both codebases, specialization for the hardware execution model was thoroughly exploited in the existing open-source implementation, while specialization for value ranges and algebraic properties were almost entirely unexploited. This asymmetry suggests a large amount of mature numerical software is leaving comparable gains unclaimed.

1.5 Thesis Organization

Chapter 2 develops specialization as a concept, covering the cost of generality, binding time, partial evaluation, multi-stage programming and BuildIt, and the SIMD substrate on which the later results build. Chapter 3 surveys prior work and characterizes the applicability gap between academic and production codebases in detail. Chapter 4 presents the framework itself. Chapters 5 and 6 apply it, first to an easy target and then to a more challenging one. Chapter 7 evaluates both the performance results and the framework that produced them, including threats to validity. Chapter 8 concludes.

Chapter 2

Background

2.1 From Hardware to Software

A computer can be said to produce its output by applying a fixed procedure to a variable input, either provided by the user or available within the environment. In general, we encode the procedure into a format interpretable by some hardware, which then executes that procedure, optionally pulling data from I/O resources to support the execution. This description neatly fits the execution model of typical CPUs; a program is encoded as an instruction stream in a format defined by the instruction set architecture, and both the instructions and supplemental data are accessed from off-chip DRAM. This general model also applies to GPUs, DSPs, and a wide class of specialized accelerator chips. The primary difference between these hardware architectures is the set of tradeoffs each chooses to make regarding flexibility, performance, and efficiency.

The unfortunate reality of hardware engineering is that an improvement to one of these three criteria almost always comes at the expense of the other two. In order to meet the demands of operating systems and the diverse programs they contain, CPUs are exceptionally *flexible*; they support a wide variety of complex instructions and execution modes, and they include sophisticated features such as virtual memory and interrupts. GPUs give up much of this flexibility for the sake of *performance*; they can only perform a small suite of arithmetic operations, but can perform those operations with dozens of times more throughput than a CPU with similar die area. A small DSP may have the limited execution capabilities of a GPU and low throughput of a CPU, but will be much more *efficient* than either, performing the same work under much tighter power and die area constraints.

These tradeoffs are most easily observed in hardware, but their effects apply analogously to the software we run on that hardware. For the same reason a GPU can multiply matrices faster than a CPU, a specialized program that is purpose-built for a particular task will run faster than a general program that is configured to perform that task. Flexible code often compiles into a larger, costlier, and more complex stream of instructions.

In some sense, flexibility is the least negotiable of the three properties. Slow or inefficient

execution may not be a problem for some computational procedures, but an environment that is too inflexible to run that procedure at all is useless. A natural question, then, is whether or not it is possible to have our cake and eat it too: that is, to run computations quickly and efficiently without sacrificing much-needed flexibility.

In the realm of hardware, the common approach is to package a number of distinct processors optimized for separate tasks into a single system. A typical system-on-a-chip might contain separate accelerators for tensor operations, graphical applications, and video decoding alongside its general purpose CPU cores [5, 6]. By integrating several domain-specific processors into a single system, modern hardware retains the flexibility of a CPU while dramatically accelerating performance on key workloads.

This thesis explores how analogous techniques can accelerate software. By infusing information about the computational problem into the structure of a program, it is possible to accelerate that program by an order of magnitude. More importantly, I find that specialization remains an effective tool regardless of how well-optimized a codebase is; in particular, it can simplify the process of finding major speedups in projects where traditional performance engineering paradigms fall short.

2.2 Software Specialization

Generality in software is costly precisely because more information must be resolved at runtime to support the additional flexibility required of an implementation that handles a broader set of constraints. These costs are pervasive even at small scales, and they manifest as additional work for the hardware. A loop whose trip count is a runtime value necessitates a test and branch on every iteration. A value that could theoretically be large wastes most of the bits in its storage type when it happens to be small. A routine reached through a function pointer must be dispatched dynamically and cannot be inlined into any of its callers.

The cost of generality can even leak into a program without any observable improvement in its flexibility. For example, whenever we use a function to consolidate repeated logic in a program, we reuse the same instructions to execute that function at every callsite, even though different calls may impose different constraints upon the function’s parameters. Therefore, the instructions the compiler generates must be general enough to support the union of all of those constraints, which can make every invocation slower than even the slowest callsite would be if its constraints were fully exploited. The fundamental issue is that a processor executes the same general instructions on every call regardless of which case actually arrived. If we could choose to use a cheaper instruction stream exactly for the cheaper cases, and fall back to a slower one for the more general cases, we could recover this performance gap. This is exactly the use case for specialization.

We define specialization as embedding additional information about a computational problem into an implementation of that problem, so that the implementation is permitted to assume something the general one is not, and can avoid work that was only ever necessary when the assumption failed. The benefits of specializing a program for a particular input are akin to

those of designing a custom hardware accelerator for some task. By embedding assumptions about the workload into the circuitry (analogously, the instruction stream), it is possible to dramatically reduce overhead and thereby compute an output much more efficiently.

Information that a program specializes for can come in many forms. It may concern the input, such as a fixed frame size, a filter extent that stops changing once a pipeline is configured, or a value domain narrower than the declared type permits; it could relate to provable properties of the computation, such as symmetry in a coefficient table or an identity that holds for the particular constants a program was built with; or it could be a property of the environment, such as the vector width and ISA extensions supported by the specific processor a program is running on. These are usually presented as a collection of unrelated optimization techniques, but they all share one key property: they allow an additional fact about the problem to be embodied by the implementation. If this fact fundamentally reshapes the implementation, we refer to the result as *structural specialization*. This is the broad technique that captures all of the optimizations we will discuss in this thesis.

In production software, specialization is usually visible only in its narrowest form, as a fast path for a common shape or a hand-written kernel for a common block size. The useful question is therefore not whether a program is specialized, since every seriously optimized program is specialized to some degree, but which information it has not yet exploited. Chapter 4 transforms that question into a procedure, which we will then apply in Chapters 5 and 6 to uncover real speedups in popular production software.

The rest of this section will touch on the terminology of specialization in the context of software optimization. We will discuss the specialization framework for performance engineering in greater detail in Chapter 4.

2.2.1 Binding Time

Binding time is the moment at which the information supporting a specialization becomes available, and it is the property that decides what can be done with that information. Some facts are settled while the program is being written or built. A coefficient table declared as literals, a target instruction set fixed by a build flag, and an invariant that can be established by reading the function which produces a value are all of this kind, and each can be compiled directly into the only implementation the program ships.

Other facts arrive only once the program starts. The processor it is running on, the resolution of the video it has been handed, and the filter configuration a user supplied are all unknown at build time but fixed at the very beginning of runtime execution. Still other information arrives with each invocation, but holds across a long inner loop, which is nearly as useful. At the far end are quantities for which the latency between learning about the parameter and acting on the parameter is tiny. These are values such as the pixel data itself, for which we only learn about the kind of value we are working with at the moment we need it for a computation. We cannot specialize for these values because the process of exploiting the specialization would take more time than operating dynamically on the value itself. This sets a floor on what any residual program must still compute.

Binding time is a useful parameter upon which to categorize different kinds of specializations because it helps determine the most efficient way to implement a particular specialization. Processor capability is late-bound but has a handful of possible values, so a library can compile scalar, AVX2 and AVX-512 kernels ahead of time and select among them at startup. This enables specialization at runtime without requiring compilation at runtime. By contrast, the same approach is not possible when the late-bound information is high-dimensional. A complete expression tree, a neural-network graph, or a combination of many independent parameters describes more possible programs than could be enumerated, let alone compiled, in advance. There, the choice is to specialize selected components and compose them dynamically, or to generate and compile a residual program while the application runs.

Earlier binding generally permits a more direct and cheaper specialization, while later binding retains more flexibility; neither is better in the abstract. However, in current production software, most specializations concentrate almost entirely on the earliest band. Build-time facts are the ones a hand-written kernel can capture, because capturing them costs one kernel. The load-time and per-invocation bands are where useful facts are plentiful and exploitation is rare, precisely because covering them by hand means writing and maintaining a separate variant for every case. Both case studies in this thesis take their largest gains from that band, and Chapter 4 treats the choice of materialization strategy as a function of binding time and dimensionality, rather than as a consequence of which specific kind of fact is involved.

2.2.2 Partial Evaluation

Given a program p and a portion s of its input that is known in advance, a partial evaluator constructs a residual program p_s which accepts the remaining dynamic input and produces the same result as $p(s, \cdot)$ [1, 7]. Everything that depended only on s has already happened by the time p_s exists, so branches on static conditions resolve, loops with static bounds unroll, and many other overheads disappear before the dynamic input arrives.

This framing has powerful consequences for many kinds of programs. Let p be an interpreter and s a source program, such that $p(s, a)$ is the result of interpreting the program on some input a . Under the partial evaluation framework, p_s is that source program compiled, and the partial evaluator that produced it has acted as a compiler [7]. This means just-in-time compilation need not be treated as a separate discipline with its own machinery; a JIT is what one gets by specializing an existing interpreter with respect to the program it was asked to run. Chapter 5 uses this approach directly, and the practical payoff is that the resulting compiler is derived from an interpreter that was already written, already correct and already reviewed, using only the minimal code transformations needed to run the interpreter through a partial evaluator.

Fully automatic partial evaluation has nevertheless seen limited adoption in production software, for reasons Chapter 3 surveys. The short version is that inferring which values are static is fragile: an incidental change to the source can silently reclassify a value as dynamic, at which point the residual program quietly stops being fast while every test continues to pass. Code growth is difficult to bound, and the systems that do this well have generally required adopting a nonstandard language or toolchain, which is by itself unacceptable for

many projects. By instead manually specifying the components of a program to be partially evaluated, we can make the stronger performance guarantees necessary to make this framing viable.

2.3 Multi-Stage Programming

Multi-stage programming makes the static and dynamic split explicit instead of inferred. The programmer separates first-stage values, which are available while the program is being generated, from second-stage values, which are available only while the generated program executes, and the separation is expressed in the type system [8]. Running the first stage then constructs a residual implementation whose loop bounds, constants, branches and data layout have already been resolved. Nothing about the decomposition is guessed, so nothing about it can silently regress: a value is second-stage because it was declared that way, and the compiler enforces the distinction on the programmer's behalf.

This approach makes it possible to generate a variety of useful primitives for writing specialized code. A loop whose bound is first-stage runs during generation, so the residual program contains its body unrolled rather than the loop itself. A branch on a first-stage condition is decided during generation, so only the surviving arm is emitted and the test is gone. Arithmetic among first-stage values is performed before the residual program exists, so it appears there as a literal. These are the same transformations an optimizing compiler performs whenever it can prove the operands are fixed; the difference is that the staged program is more powerful and explicit in the kinds of transformations it performs, and its correctness does not require the proof because the programmer has asserted the split in the type system.

Staging is also useful at every binding time, which is what makes it the right tool for our applications. A build may run a generator to emit kernels for a known family of widths. An application may run one at initialization to construct a kernel for a configuration it has just discovered. A just-in-time compiler applies the same idea to information that arrives at runtime. Staging is therefore more general than JIT compilation rather than a synonym for it, and runtime compilation is required only in the narrow case where the information is both unavailable until runtime and too high-dimensional to cover with a finite precompiled family.

2.3.1 BuildIt

BuildIt is a C++ library that provides this separation in ordinary C++ source [9, 10]. A second-stage value is declared by wrapping its type, so a `double` that will only be known when the generated kernel runs is written as a `dyn_var<double>`, and everything else in the program remains a first-stage value by default. Operators on second-stage values are overloaded to record the operation rather than perform it, and the generator recovers control flow by re-executing the program along each path a second-stage condition can take. What comes out is C source, which may use explicit vector intrinsics rather than relying on

the downstream compiler’s autovectorizer to rediscover the structure that was just built deliberately.

BuildIt is especially attractive for this work because it introduces no new language and requires no special compiler. A staged program is C++ that a maintainer can read and a standard toolchain can compile, and the generator sits in the same repository as the kernel it produces. This is often a necessary property for production adoption, as we will discuss in Chapter 4.

2.4 SIMD and AVX-512

SIMD is the most familiar case of specializing for a property of the execution environment. A vector implementation assumes an environment that provides vector registers and applies a single instruction to several data elements at once. Because that assumption does not hold on every processor, a portable program resolves it through runtime dispatch: it ships several implementations of the same kernel and tests the processor’s capabilities at startup to pick one. This mechanism is already thoroughly established in mature numerical and multimedia software. Therefore, this thesis does not merely apply AVX-512 to accelerate scalar code; instead, AVX-512 intrinsic code is the baseline upon which we seek to improve. The primary contribution of this thesis in Chapter 6 will build atop existing vector code rather than treating vectorization itself as the result.

Because many of the VMAF changes we have made depend on the particular capabilities of AVX-512, we will touch on them here. A 512-bit register holds sixty-four 8-bit elements, thirty-two 16-bit elements, or sixteen 32-bit elements, and every instruction can be predicated by a lane mask [11]. AVX-512 inherits the quirk of AVX2 that operations crossing 128-bit lanes are more expensive than those that do not. Nonetheless, AVX-512 provides more powerful native instructions for cross-lane permutes and shuffles than did AVX2. Several instruction families recur in Chapter 6. In particular, the packed multiply-add instruction `pmaddwd` multiplies 16-bit lanes pairwise and accumulates into 32-bit lanes, which fuses the two halves of a dot product into one operation, and `pmaddubsw` does the same for unsigned by signed 8-bit products accumulating into 16 bits.

The power of specialization lies in the interaction between these capabilities and information about the data. The number of elements a vector instruction processes is inversely proportional to the width of their representation, so the same 512-bit register that holds 16 32-bit values holds 32 16-bit values, and does twice the useful work per instruction in an operation over the narrower elements. Any fact that allows data to stay in a narrower type therefore converts directly into throughput, with no change to the hardware target. A proof that a value never leaves a narrow range can achieve this, and so can an algebraic reformulation that permits a narrower operation. We will find that because the search space for these kinds of optimizations is so large under traditional performance engineering frameworks, it is worthwhile to search for these kinds of algebraic properties that admit cheaper operations even in kernels that an expert has already vectorized.

2.5 Multimedia Software

The claim this thesis makes is not that specialization produces speedups, which is already well established, but that it continues to produce them in software that experts have already optimized heavily. Multimedia software is an excellent target for several reasons.

First, multimedia programs are computationally dense and throughput-critical. Processing 4K video in real time requires moving and transforming billions of samples per second, and the kernels that do so are limited by arithmetic and by the memory system rather than by waiting on anything external, so a genuine reduction in work appears as a reduction in time rather than being absorbed by an unrelated bottleneck. The kernels are also well delimited. A video pipeline is a small number of hot loops with clear boundaries and stable interfaces, which makes it possible to attribute a change to a specific kernel and to measure that kernel in isolation as well as end to end.

High-quality multimedia software is also already saturated with expert hand-tuning. Production decoders and encoders carry hundreds of hand-written assembly kernels dispatched on processor capability, to the point that the reference AV1 decoder `dav1d` is roughly three-quarters hand-written assembly [12]; Chapter 3 surveys the practice in more detail. A speedup obtained on such a program is demonstrably a speedup over a strong baseline.

Finally, the multimedia domain imposes correctness expectations that generally do not admit lossy transformations of a particular algorithm. Decoders are held to conformance suites and bit-exact output, and a quality metric whose scores are compared across software versions cannot afford to change them. The accuracy-for-speed trades that are available in many other domains are often impossible here. Every optimization must instead compute exactly what the original computed while doing less work to get there. This is exactly the case we make for specialization throughout this thesis.

We will apply specialization techniques to two multimedia programs: FFmpeg and VMAF. FFmpeg is a general purpose video and audio processing toolkit used throughout the media industry, while VMAF is a powerful perceptual video quality metric used to measure the quality degradation of a video encoding process. We will discuss these programs in more detail in their respective chapters.

The FFmpeg filters we target in Chapter 5 are relatively unoptimized, and as such we will use FFmpeg as a calibrating example rather than a primary result. In Chapter 6 we will switch our attention to VMAF, which is very well-optimized and contains dozens of hand-written AVX2 and AVX-512 kernels. We will produce substantial end-to-end speedups on real VMAF workloads as our primary demonstration that specialization-centric optimization can yield an improvement over the state of the art.

Chapter 3

Related Work

3.1 Partial Evaluation and Automatic Specialization

Specialization as a program transformation has been studied extensively in the literature. A partial evaluator takes a program and part of its input, and produces a residual program that computes the same result given the remaining input, with everything derivable from the known part already carried out [1]. The Futamura projections make the consequences precise: specializing an interpreter to a fixed program yields a compiled program, and specializing the specializer itself yields a compiler generator [7]. The staged convolution of Chapter 6 is the first projection applied to a filter rather than to a language, and the FFmpeg work of Chapter 5 is the first projection applied to an interpreter in the ordinary sense.

The classical machinery for deciding what to specialize is binding-time analysis, which partitions a program’s data into what is known at specialization time and what is not. However, many of the largest performance opportunities in a program involve additional information about the inputs to a program, or require insights that are too complex for binding-time analysis to discover in practice. We discuss a richer set of criteria upon which to find specialization opportunities in Chapter 4.

Production compilers perform their own automatic specialization along narrower axes: loop versioning specializes on pointer alignments and values, function multiversioning specializes on the instruction set, and call devirtualization specializes for a program’s control flow. Profile-guided optimization extends this to statistical facts, specializing layout and inlining decisions on observed behavior rather than on proved invariants. All of these are valuable and all of them are already applied to the baseline this thesis improves on, since every measurement here is taken against a fully optimizing build.

We do not claim that automatic specialization is ineffective, but that it is bounded by what a sound analysis can practically prove from the source. The axes exploited in Chapters 5 and 6 lie outside that boundary, and reaching them requires a programmer to state a fact the compiler cannot discover, together with a mechanism for embedding that fact in the program’s structure.

3.2 Multi-Stage Programming and Staged DSLs

Multi-stage programming gives the programmer explicit control over that embedding. MetaML and its descendants introduce staging annotations into the language itself, so that a program can construct and splice fragments of a later-stage program under type-safe rules [8]. Template Haskell provides comparable facilities through compile-time metaprogramming [13], while Lightweight Modular Staging achieves staging as a library by overloading types in the host language, so that ordinary Scala code emits an intermediate representation when applied to staged values [14]. Terra pairs a high-level scripting language with a low-level staged language compiled through LLVM, and is targeted towards performance engineering applications such as ours [15]. These systems differ in expressiveness and in type safety, but generally require writing and maintaining staging code that is structurally different from the code it replaces, and thus harder to reason about and maintain.

BuildIt provides a multi-stage framework that solves this problem [9, 10]. It offers multi-stage primitives as a pure C++ library: staged values are ordinary C++ objects of a library type, the first stage is an ordinary C++ program, and the output is C source that any existing toolchain can compile. No language extension is involved, and the generated code is readable C rather than an intermediate representation. Moreover, BuildIt handles loops and branches on second-stage values with native C++ control flow primitives such as `if` and `while`, rather than the function calls and lambdas many other staged languages require. Because it seamlessly stages most C++ language primitives, staged programs in BuildIt are often structured very similarly to their unstaged counterparts, with the only differences expressed in the type system. Because BuildIt code is much more intuitive to read than other staged languages, it was a natural choice for the production codebases we target, where maintainability is equally important to performance.

3.3 Domain-Specific Compilers for Image and Video Pipelines

A parallel line of work builds domain-specific compilers for image and array pipelines. Halide separates the algorithm from the schedule, so that the same functional description can be compiled under many different loop orders, tilings and vectorization strategies, with autoscheduling to search that space [3]. TVM applies the same separation to tensor programs with an emphasis on automated search over schedules [4]. By contrast, Exo provides the programmer explicit and checkable control over the rewriting steps that take a reference implementation to an optimized one [16]. FFTW is the oldest success in this family and the closest in spirit to the present work, since it generates straight-line code for transform kernels ahead of time and selects among them at run time [17].

These systems are extremely effective at what they target, which is the space of loop transformations: what order to traverse the iteration space in, where to tile, what to vectorize, and what to fuse. That is a different space from the one this thesis operates in. Halide will not decide that a 16-bit multiply-accumulate can be replaced by two byte-wise ones because

the filter coefficients happen to be small, and it will not decide that a prefix sum can run in 8-bit lanes because the values being summed are provably boolean (both of which are optimizations we exploit in Chapter 6). These numerical and value-domain properties of the kernel body are outside the scope of a scheduling-focused language like Halide, and they are where the biggest speedups in our results come from.

The adoption barrier for domain-specific compilers is the same one the staged languages face. Using Halide or TVM for a kernel means rewriting that kernel in a new language and accepting its compiler into the build. Many production multimedia projects have declined to do this, despite the vast amounts of hand-written assembly they often contain. These are projects that care enormously about performance and have still not adopted the tools that the literature offers them.

3.4 Hand-Specialization in Production Multimedia

What production multimedia codebases do instead is specialize by hand, at very large scale. `dav1d`, `x264`, `x265` and `SVT-AV1` each carry hundreds of hand-written assembly kernels, dispatched at run time against detected processor features [12, 18, 19, 20, 21]. This is real specialization and it is highly effective, but it is manual, coarse-grained, and confined to a small number of axes: the instruction set, the block or macroblock size, and the bit depth. These axes share a property that makes them tractable by hand, which is that each takes a handful of values known in advance, so a maintainer can enumerate them.

`libvmaf` is a member of this family and is the specific baseline this thesis improves on [22, 23]. Its feature kernels already carry hand-written AVX2 and AVX-512 implementations behind runtime dispatch, written by contributors who understood the hardware well. This matters for how the results of Chapter 7 should be read: with three exceptions noted there, the comparison is not specialization against naive C but specialization against tuned SIMD.

The ceiling this approach runs into is combinatorial rather than intellectual. Specialization axes multiply: supporting three instruction sets, two bit depths and four block sizes is twenty-four kernels to write and to keep correct as the algorithm evolves. Because every one of those kernels is a separate artifact that a human must maintain, projects specialize only along the few axes whose payoff justifies the burden, and leave the rest unexploited. The unexploited axes are not obscure. They include exactly the value-range and algebraic facts that produce some of the largest speedups in this thesis. They are often left on the table not because they are unknown to the maintainers but because writing and maintaining another twenty-four kernels by hand to capture them is not worth the effort. Staging changes that calculation, because a generated family of kernels costs roughly what one hand-written kernel costs to maintain.

3.5 JIT Compilation in Media and Signal Pipelines

Run-time code generation appears in media pipelines too, though less often than the performance stakes might suggest. FFmpeg embeds an expression evaluator used by filters such as `geq`, which parses a user-supplied formula once and then interprets it per pixel; this interpreter is the baseline for Chapter 5 [24]. Elsewhere, shading and signal-processing systems have adopted LLVM-backed just-in-time compilation to turn user-supplied programs into machine code before execution, as Open Shading Language does for the shaders a renderer is handed at runtime [25].

There is a distinction between the traditional view of runtime code generation, and the specialization view. Traditionally, JIT compilation is seen as a massive endeavor that involves building or embedding a compiler, lowering a user program through an intermediate representation, and emitting machine code at runtime. This approach is powerful and correspondingly expensive, both to implement and to review, and it brings a large dependency into the build. From the specialization perspective, runtime code generation becomes much simpler: take the interpreter that already exists, which is already correct and already reviewed, and stage it with respect to one of its inputs. The result of the second approach is a routine whose output is readable source code, whose correctness argument is that it agrees with the interpreter it was derived from, and whose implementation is a fraction of the size. Chapter 5 takes the second route for precisely these reasons.

3.6 The Applicability Gap

Taken together, the work surveyed above establishes that specialization is powerful and that the machinery to apply it exists. What it does not establish is how much of that power survives contact with a codebase that is already fast and already maintained by people with other priorities.

Reported speedups in the literature are most often demonstrated in one of three settings: on clean-slate kernels written for the paper, against naive or scalar baselines, or on full rewrites of a kernel into a domain-specific language. Each of these makes the result difficult to transfer. A large multiple over a scalar baseline mostly measures vectorization, which the production codebase has already done. A clean-slate kernel does not have to preserve an existing interface, an existing fallback path, or bit-exact output. A DSL rewrite answers a question about the DSL rather than about the technique, and it is an answer the production project has already declined to act on.

Production codebases, meanwhile, impose constraints that the literature rarely models. There must often be no new language, compiler or run-time dependency, or the patch is unmergeable. Output must be bit-identical, or if it cannot be, the deviation must be bounded and acceptably small for the application in question. The code must be readable and maintainable; generated source files must have a generator program that can be understood and modified by the maintainers. Lastly, changes must be incremental, adoptable one kernel at a time, because a full rewrite in a single patch is an unacceptable burden to existing maintainers.

We have strong evidence that specialization helps unoptimized code, and weak evidence that it helps code which is already the state of the art. The second case is the one that determines whether any of this work can reach production software. This thesis closes the gap by construction: the same technique is applied to a soft target in Chapter 5, where the baseline is an interpreter and the achievable multiple is large, and to a hard target in Chapter 6, where the baseline is expert-tuned SIMD and the achievable multiple is correspondingly smaller. Both are evaluated under the production constraints listed above, and the difference between the two results is itself a finding: it quantifies how much of a specialization result survives after conventional performance engineering has already been done.

Chapter 4

A Framework for Structural Specialization

4.1 Structural Specialization

Chapter 2 defined specialization as embedding additional information about a computational problem into an implementation. This chapter formalizes that idea as a performance engineering framework that guides the process of discovering and implementing new optimizations.

Structural specialization changes the implementation itself in response to a fact about the deployed computation, rather than merely passing that fact as another parameter to a general routine. The intended scope is compute-bound kernels whose required outputs remain unchanged. It excludes approximations, distributed scheduling decisions, and I/O optimizations whose dominant costs lie outside the computation. The framework asks three key questions of any kernel to be optimized:

- **Axis of specialization.** What information about the input, computation, or environment is true in deployment but unexploited in the implementation?
- **Binding time.** When does that information become known?
- **Implementation.** How can the specialization be materialized: a static rewrite, dispatch among a finite family, composition of pre-specialized components, or runtime code generation?

Note that none of the individual performance engineering techniques we will discuss are novel. Narrowing an integer type to widen a vector, unrolling a loop whose bound is fixed, reassociating a symmetric filter, dispatching on processor capability, and compiling an expression at runtime are all established practice, and a working performance engineer will recognize every one of them. What this framework instead contributes is the observation that they are variants of the same underlying technique. Each one takes a fact about the problem that the implementation was not previously allowed to assume and rebuilds the implementation around it. Treating them as unrelated tricks is what makes the remaining

opportunities hard to see, because an engineer who has already vectorized a kernel has no reason to ask what else the kernel does not know. Treating them as one question with several answers turns the search into something that can be conducted systematically, which is what §4.4 sets out to do.

4.2 Specialization Axes

The axes below are a generative checklist, not a claim that every optimization belongs to a single pure category. Shape, value domain, and algebraic properties are forms of input specialization: they describe facts about the data or computation supplied to a kernel. Environmental specialization can formally be viewed as another input, namely the target machine, but it is listed separately because conventional performance engineering and compilers already exploit it routinely. Its distinct practical history makes it useful as a baseline against which to judge the less-exploited axes.

4.2.1 Shape Specialization

Shape specialization exploits dimensions, kernel extent, stride, alignment, or another structural property of an input. Consider a one-dimensional blur that accepts an array of coefficients and a width. Its general implementation loops over the width, reloads a coefficient on every iteration, and must keep enough state to handle any legal number of taps. Fixing the width at five removes all of that at once: the loop-bound test and its branch disappear, the five coefficients and input pixels stay in registers across each local window, and the remainder handling for a partial final iteration is never emitted. What is left is five multiply-adds in a straight line.

This is an example of a broader fact: generality often makes certain optimizations inexpressible where specialization would make them trivial. Just as the loop-bound branch is fundamentally unavoidable in a variable-shape convolution, a general implementation is often incapable of reaching the same throughput ceiling as a specialized one. Machines commonly provide operations whose operands must be fixed when the program is written rather than supplied as values, because the operand is encoded into the instruction itself rather than read from a register. Where the fastest formulation of a kernel depends on such an operation, a width known only at runtime cannot use it at all. The general implementation is then not a slower version of the specialized one; it is a structurally different algorithm that must reach the same result by other means. This is the case where staging earns its keep, since a generator can treat the width as fixed while the data remains variable and emit one straight-line kernel per width, which is a program no single hand-written routine can express. Chapter 6 develops a kernel of this kind.

A filter width is often known only at initialization or invocation rather than at build time, which is what makes the axis interesting. A small finite set of possible widths can be covered by variants generated ahead of time and selected at runtime. A large or unbounded set may

instead require partial specialization or runtime compilation. Chapter 6 presents an instance of the finite-family approach; Chapter 5 demonstrates the runtime-compilation approach.

4.2.2 Value-Range Specialization

Value-range specialization exploits a stronger property of the values themselves than the source type conveys. Consider an intermediate quantity in a signal processing pipeline, a partial sum or a scaled difference, which the surrounding algorithm guarantees can never exceed twenty bits. It must be stored in a 32-bit integer. The wasted space in the storage type is unavoidable: the language only offers 8, 16, 32 and 64-bit integers, and there is no choice but to pick the minimum that can always fit the data. The declaration is the tightest one available, and it still overstates the requirement by twelve bits on every value.

That gap is the general condition rather than a defect of any particular program. Bounds that arise from an algorithm or an input can be arbitrary and difficult to express statically: a partial product may have a particular upper bound based on a particular filter coefficient, or a program input might have a variable number of bits. These constraints are inexpressible statically, so the type system is used as a poor proxy: the partial product will be stored in the smallest possible width in the type system that can contain it, and the variable-width input will be defensively stored in a type that handles the maximum number of bits the user could supply. A type system's granularity is coarse relative to the constraints a programmer is actually able to establish statically, and is completely incapable of capturing constraints that are discovered dynamically. In both cases, the slack in the storage type is invisible thereafter, and later stages of the program may do more work to process those values than would be strictly necessary because the information about the value bounds has been lost.

Value-range specialization is the process of recovering this slack. By propagating constraints on values as they are transformed throughout a program, it becomes possible to prove tighter bounds on intermediate values when such a bound would make a computation faster than would otherwise be possible. For instance, a quantity known to fit in twenty bits can be accumulated thousands of times before it threatens a 32-bit register, so a kernel summing such values needs no widening to 64 bits, where a general implementation that must respect the declared range has no choice but to widen and therefore to halve its lane count. The same knowledge can eliminate arithmetic entirely: an expensive function of a twenty-bit argument can be tabulated in 2^{20} entries, small enough to sit in the last-level cache of a modern processor, where the same table over the declared 32-bit range would need 2^{32} entries and could not be cache-resident at all. Neither rewrite changes the answer, and neither is available to an implementation that must assume the full range of the declared type.

So far the constraint has been on which values are *possible*, and that is only half the axis. A second and equally productive kind of knowledge concerns which values are *likely*. The set of values a variable may take is often no smaller than its type permits, while the distribution over that set is severely skewed, and a specialization can exploit the skew without ever restricting the range. Sparse data is the clearest instance. In an array that is mostly zero, a check for a run of zeros followed by a branch that skips the work entirely is worth its cost because it will be taken often. A structurally identical fast path for runs of any other single

value will essentially never be taken, and the program pays the test on every iteration to learn nothing. The two specializations differ in no respect except how often the predicted case occurs, and that difference makes one valuable and the other useless.

This distinction also makes the distributional case qualitatively different. A range constraint is a claim about what the program can produce, and can often be proven by reading the code that produces it. A distributional claim cannot be established that way at all, since nothing in the program forbids the unlikely input; it is a claim about the workload, it holds only in expectation, and a change in the data can retract it silently while every test still passes. The specialization must therefore remain correct when the prediction fails, which in practice means a cheap guard with a general fallback, and it should be sized by how much it wins on the common case rather than by its best case. Chapter 6 presents a selection kernel of this kind, where the input is sparse enough that a complex quickselect algorithm reduces into filling a run of equal values, and the operation becomes something closer to a memory fill than a sort.

The knowledge that a value is narrower than its type, or distributed unevenly within it, reaches an engineer from either of two directions. Sometimes the fact comes from the deployment: a video's samples occupy at most ten bits, so intermediate products stay far below the width their type reserves, and nothing in the source establishes this. Sometimes it comes from inside the program: an array of 16-bit values contains only zeros and ones because the only thing that writes it is a threshold test, and reading that one function settles the matter for every input the program could receive. Both narrow the representation in the same way and buy the same kind of speedup; where they differ is in how they are implemented and maintained. An invariant established by a producing function is provable and survives retuning; a property of typical inputs can only be observed, and therefore wants a runtime guard with a general fallback or must be engineered away until the kernel no longer depends on it.

This axis is especially valuable in production C and C++ because type declarations describe a storage capacity rather than the set of values an algorithm actually produces, let alone the distribution it produces them with. Compilers must respect the capacity, and have no vocabulary for the distribution at all.¹ A specialization may use the smaller set only once its author has established that the stronger property holds.

Everything above concerns numeric values, because that is where the axis pays most often, but what it really exploits is any fact about an input that the general implementation was not permitted to assume. A bound on the range is the common case, and a skew in the distribution is the next most common. In the limiting case, an input might be fixed outright for the lifetime of a computation, where the set of possible values collapses to one. For

¹ A compiler can recover some of this gap on its own: bitwidth analysis infers a tighter width than the declared type for each value by propagating ranges through the program, and was shown to shrink silicon substantially when the result drives hardware synthesis rather than instruction selection [26]. However, this analysis cannot model bounds that hold only because of a fact external to the program source. LLVM and GCC today also support builtins like `__builtin_expect_with_probability` that try to bridge this gap, but these functions are primarily used for boolean types to inform branch and code layout choices. They are generally not effective for the kinds of numeric bounds we discuss in this thesis.

instance, an arithmetic expression that a user supplies once and that is then evaluated over every pixel of every frame is a value of exactly this kind: nothing about it is narrow in the numeric sense, but the general implementation must re-read it on every use where a specialized one can bind it once and compile it away. Chapter 5 develops that case, and §4.3.4 explains why an input of this shape usually has to be materialized by generating code rather than by selecting among variants prepared in advance.

4.2.3 Algebraic Specialization

Algebraic specialization uses an identity or numerical property of the computation itself. A five-tap blur with coefficients $[a, b, c, b, a]$, for example, is normally written as $ax_0 + bx_1 + cx_2 + bx_3 + ax_4$. Symmetry permits the equivalent form $a(x_0 + x_4) + b(x_1 + x_3) + cx_2$, reducing five multiplications to three. This saving does not depend on the pixel values; it follows from a fact about the fixed computation.

Other identities depend on the particular constants rather than on the shape of the expression. A weighted average normally divides by the sum of its weights, and division is among the most expensive arithmetic operations a processor offers. If the weights are fixed and happen to sum to a power of two, the division can be replaced by a shift. If they sum to some other fixed constant, division can still be replaced by a multiplication with a magic constant and a shift [27]. This kind of rewrite is routine in production filter code.

The distinguishing feature of the axis is that the justification lives in the constants rather than in the code. The rewrite above is valid for one particular set of weights and becomes wrong for another, even though nothing about the types, the shapes, or the surrounding program would change to signal it. A reader of the specialized kernel sees a shift and has no way to recover the fact that the shift is only correct because of a property of a table declared somewhere else. That is what makes algebraic specialization both the subtlest axis to find and the easiest to invalidate later.

The facts that make these optimizations correct are commonly known at build time, because they arise from an algorithm or literal constants. They are nevertheless specializations: a general implementation cannot use the identity unless it structurally captures the relevant precondition. This axis often carries the strongest maintenance obligation, since a proof derived from a coefficient table can fail if that table is later retuned and necessitate a rewrite of the specialized kernel.

4.2.4 Environmental Specialization

Environmental specialization selects an implementation for properties of the execution environment: available ISA extensions, vector width, cache geometry, or processor-specific instruction costs. Runtime ISA dispatch among scalar, AVX2, and AVX-512 kernels is its canonical form. For a pixel loop, a library might compile a scalar routine, an AVX2 routine that handles eight elements at a time, and an AVX-512 routine that handles sixteen. A capability test at startup selects the strongest implementation the current processor can

execute. The finite set of alternatives is written or generated ahead of time, while the runtime test selects a compatible one.

This is already standard practice in multimedia software, and compilers can sometimes perform a restricted form of it through autovectorization or function multiversioning. Consequently, it is not in itself evidence of unexplored headroom. In this thesis, it establishes the strength of the baseline and composes with the other axes. The most important wins use input or algebraic information to produce a better AVX-512 implementation than would be possible with naive vector-widening.

This composition between “traditional” performance engineering and specialization is key to effectively using the framework. The two multiply rather than overlap, which is why a codebase that has already been vectorized thoroughly may still have substantial headroom along the other axes. Exhausting the environmental specialization axis says nothing about whether the others have been explored. A machine with wider operations is not uniformly faster than one with narrower operations, because the cost of an operation depends on more than its width. An implementation must be well-tuned to the cost model of the instruction set (or better, the microarchitecture) to achieve strong performance. Chapter 7 reports instances where naive vector widening has no impact on performance, while exploiting the features of the wider SIMD architecture brings a substantial speedup, and further specialization compounds upon that win.

4.3 Binding Time and Implementation Strategy

A single specialization axis may support several implementation strategies. The appropriate strategy follows from the binding time and dimensionality of the information, not from whether the information concerns shape, values, algebra, or the environment.

4.3.1 Static Specialization

When the relevant fact is known while building the program, a single ahead-of-time implementation can embody it, and no dispatch is required because there is nothing to choose between. The weight-normalization rewrite of the previous section is of this kind. The weights are compile-time constants, so whether they sum to a power of two is settled before the program ever runs, and the specialized kernel can simply be the only kernel. The symmetric five-tap blur is static in the same way: the coefficient pattern is fixed when the implementation is built, so the reassociation can be written directly into the source.

Static specialization is the cheapest strategy to execute because it has no runtime cost whatsoever: it can be expressed and exploited entirely at build-time. However, it can be fragile to updates because it depends heavily on internal details of the surrounding source code. If it depends on a particular property of certain filter weights, for example, that filter can no longer be safely changed by simply updating a numeric parameter if that update would break the assumed property. Therefore, static specializations must generally be guarded by

a proof, a static assertion, or a maintenance process that prevents the fact from changing unnoticed.

4.3.2 Finite Runtime Families

When a runtime property has a small domain, a program can compile a finite family of variants and dynamically select one. ISA dispatch is the most common case, but fixed filter widths and a small set of transforms can be treated similarly. A library supporting only three-, five-, and seven-tap filters can generate one unrolled kernel for each width and use the general loop only for an unexpected width. The implementation retains a general fallback for unsupported hardware or inputs. This is runtime specialization without runtime compilation: the dynamic work selects an already-existing residual program.

4.3.3 Partial Specialization

A high-dimensional runtime input can make a full precompiled family impractical. A machine-learning runtime, for example, may receive graphs with arbitrary sequences of matrix operations, activations, and reshapes. Compiling one monolithic kernel for every possible graph is infeasible. One response is to specialize manageable components, such as a fixed library of fused matrix-and-activation kernels, and compose them dynamically for each graph. This reduces the generality cost inside each component without requiring a separate binary for every possible graph. The residual dynamic composition is the price of avoiding that combinatorial explosion.

4.3.4 Runtime Compilation

When the high-dimensional runtime input determines the computation itself, full specialization requires creating a new residual program at runtime. Consider a program that evaluates a user-supplied formula over a large collection of data points, such as an expression applied to every pixel of every frame of a video. For an expression as small as $ax + b$, an interpreter walks a tree of nodes, decides at each node which operation to perform, and fetches that node's operands through the same indirection, and it repeats all of this for every pixel. The useful arithmetic is one multiplication and one addition. Everything else is the interpreter paying for the expressions it might have been given instead.

No ahead-of-time implementation can remove that overhead, because the space of formulas a user might type is unbounded and the specific formula is not known until the program is already running. Once it is known, however, it stays fixed for the remainder of the stream. Compiling it once into a residual kernel and running that kernel over every pixel converts a per-pixel traversal into a one-time cost. Multi-stage compilation then serves as a JIT compiler.

Chapter 5 presents this kind of runtime specialization and contrasts it with the build-time generation used in Chapter 6. Both use staging, but only the former must compile a new

implementation while the application runs.

4.4 A Procedure for Finding Opportunities

The framework is intended to guide discovery, not merely describe an optimization after it succeeds. First, profile the application to identify a kernel whose share of runtime can justify the work, and determine whether computation rather than an external bottleneck limits it. Next, enumerate the generality that kernel pays for: dynamic loop bounds, over-wide representations, general-case branches, worst-case bounds, interpreter dispatch, or a conservative target ISA.

For each cost, ask what is actually true in the deployed setting. The answer may be supplied by an algorithmic invariant, literal constants, caller contracts, an input configuration, or a hardware capability. Classify the fact by the axes in §4.2, determine its binding time, and choose a materialization strategy from §4.3. Then sketch the residual implementation that could assume the fact outright and estimate both its local speedup and its end-to-end ceiling, ensuring that the implementation does not sacrifice its correctness obligations on the edge cases. This procedure makes the key human task explicit: performance engineering begins not with recognizing an instruction pattern, but with recovering the information that the general implementation has discarded.

4.5 Correctness Obligations

A specialization is sound only when the fact it assumes holds for every execution that reaches it. The strongest obligation is a proof from an algorithmic invariant. For instance, pixel intensities may occupy at most eight bits because of what the preceding stage of the pipeline is defined to produce, not because of any constant chosen by a person, so the narrowed representation cannot be invalidated by retuning a table. It could only be invalidated by changing what the producing stage computes, which is a change no reviewer would mistake for a refactor.

A proof from literal constants is weaker in maintenance terms even when it is equally rigorous. A division normalization may be replaced by a shift because one particular set of weights sums to a power of two, and retuning those weights invalidates it while changing nothing else. Specializations of this kind should encode their precondition as a static assertion over the constants wherever the language permits, so that the build fails rather than the output silently drifting, and should document the dependency explicitly where it does not.

Finally, a fact known only at runtime must be tested by a guard, with a general fallback for executions that fail it. ISA dispatch is one example. A runtime shape fast path is another. A guard trades some runtime cost for soundness, but that cost is entirely amortized by the speedup in the specialized kernels for a sufficiently small specialization state space.

Different algorithms will also have different output contracts, and these must be checked

independently of the argument establishing a specialization. Integer transformations may permit exact equivalence, while floating-point reassociation can be a deliberate but measurable numerical trade. The acceptance criterion and the evidence supporting it will often drive the acceptable kinds of transformations the specialization can perform.

4.6 A Cost Model: When Specialization Pays

Specialization is worthwhile only when the information it exploits removes the bottleneck that matters. A value-range rewrite that halves element width can approximately double SIMD lane density, but its gain is smaller if stores or a later wide representation dominate. An algebraic identity can remove many operations from a kernel that occupies one percent of runtime, yet its end-to-end contribution cannot exceed that one percent. If a kernel has baseline share f and local speedup s , its maximum whole-program time reduction is $f(1 - 1/s)$.

Binding time adds further costs. A dispatch has a small selection cost and requires maintaining fallbacks. Generated variants consume code size and instruction-cache capacity. Runtime generation and compilation must be amortized over the subsequent calls to the generated kernel. These costs do not argue against specialization; they identify which variant family is worth materializing and when a partial solution is preferable to a fully specialized one.

4.7 Managing the Combinatorial Explosion

Combining several facts can suggest an exponential family of variants. This is why production libraries often stop at a few ISA variants even when more information is available. Staging changes the engineering tradeoff and reduces the maintenance burden, but does not change the underlying combinatorics. One staged source can produce many residual kernels, and each emitted variant pays binary size, cache capacity, and compilation time for a speedup that may or may not be worthwhile.

The strategies in §4.3 provide ways to control this growth. Static specialization emits one implementation. Finite runtime families limit the supported domain explicitly. Partial specialization concentrates variants in reusable components. Runtime compilation creates only the variants a workload actually requests and can cache them when reuse justifies it. The appropriate choice is the smallest strategy that captures the information responsible for the expected saving.

4.8 Deployability Criteria

Performance alone does not make a specialization deployable in a mature project. This thesis evaluates candidates against five criteria. First, the specialized path must preserve bit-identical output or state and justify an explicit numerical tolerance. Second, it should

avoid imposing a new language, compiler, or mandatory build dependency. Third, it should admit incremental adoption alongside the existing implementation. Fourth, unsupported hardware and inputs must retain a maintained general fallback. Fifth, the implementation or generator must be reviewable by maintainers who did not create it. The case studies and evaluation assess these criteria alongside the performance benefits of each specialization, because an optimization that cannot survive review and maintenance will not be viable in production software.

Chapter 5

Case Study: FFmpeg

Before turning to a codebase where every hot loop has already been hand-vectorized, we will investigate an application of Chapter 4 to two easier targets: the `geq` and `convolution` filters within FFmpeg. Both are real filters that ship in the standard distribution and are occasionally used in production, but neither has been the object of sustained optimization effort. The results in this chapter are large for exactly that reason. They should be read as a verification that the method is effective in real software, rather than as a description of its capabilities in the face of highly optimized code. That evidence comes in Chapter 6, where the numbers are much smaller and correspondingly harder won.

Both filters were reimplemented using the BuildIt multi-stage programming framework as native FFmpeg filters, `buildit_geq` and `buildit_convolution`, with option strings identical to the originals. A user can make use of the optimized, faster path by swapping the binary and prepending `buildit_` to their filter names, without changing anything else about their workload. The filters are fully integrated with FFmpeg’s filtergraph infrastructure.

All measurements in this chapter were taken on an EC2 `m8azn.xlarge` instance, as it is a representative host from the set we use for the more rigorous VMAF benchmarks in Chapter 7.

5.1 FFmpeg Filter Architecture

An FFmpeg filter has two distinct phases. At initialization, it receives its configuration as a string, parses it, validates it, and allocates whatever state it needs. It is later handed frames to filter one at a time and must process each to produce an output frame.¹ The parameters fixed during the first phase do not change during the second: a filtergraph is built once and then run over an entire stream.

That split is a binding-time boundary in the sense of §4.3, and it is the boundary this chapter exploits twice. In both filters, the information that makes specialization possible arrives

¹ Some filters, like the `libvmaf` filter, do not produce any output frames, while others may produce multiple output frames per input frame; it depends on the filter in question. Both BuildIt-accelerated filters we implement are 1-to-1 between input and output.

during initialization; specializing for the input configuration can reduce the work for the millions of pixels the filter must process thereafter. The ratio between the initialization and execution phases is what determines whether specializing is worthwhile. Video uniquely requires an enormous amount of computation that makes this tradeoff pay off faster than for other data modalities: for instance, a single second of 2160p footage is roughly two hundred million pixels.

5.2 The Convolution Filter

The `convolution` filter applies a user-supplied matrix to each plane of each frame. The matrix, its extent, and the normalization divisor all arrive as strings at initialization and are fixed thereafter. The stock implementation has width-specialized implementations for 3×3 , 5×5 , 7×7 and partially specialized implementations for $N \times 1$ or $1 \times N$ filters. However, it mostly uses these specializations for correctness rather than for ideal performance, and the implementations are not written in a manner conducive to vectorization.

Extending the width-specialization to full filter-specialization (that is, specialization for both the filter shape and its coefficients) is straightforward and leads to many more opportunities for speedups than the original implementation exploits. With the extent known, the tap loop unrolls completely; with the weights known, each multiply takes a literal operand rather than a load; and any tap whose weight is zero can disappear from the emitted code entirely. That last optimization is key: although a stronger family of width-specialized baseline kernels could theoretically be better unrolled and have a faster vectorized inner loop, it is not possible to automatically take advantage of properties about the filter structure without more fine-grained specialization. By using a second-stage compiler on a particular realization of the filter weights, we can find a broad suite of work-reduction opportunities autonomously by allowing the compiler to exploit reuse and sparsity on its own. We implement this with a straightforward translation of the kernel to `BuildIt`, which effectively involved a direct transliteration but with `dyn_var` for runtime variables such as the pixel grid, and `static_var` for first-stage values like the filter array.

Note that the stock filter is not one baseline but two. FFmpeg supplies a hand-written SSE4 assembly kernel for the 3×3 square case at eight bits, and nothing at all for any other extent or for the separable row and column modes, which fall back to the C loop. Of the three configurations measured in §5.3.3, only the 3×3 sobel is compared against vectorized code; the 21-tap Gaussian and the 7×7 LoG are compared against scalar C. Disabling FFmpeg’s runtime SIMD dispatch confirms this directly: it slows the stock 3×3 filter by a factor of 1.8 and leaves the other two unchanged to within noise.

There are a few key points about these filters that change how the results should be interpreted. The first point concerns the latter two entirely unvectorized kernels. There the comparison is partly between what an optimizing compiler can do with each of two scalar loops. The stock filter builds an array of pointers, one per tap, and reads through them in its inner loop; to vectorize across the row the compiler would need to establish that all of those runtime pointer values are distinct, non-aliasing, and contiguous, which it cannot, so the loop stays scalar.

The generated kernel indexes a single base pointer with compile-time offsets and vectorizes readily. That is a real advantage, but it is an advantage over code that happens to defeat an autovectorizer rather than over a serious attempt at the problem.

The second point concerns the 3×3 case, which is the demanding one and the only place in this chapter where the baseline was explicitly written to be fast. Even there, the hand-written kernel is handicapped because it must work for an arbitrary user-supplied matrix and targets an older SIMD architecture. The SSE4 3×3 filter processes four pixels per iteration and performs all nine multiply-accumulates whatever the weights happen to be. By contrast, the BuildIt-generated kernel is built with `-march=native` and can thus autovectorize to 512 bits wide, and it performs less work for common kernels whenever possible (e.g. 6 multiplies instead of 9 for a Sobel kernel, since three of the Sobel weights are zero and their taps can be skipped entirely).²

The last point is that FFmpeg’s own response to this situation is itself an argument for the specialization framework. It does ship one 512-bit AVX-512 convolution kernel, but not in this filter: it lives in the separate `sobel` filter, whose weights are fixed by the filter rather than supplied by the user. Faced with a general operator too slow in its general form, the project hand-wrote a specialized copy for the one case common enough to be worth the effort, and users of any other matrix get the general path. Staging the specializer makes it possible to achieve the same performance as that hand-tuned kernel for a wider range of different filter coefficients than the hardcoded ones in the `sobel` filter.³

5.3 The geq Filter

The `geq` filter is the more interesting of the two. It applies a user-supplied arithmetic expression to every pixel: the user writes something like `lum='p(X,Y)*0.5'` on the command line, and the filter evaluates that expression once per pixel, per plane, per frame.

The expression arrives as a string, is parsed into an abstract syntax tree at initialization, and is thereafter evaluated by a recursive interpreter. Evaluating one pixel means walking the tree, dispatching on each node’s operator, and fetching operands through that indirection. For a 2160p stream, the interpreter performs that walk roughly two hundred million times per second of footage, and the arithmetic the user actually asked for is a small fraction of the work done to reach it. The simplest possible expression, the identity `p(X,Y)`, does nothing but copy a pixel, and measuring it isolates the overhead exactly. Everything the filter does beyond actually copying the pixel is interpreter overhead. We will see in §5.3.3 that this overhead remains substantial even as the branch predictor learns the structure of the AST.

² Note that this is not a comparison between two filters compiled under different settings. The original FFmpeg implementation also uses `-march=native`, but the scalar implementation cannot be properly vectorized due to the way it accesses pixels, while the SSE4 implementation is already hand-written assembly and cannot be automatically extended to 512-bit. However, this is still a relatively “cheap” performance win; more impressive results follow in §5.3 and Chapter 6.

³ The `sobel` filter does not quite use identical weights to the unidirectional Sobel filters we will explore later, since `sobel` combines both vertical and horizontal results with `hypot` to compute gradient magnitude. Therefore, the BuildIt-staged convolutions and the AVX-512 `sobel` cannot be directly compared.

5.3.1 Applying the Procedure

We will use `geq` as a worked example of applying the specialization framework to a real program.

The specialization procedure of §4.4 applies almost mechanically here because the kernel is hot, and it is compute-bound rather than limited by anything external. The generality it pays for is easy to name: the interpreter must be able to evaluate any expression a user might supply, so it carries a tree walk and an operator dispatch on every node of every pixel. In practice, however, that the expression is fixed for the entire lifetime of the filter. It is supplied once at initialization, and it never changes.

We will exploit partial value specialization to accelerate the `geq` kernel. Concretely, we can treat the interpreter as accepting two inputs: the interpreted program source string, and the video data. At filter-initialization time, we can bind to the source string, which will not change for the rest of the execution, and specialize the control flow of the interpreter for the particular AST that the program source string generates. This specialization has a more common name. Specializing an interpreter with respect to a fixed program is the first Futamura projection, and its residual is a compiler: what comes out is the expression, compiled. In other words, specialization turns the `geq` expression evaluator from an interpreter to a JIT compiler.

5.3.2 Staging the Interpreter

Staging the interpreter is a matter of deciding which of its inputs are known first. The expression tree is first-stage, since it is fixed once the filtergraph is built; the pixel coordinates and sample values are second-stage, since they vary per pixel. Marking the types accordingly with `BuildIt` causes the tree walk to happen once, during generation, rather than per pixel. The traversal and the operator dispatch are evaluated away, and what remains is the arithmetic they were dispatching to.

Code Block 5.1 shows what that marking actually amounts to. The staged evaluator is the same recursive switch as the original, with the same cases in the same order, dispatching on the same node types and recursing on the same children. What changed is the type of the values flowing through it: the return type becomes `dyn_var<double>`, every temporary bound from a recursive call becomes a `dyn_var`, and the `const_values` and `var` pointers in the parser state become `dyn_var` pointers rather than plain ones. Nothing beyond that is a decision made by the programmer. The arithmetic written over those values is unchanged in source, and `BuildIt`'s operator overloads record it into the generated program instead of evaluating it, so the operator cases come across verbatim.

This is much less work than a typical JIT compiler takes to build. There is no code generator to write and no intermediate representation to design, because the generated program is produced as a side effect of running the evaluator once with staged types. The two files are 855 and 889 lines respectively, and the added lines are almost entirely the staged `libm` wrappers and the entry point that compiles and loads the resulting kernel; no function in the evaluator was restructured.

Original interpreter

```
static double
eval_expr(Parser *p, AVExpr *e)
{
    switch (e->type) {
    case e_value: return e->value;
    case e_const: return e->value *
        p->const_values[e->const_index]; /*
        ↪ double */
    case e_sqrt: return e->value *
        sqrt(eval_expr(p, e->param[0]));
    /* ... other unary cases ... */
    default: {
        double d =
            eval_expr(p, e->param[0]);
        double d2 =
            eval_expr(p, e->param[1]);
        switch (e->type) {
        case e_mul: return e->value * (d * d2);
        case e_pow: return e->value *
            pow(d, d2);
        case e_bitand: return
            isnan(d) ||
            isnan(d2) ? NAN :
            e->value * ((long int)d &
                (long int)d2);
        /* ... other binary cases ... */
        }
    }
    }
    return NAN;
}
```

Staged with BuildIt

```
static dyn_var<double>
eval_expr(Parser *p, AVExpr *e)
{
    switch (e->type) {
    case e_value: return e->value;
    case e_const: return e->value *
        p->const_values[e->const_index];
    case e_sqrt: return e->value *
        buildit_sqrt(eval_expr(p, e->param[0]));
    /* ... other unary cases ... */
    default: {
        dyn_var<double> d =
            eval_expr(p, e->param[0]);
        dyn_var<double> d2 =
            eval_expr(p, e->param[1]);
        switch (e->type) {
        case e_mul: return e->value * (d * d2);
        case e_pow: return e->value *
            buildit_pow(d, d2);
        case e_bitand: return
            buildit_isnan(d) ||
            buildit_isnan(d2) ? NAN :
            e->value * (cast_to<long int>(d) &
                cast_to<long int>(d2));
        /* ... other binary cases ... */
        }
    }
    }
    return NAN;
}
```

Code Block 5.1: The `geq` expression evaluator before and after staging, abridged and reflowed so that corresponding lines sit opposite one another. Read across a row: the dispatch structure, the recursion and the arithmetic are identical, and every difference is a change of type. `double` becomes `dyn_var<double>` on the return type and on the temporaries bound from recursive calls; in the parser state, not shown, `double *const_values` likewise becomes `dyn_var<const double *>`. The operator cases are then unchanged in source, because the overloads on `dyn_var` record the operation rather than performing it. Only values that escape the type system need a real edit, and the last case shows both kinds: a C cast must be written `builder::cast_to`, and calls into `libm` must go through staged wrappers so that they emit a call in the generated program rather than folding to a constant. Namespace qualifiers, the C++ scoping of the enumerators, and the `avpriv_` prefix on the wrapper names are elided for visibility in this snippet.

The output of the BuildIt-staged interpreter is straight-line C code. There is no tree, no dispatch, and no indirection through operand pointers. The generated second-stage code contains only the operations the expression actually names, on values held in registers.

Code Block 5.2 is an instance of that, produced by running the filter rather than reconstructed by hand. The invocation asks for a brightness ramp on the luma plane, `lum='p(X,Y)*1.5-30'`, and the listing is the file BuildIt writes out and hands to the C compiler during filter initialization. It contains no reference to an expression tree, no `switch`, and no function pointer. The tap into the source plane, the two arithmetic operations the expression names, and nothing else survive in the loop body; the tree walk and the dispatch that reached each of those operations happened once, at generation time.

5.3.3 Results

Figure 5.1 gives the steady-state speedup of each filter over its stock counterpart, at both 1080p and 2160p. Every filtered output was checked against the stock filter's output with an Adler-32 hash over the decoded frames, and all of them match exactly, so these are like-for-like comparisons rather than the result of computing something cheaper.

Note these are filter-only figures, meaning they exclude various fixed overheads associated with the `ffmpeg` harness. A timed `ffmpeg` invocation includes the whole pipeline in its final runtime: demuxing, decoding, filtering and hashing. Both implementations pay the first, second and fourth identically, so a ratio taken from raw wall times is diluted by whatever share of the run was not filtering. To remove it, a null filter was timed at the same resolutions and durations, giving the cost of everything except filtering directly; subtracting its fitted rate leaves the filter's own. The correction is particularly impactful at high resolutions: for the 3×3 sobel at 2160p, 70% of the stock filter's measured cost is decoding, so the uncorrected figure understates the filter by a factor of two and a half. For the `geq` expressions it is worth one or two percent. Both sets of numbers are tabulated in Appendix B.

The `geq` results range from $2.7\times$ to $11.6\times$ at 2160p, depending on the complexity of the expression. The identity expression gains $2.7\times$ while doing no arithmetic at all, which is a direct measurement of interpreter overhead: removing the tree walk alone nearly triples throughput. Richer expressions gain more, up to $11.6\times$ for one combining a sine, a clip and several multiplies. In general, more complex expressions increase the number of operations (which is equally expensive for both implementations), but also increase the AST tree depth, which is extremely expensive for the interpreter and free for the compiler. However, the ordering is not purely increasing in complexity. The heaviest expression tested, which adds a squared term, two transcendentals and a hypotenuse, gains only $6.9\times$, less than the lighter `planeosc`. The reason is that the win is a ratio: specialization removes dispatch, so its benefit is largest when dispatch is the largest share of the work, and an expression that does more real arithmetic per pixel dilutes it.

The `convolution` results range from $3.4\times$ to $9.5\times$ at the same resolution, and the spread across them is explained almost entirely by which baseline each one faces. The 3×3 sobel gains least, at $3.4\times$, and it is the only configuration measured against hand-written vector assembly.

```

ffmpeg -i in.mkv -vf "buildit_geq=lum='p(X,Y)*1.5-30'" out.mkv
double execute (void* arg0, const double* arg1, double* arg2) {
    const double* var0 = arg1;
    void* var1 = arg0;
    double* var2 = arg2;
    double var3 = 1.0 * var0[1];           /* Y */
    double var4 = 1.0 * var0[0];           /* X */
    void* var5 = var1;
    double var6 = var3;
    double var7 = var4;
    void* var8 = var5;
    int var9;
    int var10;
    BuildItGEQContext* var11 = (BuildItGEQContext*)var8;
    AVFrame* var12 = var11->picref;
    const unsigned char* var13 = var12->data[0];
    int var14 = var12->linesize[0];
    int var15 = var12->width;
    int var16 = var12->height;
    if (!var13) {
        /* ... the same expression again, over a zero pixel ... */
    }
    int var25;
    int var26;
    var9 = var7 = ff_clipd(var7, 0, var15 - 1);
    var10 = var6 = ff_clipd(var6, 0, var16 - 1);
    var25 = ff_min(var9 + 1, var15 - 1);
    var26 = ff_min(var10 + 1, var16 - 1);
    var7 = var7 - var9;
    var6 = var6 - var10;
    /* p(X,Y) */
    double var27 =
        (1 - var6) * ((1 - var7) * var13[var9 + var10 * var14]
            + var7 * var13[var25 + var10 * var14])
        + var6 * ((1 - var7) * var13[var9 + var26 * var14]
            + var7 * var13[var25 + var26 * var14]);
    double var28 = 1.0 * var27;
    double var29 = 1.5;
    double var30 = 1.0;
    var30 = var30 / 0.0;
    double var31 = 1.0 * (var28 * var29); /* p(X,Y)*1.5 */
    double var32 = -30.0;
    double var33 = 1.0;
    var33 = var33 / 0.0;
    double var34 = 1.0 * (var31 + var32); /* p(X,Y)*1.5-30 */
    return var34;
}

```

Code Block 5.2: The invocation, and the second-stage program the staged evaluator emits for it. This is the generator’s own output, including its numbered temporaries and its argument-copy prologue; only the null-plane fallback branch is elided, and it is a specialized copy of the same expression evaluated over a zero pixel. **var27** is the bilinear tap that **p(X,Y)** performs, with the frame geometry read from the context rather than from the expression; **var31** and **var34** are the multiply and the add that the expression names. Two features are artifacts of the generator rather than work. The `1.0 *` factors are `e->value`, the per-node sign and coefficient that the parser folds into each node; they are first-stage constants, so they arrive as literals and the second-stage compiler removes the multiply. The `var30 = 1.0; var30 = var30 / 0.0;` pairs are the infinity temporary that the binary-operator case of Code Block 5.1 declares for the division and modulo paths, which are not taken here, so it is dead on arrival. These artifacts are easy for the second-stage compiler to eliminate. Critically, the final expression includes no expensive AST walking logic; the expression tree has been fully evaluated and emitted into the second-stage code. Every operator that the interpreter would have dispatched on has become a line of arithmetic.

The 21-tap Gaussian gains $7.8\times$ and the 7×7 LoG $9.5\times$, and both are measured against scalar C, because FFmpeg’s assembly covers only the 3×3 square case. The convolution numbers are less tightly bounded than the `geq` ones for the same reason the timing correction above was necessary. The 3×3 sobel is cheap enough that most of what a naive measurement records is the decoder rather than the filter, so it is the least determined entry in the set.

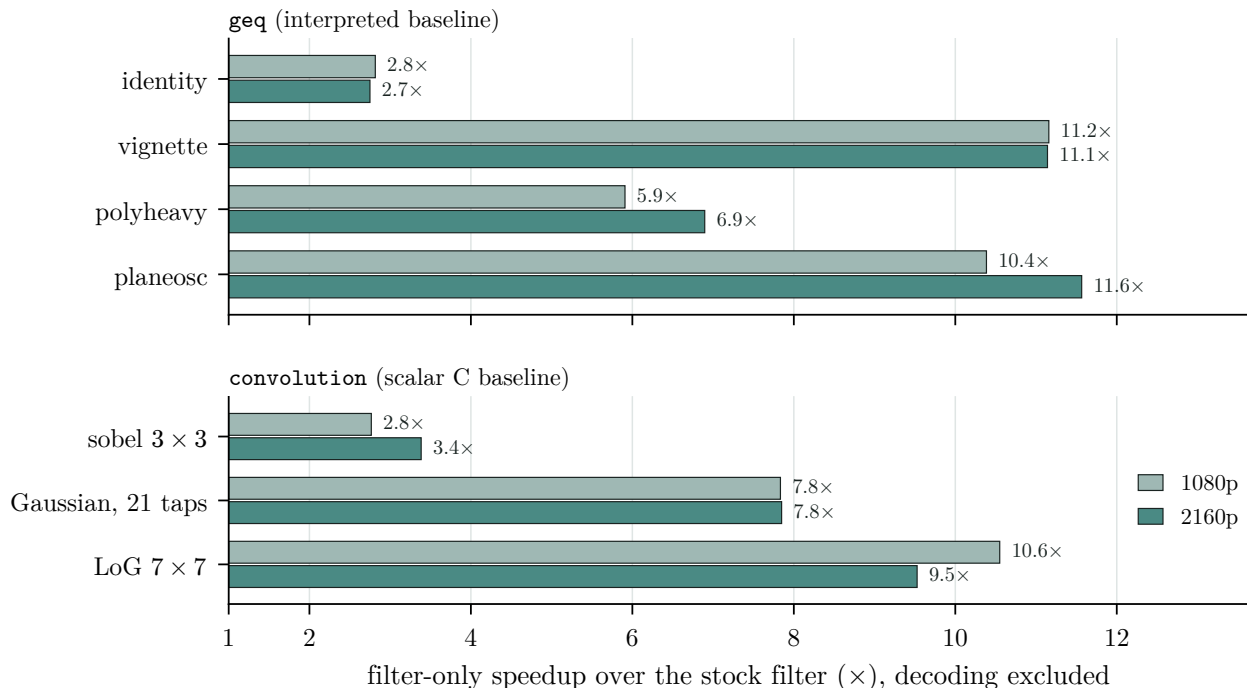


Figure 5.1: Steady-state filter-only speedup of the BuildIt filters over their stock counterparts, at two resolutions, on an EC2 `m8azn.xlarge`. Decoding, demuxing and hashing are removed by subtracting a null-filter baseline measured at the same resolutions and durations, so these describe the filter alone; the uncorrected end-to-end figures are in Appendix B. Steady state additionally excludes the one-time cost of generating and compiling the kernel, which is reported separately in Figure 5.2. Every output was verified bit-identical to the stock filter’s. The `geq` gains scale with how much of the stock filter’s work was interpreter dispatch rather than arithmetic; the `convolution` gains are set mainly by whether FFmpeg ships vector assembly for that configuration, which it does only for the 3×3 square case.

It is important to note that these figures are steady-state results, and they amortize the cost of generating and compiling a kernel at runtime during the filter initialization. Running time is linear in clip length for both implementations, so each is characterized by its fixed cost and steady-state throughput; specialized kernels have higher throughput at the price of a higher fixed cost. That fixed cost is between 0.06 and 0.63 seconds for every filter here, covering the whole of parsing, generating, compiling and loading the specialized kernel. It scales with the size of the expression rather than with the size of the frame, as it should: the two heaviest `geq` expressions cost 0.3 to 0.6 seconds to compile while the 3×3 sobel costs 0.06, and the fixed cost is independent of resolution.

In practice, this fixed cost dampens the speedup that an actual user sees, which is what Figure 5.2 plots. Because the fixed cost is spread over the whole stream while the saving accrues per frame, the observed speedup starts low and climbs toward the steady-state value as the clip lengthens. Because video is so dense, however, the specialized kernels become worthwhile very quickly. Every `geq` expression is already ahead on a one-second clip, and the break-even point computed from the fits is under a fifth of a second for most of them. The convolutions are slower to break even because they have less to gain in the first place. Filtergraphs are normally run over whole files, so in realistic use the crossover is reached before the output has begun to accumulate, and the steady-state numbers are the ones that matter. Where they would not be is short-clip work: batch thumbnailing, or a preview pass over a few frames, where a filter as cheap as sobel never earns its compile back.

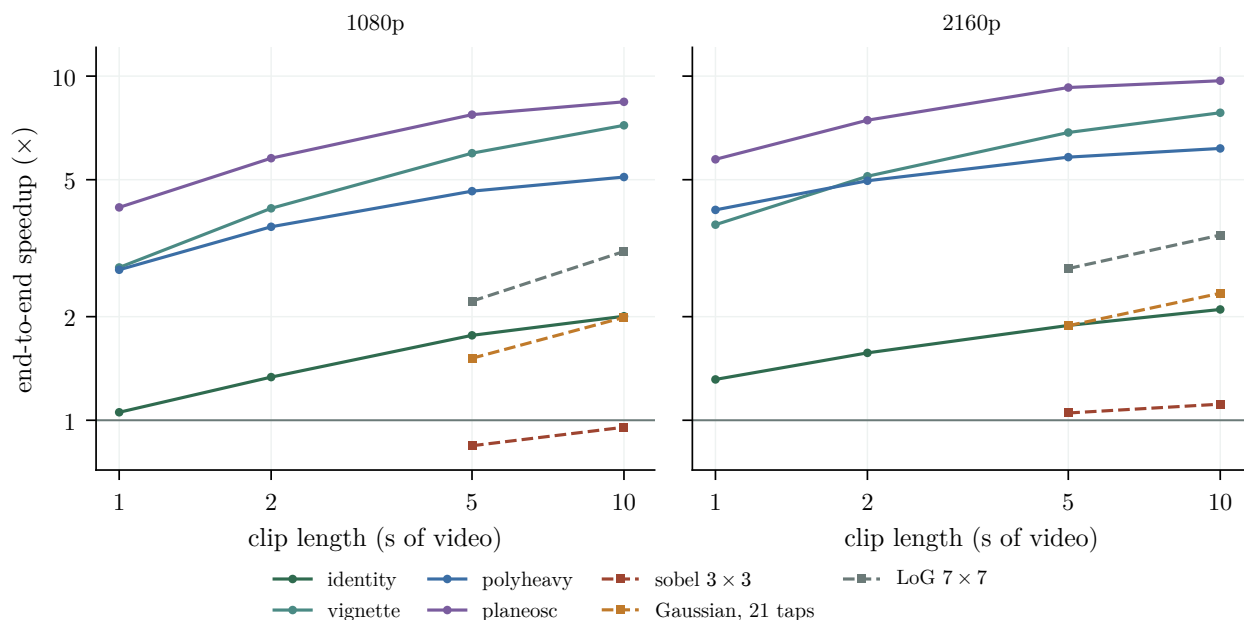


Figure 5.2: Observed end-to-end speedup against clip length, at both resolutions, on log axes. The one-time cost of generating and compiling a kernel is a fixed charge spread over the whole stream, so each curve starts below its steady-state value and climbs toward it; the horizontal line is parity. Solid lines are `geq`, dashed are `convolution`. Only the 3×3 sobel, which has the least to gain, is still near parity at ten seconds of 1080p.

5.4 Limitations

This chapter establishes that the procedure of Chapter 4 can be followed mechanically on real code, that the resulting filters are drop-in replacements rather than research artifacts, and that the runtime cost of specializing is recovered almost immediately in the setting where these filters are actually used. However, it does not establish the harder claim that specialization yields speedups even in heavily-optimized code. Both baselines here

were relatively unoptimized: one was an AST interpreter, and the other was scalar C that happened to defeat the compiler’s vectorizer. Neither had received the sustained attention from performance engineers that would make such a speedup surprising, and the large speedup numbers we attained reflect that fact.

In the next chapter, we will demonstrate the benefits of the specialization framework for performance engineering as applied to VMAF, where the baseline is much more heavily optimized.

Chapter 6

Case Study: VMAF

We will now move to a much harder target: `libvmaf`. The central thesis question revolves not around whether specialization can accelerate underoptimized code, as we saw in Chapter 5, but rather whether it can still provide additional benefits in expert-tuned code. Therefore, a positive result here will demonstrate that specialization is a generalizable performance engineering framework, rather than a niche technique that only works in toy examples. The chapter will be structured around the specialization axes defined in Chapter 4, with each section focusing on a specific feature of `libvmaf` and the corresponding specialization applied to it.

6.1 `libvmaf` as an Adversarial Baseline

Video Multimethod Assessment Fusion (VMAF) is a state-of-the-art, open-source video quality assessment program developed at Netflix that evaluates the perceptual degradation between a pristine “reference” video and a compressed “distorted” video much more accurately than traditional methods [22, 23]. The need for such a program arises from a fundamental limitation of digital video: it is too dense to work with losslessly at scale. An uncompressed video signal carries an extraordinary amount of information, nearly 12 Gb/s for a standard 4K video stream at 60 frames per second. Lossless compression techniques can bring this down by a factor of 3 to 5, but lossy encoding is generally necessary to make a high-quality video streamable in real time over a typical internet connection. Most video codecs exploit the fact that video is typically sparse in the frequency domain, and it is often possible to heavily quantize the higher-frequency components of a video signal to save data with little visible quality loss [28]. However, the relationship between the specific quantization decisions made by a video encoder and the final effect on perceptual quality is complex, and it is not well-modeled by simple signal-processing metrics for signal degradation such as signal-to-noise ratio. VMAF bridges this gap by considering the behavior of the human visual system when producing its scores. It combines several expert-derived psychovisual video features into a single score using machine learning techniques. As a result, VMAF scores are well-correlated with human evaluations of how a lossy video looks versus its lossless source [23]. As such,

VMAF is widely used in a feedback loop with a video encoder to help select parameters that minimize quality loss while maximizing compression.

The `libvmaf` project, which is the C implementation of VMAF meant for production environments, implements a harness that can run multiple different VMAF models. To compute a score, the library runs the necessary feature extractors defined by the model configuration, then combines the feature scores through a machine learning model whose parameters are also model-specific. In practice, feature extraction dominates the total runtime.

Because VMAF is the industry standard for perceptual video quality evaluation, it is heavily utilized and has been the target of immense performance engineering effort over several years. `libvmaf` includes over fifteen thousand lines of hand-written SIMD intrinsic code, nearly 40% of the project’s total source code, to accelerate its most expensive feature extractors. These kernels are already well-optimized, particularly for x86 platforms, and they have strict correctness requirements: every SIMD integer feature extractor produces bit-identical output to the scalar implementation, and the floating-point SIMD extractors produce output that is identical to the scalar implementation up to floating-point reassociation.

There are two model families that are used in production: the v0 and v1 models. The v0 family is older, but more stable and is still the default in the latest version of `libvmaf`. It uses exclusively the integer VIF, ADM, and motion features. The v1 family was released in June 2026 and is Netflix’s newest official recommendation [22]. It keeps the integer ADM and motion from v0, but swaps VIF for the floating-point CAMBI and SpEED-QA features. Because v1 is newer, the vectorized feature extractors used by v1 have received less attention from the open-source community thus far; however, the original implementations by Netflix are still well-optimized and several times faster than their scalar counterparts.

The extensive engineering effort that has already been invested into optimizing `libvmaf` makes it a challenging target for specialization-centric performance engineering. Any speedup achieved here will be a strong demonstration of the effectiveness of the specialization framework. In the following sections, we will survey the various new kernels and patches I wrote for the feature extractors in `libvmaf`. The high-level features, and the axes of specialization I used to exploit those features, are listed in Table 6.1.

Note that the discussion of the speedup actually achieved by each patch is largely qualitative in this section. We will quantify the end-to-end and per-kernel performance improvements for the complete patch set in Chapter 7.

6.2 VIF

We will first look at the Visual Information Fidelity (VIF) feature, which is the most expensive feature modified in the patch. VIF is a standalone image quality metric, and VMAF uses it as a partial signal of overall visual quality [29]. The main VIF kernel takes up approximately 60% of the overall runtime of the v0 model on a typical video clip, and the VIF feature as a whole around 70%. Because it dominates runtime so heavily, it already started as one of the most well-tuned kernels in all of VMAF.

Table 6.1: Axes of specialization the patch set exercises for each feature. Two stars mark an axis the optimization work leans on heavily, one star an axis it touches, and a blank an axis it does not use. The ratings describe this patch set rather than the baseline, which is already thoroughly specialized for its environment throughout.

Feature	Shape	Value range/domain	Algebraic	Environmental
VIF (integer)		*	**	**
CAMBI		**		**
Motion				**
ADM		**	**	*
Float convolution	**		*	*

The `vif_statistic_8` and `vif_statistic_16` routines implement the same algorithm, the former for 8-bit video and the latter for bit depths up to 16. The algorithm is effectively 2D integer convolution by a symmetric, rank-1 kernel whose weights are fixed at build-time, followed by some nonlinear thresholding and global accumulation. The algorithm runs at four scales on each frame: one base scale and three higher levels of subsampling. It always uses `vif_statistic_16` for higher scales (even in 8-bit video). However, because each dimension of the image is halved between each scale, the lowest scale dominates the total runtime; therefore, `vif_statistic_8` matters only for 8-bit video, while `vif_statistic_16` matters slightly for 8-bit video and heavily for 9–16 bit video.

The existing implementation of the VIF kernels already specializes for several properties of the problem. They were already hand-vectorized with AVX-512 intrinsics, which is a form of environmental specialization. They also apply three forms of algebraic specialization. The kernels exploit the fact that the kernel is rank-1 to decompose the convolution into two 1D convolutions. They also use the kernel’s symmetry to halve the multiplies per pixel by adding symmetric taps before multiplying (an example we discussed in §4.2). Finally, they make use of the fact that the convolution filter weights sum to 65536 to avoid normalizing by division, and instead normalize with a cheap bit-shift.¹ The fact that one of the best-optimized kernels in VMAF is also one of the most specialized is evidence of specialization raising the throughput ceiling in real-world software. Nonetheless, there were still some areas in which the VIF kernels were not fully optimized.

6.2.1 Cheaper widening and narrowing

There were a few locations where traditional, localized optimizations were able to bring a modest speedup. Most notably, both the 8-bit and 16-bit kernels frequently needed to compute

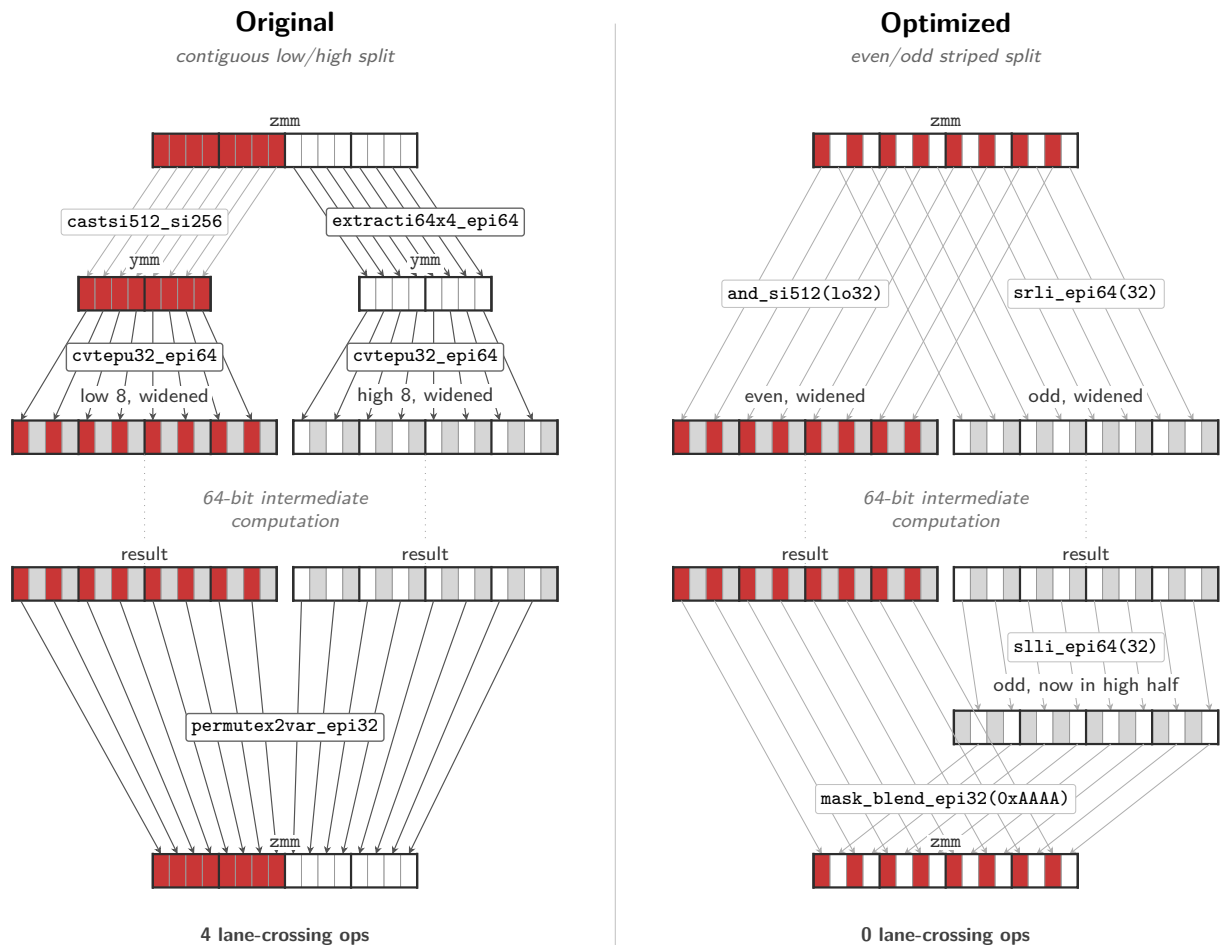
¹ Since integer VIF is the quantized version of the original floating-point VIF feature, it’s likely that the filter weights were intentionally quantized such that they would sum to 65536 to make the normalization step cheap. This lossy transformation is an example of modifying the algorithm to better suit the hardware. However I did not attempt any lossy conversions in the patch set to isolate the impact of specialization alone on performance.

64-bit products of 32-bit elements; to do so, they would load 8 32-bit elements into a YMM register, zero-extend into 64-bit elements in a ZMM register with `_mm512_cvtepu32_epi64`, then run a full-width 32-bit vector product using `_mm512_mul_epu32`. However, it is possible to avoid the conversion operations by restructuring the problem. Since `_mm512_mul_epu32` ignores the high 32-bit value in every 64-bit lane, we can instead load 16 32-bit elements into ZMM and directly call `_mm512_mul_epu32` on this register; the output will have the full-width products for *even* elements in the original register. We can repeat the process after rotating the original ZMM register by 32 bits (`_mm512_shuffle_epi32`) to move the odd elements into the even elements' position; now `_mm512_mul_epu32` will produce full-width products for the *odd* elements. The only work to do afterwards is to repack the striped elements into two contiguous registers using a permutation after accumulating the full-width products. However, since the original implementation already included a `_mm512_permutex2var_epi32` at the end to efficiently compact two ZMMs of 64-bit values into a single ZMM of 32-bit values, I was able to fold a second permutation into the same permutation mask to undo the striping for free. This optimization does not reduce the number of multiplies per element, but it swaps two expensive conversions for a single cheap in-lane shuffle. Especially on older microarchitectures where cross-lane data movement is particularly expensive, this change brings a significant speedup, as we will see in Chapter 7.

I was able to apply similar optimizations in other areas where a widening or narrowing conversion was used to combine elements. For example, the kernels often need to expand 32-bit values to 64-bit for increased precision during intermediate calculations, before reducing back to 32-bit again later. The original implementation used `_mm512_extracti64x4_epi64` to split a ZMM register into two YMMs, then `_mm512_cvtepu32_epi64` to convert each 8-wide 32-bit YMM into an 8-wide 64-bit ZMM: one with the low 8 values of the original register, and the other with the high 8 values. Then to combine the results again, it would use a 2-way permutation like `_mm512_permutex2var_epi64` or narrowing operations and an insertion like `_mm512_inserti64x4_epi64`. This is four to six expensive permutations and shuffles per round-trip. By applying the same striping trick, we can replace the low/high split with a much cheaper even/odd split. We convert the original ZMM into two ZMMs with $2\times$ wider elements by masking off the high 32 bits of each 64-bit lane (`_mm512_and_si512` with a constant mask) to get the even elements, and again right-shifting by 32 (`_mm512_srli_epi64`) to get the odd elements.² Merging these two back into a single register at the original 32-bit precision is now also much simpler: after running our intermediate computations in the two ZMM registers and bringing our result into the low 32 bits of each 64-bit element, we shift up the odd-element register by 32, then apply a blend with a striped mask. This interleaves the low 32 bits of each 64-bit block from the even register with the high 32 bits from the odd register, which effectively deinterleaves the result entirely. Figure 6.1 compares the two round-trips side by side.

Although this optimization falls under the realm of traditional performance engineering, it is also a kind of environmental specialization that is often overlooked: taking advantage of the

² This technique works for unsigned widening operations. For signed widening operations, we left-shift by 32 and then arithmetic right-shift by 32 to get the even elements, and simply convert the logical right shift to arithmetic for the odd elements.



Cell width is proportional to element width, so one 64-bit element is as wide as two 32-bit ones and every register drawn is 512 bits except the two 256-bit ymm. Heavy rules are 128-bit lane boundaries, and the grey half of a widened lane is the half holding no data. Dark arrows and outlined labels mark the lane-crossing operations; where a bundle fans out or in, the element pitch is changing and the data is being spread across lanes.

Figure 6.1: Two ways to widen a register of 32-bit elements to 64 bits, compute on it, and narrow it back. Cell width is proportional to element width, so red and white track where each element travels, and the heavy rules mark 128-bit lane boundaries. The original split (left) is contiguous, so both halves have to be carried across lane boundaries on the way out and back again: an extract, two widening conversions, and a final two-way permutation. The striped split (right) reaches the same 64-bit form using only a mask and a shift, both of which stay inside their lane, and rejoins with a shift and a blend. The two versions perform the same amount of work for the inner calculations; the striped one simply never performs any shuffles when converting in and out of the intermediate representation.

cost model of a specific SIMD architecture. SIMD programs often use the abstraction that a SIMD register is simply an array of elements, but different architectures make different decisions about how to actually implement that interface. The AVX family is built around the idea of distinct, 128-bit lanes that are cheap to move data within, but expensive to move data between. A 512-bit ZMM register is really four of these lanes packed together. Widening conversions such as `_mm512_cvtepu32_epi64` cross lanes and are thus generally slower than simple per-element shifts. Moreover, by thinking about AVX registers as packs of 128-bit lanes rather than as arrays of elements makes it easier to reason about tricks such as the 32-bit right-shift to move odd elements into even positions. Even though shifting or permuting array of 32-bit values would intuitively be expensive, shuffling data inside an individual SIMD unit is intuitively cheap. Reinterpreting bits in a manner that only makes sense under the specific lane model of AVX is a common theme throughout many of the optimizations in this patch series.

6.2.2 Narrowed multipliers

The more interesting of the two optimizations I made to VIF was one that relied much more heavily on the specialization framework. All filter taps in VIF at every scale are stored as 16-bit unsigned integers. The 8-bit kernel converts its pixels into 16-bit and interleaves pixels belonging to symmetric coefficients, then uses a single 16-bit multiply with 32-bit horizontal-accumulate (`_mm512_madd_epi16`) to accumulate the partial products of the convolution. However, the 16-bit kernel uses two separate 16-bit multipliers to extract the low and high bits of the full-width product (`_mm512_mullo_epi16/_mm512_mulhi_epi16`), interleaved them (`_mm512_unpacklo_epi16/_mm512_unpackhi_epi16`), and did not process symmetric taps together, leading to an extra `_mm512_add_epi32` for every two rows of the filter. In effect, the 16-bit implementation uses 5 instructions to do the same work as the 8-bit kernel, even though in both cases the filter taps and pixel values both fit into 16 bits.

This was not an oversight in the original implementation. The `_mm512_madd_epi16` (MADD) intrinsic specifically supports *signed* 16-bit multiplication and accumulation. Because all the filter taps are less than 32768, they fit into a signed 16-bit integer (even though the C datatype of the filter taps is `uint16_t`). However, the API contract of `vif_statistic_16` requires that it be able to support image planes with up to 16 unsigned bits per pixel. The VIF statistic kernel would produce garbage scores on 16-bit videos if MADD were used in place of the manual multiplications and interleaving logic because the unsigned pixels would overflow the input range of the MADD intrinsic.

There is an obvious opportunity here for value-range specialization. The vast majority of high bit depth videos are 10-bit or at most 12-bit, since quantization effects such as banding are already practically imperceptible with 12 bits per channel. In fact, many mainstream video codecs do not even support 16-bit video: H.264 is limited to 14-bit and AV1 to 12-bit [30, 31]. We could therefore create an alternative `vif_statistic_15` kernel that only allows 15 bits per channel, and dispatch to it instead of `vif_statistic_16` for all bit depths between 8 and 16. 15-bit unsigned pixel values would be guaranteed to fit within a 16-bit signed integer; therefore, `vif_statistic_15` would be able to replace 5 instructions with a single

MADD in the hot loop of the VIF convolution, which should significantly improve the kernel’s throughput over `vif_statistic_16`. Because videos are rarely 16-bit in practice, almost all high bit depth videos would go to the faster `vif_statistic_15` kernel, and we would be able to improve performance for the common case without sacrificing correctness for 16-bit video in a slightly slower fallback path.

This optimization worked well and provided a significant speedup, but had one major shortcoming. Although video inputs are rarely ever 16-bit, VIF internally runs all its higher scales with 16 unsigned bits of precision, regardless of the initial bit depth of the video, to avoid compounding quantization errors when downsampling. Therefore, while `vif_statistic_15` is usable for the first scale in high bit depth videos, it is unusable for scales 2–4 for input of any bit depth. The optimization is still helpful for 10-bit videos, since the first VIF scale takes up a greater share of runtime than scales 2–4 combined, but we would be able to achieve a greater end-to-end speedup by adapting the approach to work for 16-bit unsigned values.

I was ultimately able to use an algebraic specialization to efficiently reformulate the algorithm such that we could continue to use MADD-based filter tap accumulation on unsigned 16-bit values. The key was to convert unsigned 16-bit values into signed values by subtracting 32768 from each pixel value and treat the discrepancy between the computed values and the true partial products as an error term. I then made use of the algebraic properties of the filter to compute the error term in a single step outside of the hot loop such that it could be corrected in a single step, independently of the size or particular filter taps within the VIF filter (which change at each scale).

The VIF kernel computes the convolution of its filter against the reference image, distorted image, pixel-wise squares of each image, and the pixel-wise product of the two images. We first consider the case of the first-order images, which are not squared or multiplied. If the traditional summation for a convolution operation is of the form $S_x = \sum_{i=1}^k x_i f_i$, where x_i are pixels and f_i are corresponding pixel taps, then subtracting 32768 from each pixel means we compute

$$\begin{aligned} S'_x &= \sum_{i=1}^k (x_i - 32768) f_i \\ &= \left(\sum_{i=1}^k x_i f_i \right) - \left(\sum_{i=1}^k 32768 f_i \right) \\ S'_x &= S_x - 32768 \sum_{i=1}^k f_i \end{aligned}$$

Because the filter weights are known at build time to sum to 65536, we can compute the error term $32768 \sum_{i=1}^k f_i = 32768 \cdot 65536 = 2^{31}$ statically. Thus we can add 2^{31} to the final result of the convolution with the range-shifted pixel values to correct the computed value. Introducing only a single constant addition to the end of the kernel would be very cheap and would have effectively no effect on end-to-end throughput. However, the convolution already adds a constant rounding term to the final output before right-shifting for normalization.

By folding the correction term and the rounding term into the same constant vector, the correction becomes completely free.

For the convolutions with image products, the error term is more complex. Let x_i be the first image and y_i be the second image; we wish to compute $S_{xy} = \sum_{i=1}^k x_i y_i f_i$. If we subtract 32768 from all pixel values, we instead compute

$$\begin{aligned}
S'_{xy} &= \sum_{i=1}^k (x_i - 32768)(y_i - 32768)f_i \\
&= \sum_{i=1}^k (x_i y_i - 32768x_i - 32768y_i + 32768 \cdot 32768)f_i \\
&= \left(\sum_{i=1}^k x_i y_i f_i \right) - 32768 \left(\sum_{i=1}^k x_i f_i \right) - 32768 \left(\sum_{i=1}^k y_i f_i \right) + 32768 \cdot 32768 \cdot \sum_{i=1}^k f_i \\
&= S_{xy} - 32768S_x - 32768S_y + 2^{15} \cdot 2^{15} \cdot 2^{16}
\end{aligned}$$

$$S'_{xy} = S_{xy} - 32768S_x - 32768S_y + 2^{46}$$

The correction term is now dependent on the first-order sums S_x and S_y , which are already computed in the kernel. Therefore, the correction term costs only two bit shifts and two additions (repeating the trick from the first-order image convolutions of combining the constant 2^{46} with the existing rounding term).

In total, the cost of all the corrections is only 6 shifts and 6 additions per convolution, plus 1 subtraction per filter tap.³ The ability to use MADD saves 4 ALU operations per filter tap; in aggregate, this is still a substantial speedup over the original `vif_statistic_16` implementation. In fact, because the additional arithmetic operations needed for this trick make use of underutilized vector execution ports, performance for this version of `vif_statistic_16` was nearly identical to that of `vif_statistic_15`. Because performance was so similar, I dropped the additional `vif_statistic_15` and used the optimized `vif_statistic_16` unilaterally instead.

This optimization depended heavily on several properties of the problem and the environment: that MADD is the cheapest way to get our result but only supports signed integers, that the sum of the filter taps was 65536 with none exceeding 32768, and that we needed convolutions for both the first-order and second-order image polynomials. It is a fortunate property of the algebraic structure of the problem that we are able to apply the correction factor to the second-order image convolutions nearly for free; the residual terms happen to be data we have already computed. The final speedup can be attributed largely to environmental specialization (i.e. selecting more optimal AVX-512 instructions for our algorithm), but it would not have been possible without algebraic specialization. The correction was efficient

³ In the actual implementation, I needed to additionally work around a quirk of `_mm512_madd_epi16`: when two adjacent operands are -32768 in corresponding lanes of both inputs, the output underflows to -2^{31} instead of the true value of 2^{31} . Working around this costs an extra 3 subtractions per 2 taps to reverse the underflow, and the constant correction term has to be updated accordingly. This has effectively no impact on throughput.

because it exploited the structure of the problem in a manner that would not have been possible without working through the numerical edge cases by hand, and would certainly not have been possible for arbitrary filter weights.

I initially identified that `vif_statistic_16` was a candidate for value-range specialization: it would often be called with fewer than 16 bits per channel, but needed to conservatively support the maximum range and paid a runtime cost even on smaller inputs as a consequence. Although my final implementation did not make use of this specialization, it was a necessary intermediate step in reaching the final optimization. More generally, searching for the unspecialized components of a program can guide the process of discovery for a wider class of optimization opportunities, some of which may not neatly map into any single bucket in the specialization framework.

6.3 CAMBI

The Content-Aware Multiscale Banding Index (CAMBI) is used to detect banding artifacts in a video frame. Banding is a phenomenon where quantization levels become visible in an image, often because a gradual color gradient turns into a series of discrete steps. CAMBI computes the number of unique pixel values in a local neighborhood at multiple scales, and uses this information to determine whether banding is likely to be visible [32]. Because it performs many spatially localized computations, many of the kernels in CAMBI involve sliding windows, and the feature in general is quite expensive in VMAF v1.

6.3.1 AVX-512 vectorization

Because the v1 models were only recently released, and CAMBI was not used in prior versions of VMAF, its kernels are not as thoroughly optimized as those in VIF. Notably, CAMBI did not have any AVX-512 kernels at all; the only existing kernels were AVX2.

I hand-vectorized two functions that originally had only a scalar implementation, and added implementations to both the AVX2 and AVX-512 codepaths. The first is `compute_dp_row`, which performs a straightforward prefix sum; we will discuss this kernel in more detail in §6.3.5. The second is the anti-dithering filter, which runs a 2×2 box blur in-place to reduce noise jitters that can throw off the banding detection. I chose these particular functions to vectorize after performance profiling revealed they were taking up a significant portion of total program runtime.

My vectorized anti-dithering filter was a simple translation of the scalar code to AVX2/AVX-512. Because the scalar implementation wrote in-place, the compiler was unable to prove that the reads and writes would not alias, and therefore failed to autovectorize the loop at all. However, it is easy to see that the box blur only overwrites a particular pixel value after the last time it ever reads that pixel, since the source pixel is at the upper left corner of the 2×2 window of the convolution. By simply extending the logic to be 32-wide, and processing four rows at a time to improve cache reuse across rows, I was able to achieve a massive speedup

on this kernel. This kernel provided one of the biggest speedups in the patch series, as we will see in Chapter 7. This was one of the few “easy wins” I found in VMAF.

I also transliterated three kernels from the AVX2 source to AVX-512 with practically no change except for widening the vector width: the derivative kernel, the histogram range update kernel, and the decimation kernel. The derivative kernel computes whether each pixel equals its horizontal and vertical neighbors and produces a boolean derivative, whose 2D integral is later used to estimate banding within a local rectangle. The histogram range updater simply increments or decrements all the elements within a particular range of a 16-bit integer array. The decimation kernel performs a 2×2 nearest-neighbor image downsample in-place.

These three ported kernels showed mixed results relative to their AVX2 baselines. The speedup relative to AVX2 was highly sensitive to microarchitecture. Although some configurations became much faster with the wider vector width, others were actually slower than their AVX2 counterparts (refer to Table 7.2). It is likely that the kernels that became slower were already memory-bound, and the increased vector width of AVX-512 only led to increased cache-line splitting and no increase in actual throughput. This result is our strongest example that naively increasing parallelism does not always improve performance; beyond a certain point, specializing for properties of the algorithm or its inputs is the only reliable way to achieve a meaningful speedup.

Although we could describe all of these optimizations as environmental specialization, in truth they would have been discoverable even without the specialization framework because vectorization is one of the most well-known techniques for optimizing a software program. The takeaway from these results, then, is that traditional performance engineering techniques are still important to maximize performance independently of the specialization framework. In this case, it was only possible to find low-hanging fruit because CAMBI is a relatively new feature for which not all the obvious performance bottlenecks have already been eliminated. Once they are, it becomes much more difficult to achieve further speedups with the standard approaches to performance engineering; that is when specialization proves most powerful.

6.3.2 C-values optimization

There were two additional AVX2 kernels that I improved upon with traditional vectorization techniques. One was `calculate_c_values`, which computes certain pixel histograms that are used for determining whether banding is likely to be visible in a local neighborhood. The original AVX2 implementation updated the histograms using the aforementioned vectorized histogram update kernels, but the branch-heavy control logic that decided which pixels needed a histogram update was entirely scalar. I vectorized this logic by swapping the branches for masks and bitwise arithmetic, which is much more feasible in AVX-512 than AVX2 due to the dedicated mask registers and comparison operations that write directly to those registers. I used a single scalar loop at the end to pop off all the active bits from the final mask and execute the histogram updater kernels for pixels corresponding to only active bits. This change led to a sizable speedup over the original implementation, much more so than the naive widening to AVX-512 I began with.

Although this is a straightforward vectorization speedup, it makes use of features unique to AVX-512 that would not have been visible with a direct transliteration as for the kernels in §6.3.1. It is a better example of environmental specialization than those other kernels because it leverages specific properties of the underlying architecture to achieve a meaningful speedup.

6.3.3 mode3 load reduction

The other AVX2 kernel I improved was the `mode3` filter, which returns the mode of three adjacent 16-bit values, or their minimum when all three differ. It is applied first horizontally and then vertically to the source image, and its output feeds `calculate_c_values`. The filter denoises banding boundaries, where pixel values that differ only slightly make genuine banding harder to distinguish computationally.

The first change was algebraic. Written directly, the selection is three-way: return `a` if it matches either of the others, else `b` if it matches `c`, else the minimum of all three. The AVX2 kernel implements exactly that, with three equality comparisons, two minima and a cascade of two blends. Observing that the answer is always either $\min(a, b)$ or `c` collapses the selection to a single two-way blend, because $\max(a, b) = c$ is by itself enough to detect a duplicate involving `c`, and the remaining case is settled by comparing $\min(a, b)$ against `c`. The kernel therefore computes $\min(a, b)$ and $\max(a, b)$ once and selects between $\min(a, b)$ and `c` under the predicate $\max(a, b) = c \vee (a \neq b \wedge \min(a, b) > c)$ (picking `c` if true and $\min(a, b)$ if false). The rewrite is worth making only because the predicate is free to build: as in §6.3.2, comparisons that cost a full vector register and vector-width boolean arithmetic under AVX2 become mask registers and single-cycle mask operations under AVX-512, so a compound condition of three comparisons occupies no vector registers at all.

The second change reduces load port pressure using `_mm512_alignr_epi32` (ALIGNR), which has no AVX2 equivalent. The intrinsic concatenates two ZMM registers, shifts the pair right by an immediate multiple of 32 bits, and returns the low 512 bits of the result.⁴ The horizontal pass needs three views of the row, at offsets of 0, 16 and 32 bits, so that a 3-value window is striped across three registers. The original AVX2 kernel obtains them with three separate loads per iteration. Striping remains the right way to compute `mode3` at the wider width, but the three views need not come from three loads: consecutive iterations overlap almost entirely, so the data can be carried in registers instead of being fetched again.

Each iteration therefore keeps the 64 bytes read on the previous iteration in a live register, `prev`, and issues a single load for the next 64 bytes, `next`. A scalar prologue advances the row pointer so that all loads are 64-byte aligned and thus can execute with lower latency than the unaligned loads in the AVX2 implementation, at least a third of which straddle a cache line boundary. We can use ALIGNR to acquire any view into the sliding window shifted by a multiple of 32 bits. Specifically, we use `prev` directly as the `a` register, and offset by exactly 32 bits to get `c = _mm512_alignr_epi32(next, prev, 1)`. We cannot get the 16-bit offset register

⁴ Microarchitecturally this is generally equivalent to a two-way, 32-bit cross-lane permute, except that the permutation is specified by an immediate rather than held in a separate ZMM register.

natively with ALIGNR, since its shift granularity is 32 bits, but that view can be assembled from the two registers already in hand. Within each 32-bit lane, the required pair is the high half of `a` beside the low half of `c`, giving `b = _mm512_or_si512(_mm512_srli_epi32(a, 16), _mm512_slli_epi32(c, 16))`. After computing `mode3(a, b, c)` the window slides by setting `prev = next`.

At equal vector width this trades two of the three loads for one ALIGNR, two shifts and one bitwise OR, moving work off the load ports and onto the vector ALU ports; the register-to-register move that slides the window is resolved by the renaming stage and costs nothing. Against the AVX2 baseline, the two effects compound: each iteration also covers twice as many elements, so the kernel issues one aligned load per 64 bytes of output where the original issued six (at least two of which carried an additional cache line-crossing penalty). Both changes were needed to attain an actual speedup in this kernel: as Chapter 7 records, a version of the `mode3` filter that simply widened the AVX2 code to 512 bits and changed nothing else was slower than the code it replaced. This result parallels the performance regressions in the naively-widened kernels in §6.3.1. Widening SIMD vectors and changing nothing else about an algorithm can worsen performance just as often as it improves it; only by specializing code for the unique *capabilities* of a SIMD instruction set can a vectorized kernel reliably improve its performance.

6.3.4 Top-*k* selection

CAMBI executes a variant of the quickselect algorithm to find the largest *k* *c*-values computed by the `compute_c_values` function from §6.3.2, then computes the mean of those *k* values to generate its final score at a particular scale. Quickselect is efficient in asymptotic terms and carries an expected linear runtime⁵. However, the constant factors in a typical quickselect implementation are massive because the branches in the main loop body are poorly predicted, and the compiler cannot vectorize the implementation at all. VMAF’s two-way quickselect repeatedly loops through elements from two pointers moving towards each other from either side of the pivot, and swaps them whenever they are misordered relative to the pivot. This approach requires the CPU to keep a massive number of branches in flight during out-of-order execution, and can lead to massive slowdowns if inner loop trip counts are unpredictable because the inputs are not locally structured.

It is difficult to vectorize this algorithm with older SIMD instruction sets such as AVX2 because splitting elements based on a pivot requires conditional cross-lane moves that cannot be implemented efficiently. However, AVX-512 introduced compress and expand instructions that allow for seamless vectorized partitioning of an array. This makes it possible to create an efficient in-place quickselect relatively easily.

I implemented such a quickselect by extending the two-pointer approach already implemented in VMAF to keep track of a read head and write head for both the left and right sides. The range `[l_read, r_read]` determines the elements that are not yet safe to overwrite as they

⁵ Pathologically poor pivoting can degrade this to $O(N^2)$. In VMAF, this is generally not a problem because the data is produced from a pixel histogram whose structure is relatively uniform.

have not yet been loaded into registers or written out to the appropriate partition side, while `[l_write, r_write]` determines where the next partitioned values need to be written. The read heads run ahead of the write heads; that is, the read head range is contained entirely within the write head range. The implementation loads data from whichever of the left or right side has less slack between the read head and write head, and the corresponding read head is moved forward. Then, the algorithm distributes those elements to the appropriate sides of the pivot using two calls to `_mm512_mask_compressstoreu_ps`. The mask for the left side is generated with a greater-than comparison between the elements and the pivot, and the mask for the right side is the inverse.⁶ The write heads are advanced according to how many elements were written out to each side. This process repeats until all elements have been partitioned, at which point we can determine which side of the pivot contains the k largest elements and recurse into that side. Based on certain invariants about the number of elements processed per iteration and the way we select which head to read from, we can prove that our in-place writes never alias with ongoing reads and ensure correct output.

This implementation works, but can be improved substantially for the particular distribution that the c-values tend to follow. In particular, the c-values are highly sparse, which means that the pivot has a massive number of duplicates (i.e. 0.0) in a typical round of the quickselect routine. Instead of partitioning elements that equal the pivot into either the left or right bucket, we perform implicit 3-way partitioning: values are only moved to the left or right buckets if they are strictly less than or greater than the pivot, and are otherwise not written at all. This means the total gap between the read heads and write heads grows by the number of elements that equal the pivot on each iteration, and there may be a gap between the two write heads at the end of the quickselect read. The size of this gap defines precisely the number of elements that were found to be equal to the pivot during the calculation, and they can be filled in with a `memset`-esque fill that writes the pivot value to every array index in the range. Moreover, we can guarantee that the array is precisely sorted everywhere in the range of those elements, which means that if the fill region contains index k , we can end the quickselect early. In practice, this optimization means the quickselect almost always finishes in a single round, and it dramatically reduces the total number of elements the algorithm must process and partition.

This is a classic case of value-range specialization: the sparsity of the c-values is a property that is not visible anywhere in the source and could not be inferred at compile time, but infusing knowledge of the observed data distribution into the algorithm implementation makes it possible to dramatically reduce the total work it performs in the common case.

6.3.5 Boolean DP scan

The `compute_dp_row` kernel computes an inclusive prefix sum (also known as a scan) on the derivative values computed by the aforementioned derivative kernel in §6.3.1, then adds corresponding elements to a buffer containing its previous output. Derivative values are represented as `uint16_t`, while the sum outputs are `uint32_t`. CAMBI applies this kernel

⁶ In practice, we must split some of the elements that are exactly equal to the pivot into each bucket to avoid infinite loops when the pivot has duplicates.

to each row of the derivative matrix in series to compute a 2D spatial integral of the feature derivative across the full image. This kernel originally had only a scalar C implementation, and due to the prefix sum’s loop-carried dependency, the compiler failed to vectorize it.

There are well-known algorithms to perform a prefix sum with parallel processors (in our case, SIMD lanes) [33]. I implemented the step-efficient Hillis-Steele algorithm for the prefix sum, using the AVX-512 `_mm512_alignr_epi32` instruction to perform the cross-lane shifts required to propagate partial sums across the vector lanes. I expanded the initial 16-bit derivative values to 32 bits to avoid overflow, so the implementation was able to compute 16 elements in 4 shift-add steps within a single ZMM register. After the local prefix sum, the implementation broadcasts the current running total (that is, the sum of all blocks before the current one) and adds it to every lane, before finally adding values from the base buffer element-wise and storing the result in the output buffer.

This implementation works, but dramatically overprovisions the width of its accumulators. As we discussed in §6.3.1, the derivative kernel writes the result of a boolean expression to each derivative cell, meaning the prefix sum only ever has to handle individual derivative values that are 0 or 1. A general prefix sum on 16-bit integers needs to widen to 32 bits before local accumulation to avoid overflow in an intermediate step, but with a boolean input, there is a much tighter upper bound on the value of each accumulator in each intermediate step. We can take advantage of these constraints when designing our implementation to pack more elements into a single ZMM register and dramatically increase the arithmetic density of our calculations.

My final implementation combined value-range, algebraic, and environmental specialization to achieve maximal performance. Figure 6.2 demonstrates the key components of the procedure visually. We begin by packing a boolean derivative into each nibble of a ZMM register by striping together four 32-element, 16-bit derivative registers with shifts and bitwise ORs. This places a total of 128 elements within a single register. Because AVX-512 does not support native nibble-level arithmetic, we must be careful that our arithmetic does not overflow in any intermediate step as that may corrupt the low bits of the next nibble.

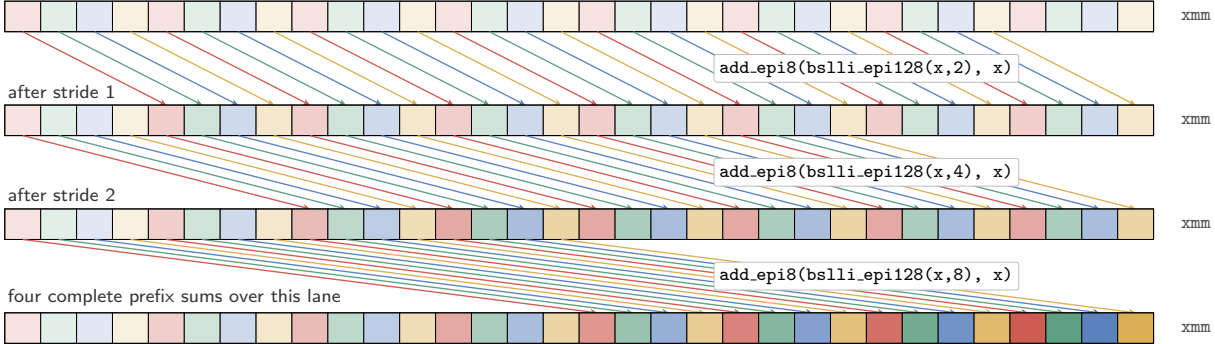
We run a first stage that computes the prefix sum of each stripe within its 128-bit lane. By restricting our first-stage prefix sums to stay within 128-bit lanes, we can use the cheaper `_mm512_bslli_epi128` for shifting in the Hillis-Steele shift-add sequence, rather than the more expensive `_mm512_alignr_epi32`. We also avoid overflowing the nibble: keeping the prefix sums in-lane restricts the maximum value to 8 within each of the four stripes, well below the maximum 15 in each nibble. This stage neatly combines environmental specialization for the cost model of AVX-512, in which lane-crossing operations are expensive, with value-range specialization that provides small, provable bounds on element-wise sums.

The second stage splits the 128-element ZMM into two 64-element ZMMs with a separate prefix sum in each 8-bit element. These now contain two stripes per 16-bit block, rather than four. We will be able to store the final prefix-sum values for the 128-element block within these split ZMMs, since each element has 8 bits and every value in the prefix sum will fall in $[0, 128] \subset [0, 2^8)$. We now run our single lane-crossing permute to place the top word of every

A Four prefix sums at once, inside one 128-bit lane

each 16-bit element carries four 0/1 values, one per nibble, in the order $A\ C\ B\ D = +0\ +64\ +32\ +96$. Shade deepens as an element accumulates more inputs

8 elements of 16 bits, 32 nibbles



Nothing crosses a lane boundary and nothing carries: each nibble reaches at most 8, so a single `add_epi8` advances all four scans at once.

one of the four 128-bit lanes of the zmm, all scanning independently

B Combining the four lanes, with a single cross-lane instruction

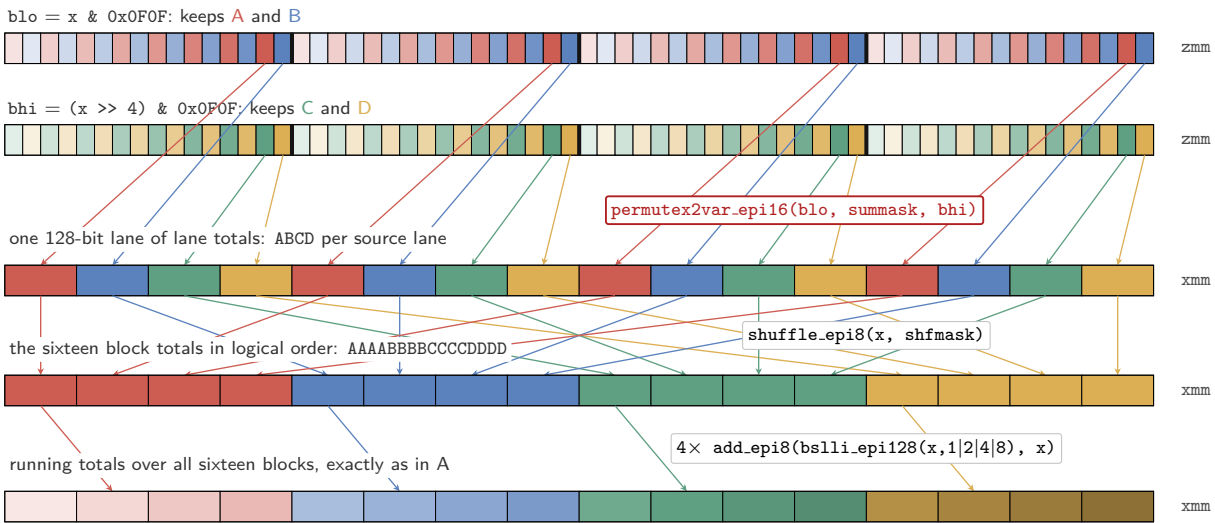


Figure 6.2: The two stages of the specialized scan. Because the derivatives are produced by comparison operators, they are provably 0 or 1, so four of them are packed into every 16-bit lane, one per nibble, and every running sum is done in 8-bit arithmetic. Cell hue identifies which of the four interleaved strips a value belongs to, and cell shade deepens as an element accumulates more inputs, so a completed scan reads as a light-to-dark ramp. Stage A scans within one 128-bit lane: nothing crosses a lane boundary and no nibble can carry into its neighbor, so a single `add_epi8` advances all four scans at once. Stage B combines the four lanes, gathering their totals into logical order and scanning those in turn. The whole construction crosses a lane boundary exactly once, at the instruction marked in red. Not shown is the final, in-lane step of adding the appropriate elements from the inter-block scan from Stage B back into the intra-block scan from stage A.

128-bit lane in both ZMMs into a single 128-bit lane.⁷ These are the sums of the 8-element stripes within each lane, and by running a prefix sum on these values we can compute the necessary “block prefix” for each stripe. After shuffling within the lane to permute the stripe sums into the appropriate order, we repeat the `_mm512_bslli_epi128` procedure from the first stage to compute the block prefixes. We finally run an in-lane permute independently on each 128-bit lane of the block prefix ZMM to move the block prefixes into the appropriate locations to be added to their corresponding stripes in the 64-element split ZMMs. These split ZMMs now contain a striped 128-element prefix sum, and these can be deinterleaved, converted, and added with the base value to compute the final output.

In total, the optimized scan kernel performs moderately more work per iteration, but computes $8\times$ more values. It uses only a single lane-crossing operation per 128 elements in its arithmetic phase, while the original implementation performs 4 lane-crossings for every 16 elements, i.e. $32\times$ more lane-crossings per 128 element block. Altogether, these changes contribute to a significant speedup over an AVX-512 Hillis-Steele scan that was already well-optimized. We quantify the exact benefits of the value-range and algebraic specializations versus the simple vectorization in §7.3.2.

6.4 Motion

The motion feature is one of the only intertemporal features in VMAF, and it assesses how rapidly the scene is moving from frame to frame. This is one of the smaller features in VMAF; it does not take up much runtime in either the v0 or v1 model, so the optimizations I made to the motion kernels were relatively light.

6.4.1 Striped widening

Like with VIF (§6.2), the motion feature widens a single ZMM of 32-bit values to two ZMMs with 64-bit values at various points to perform intermediate computations with higher precision, before later normalizing and merging the 32-bit results back into a single ZMM. It used a similar technique as the original VIF kernel to achieve this: split the ZMM into its low and high 256-bit parts and run conversion operations to widen, and then use further conversion operations and an insertion to narrow.

We can use the same striping/blending trick as in Figure 6.1 to achieve the same result without any cross-lane operations. The only difference in the motion case is dealing with signed values during the widening phase. Instead of simply masking off the high 32 bits to get the even elements, we must turn the high 32 bits into a sign extension. This can be done by swapping the bitwise AND for a 32-bit left shift, followed by a 32-bit arithmetic right shift, i.e. `out = _mm512_srai_epi64(_mm512_slli_epi64(val, 32), 32)`. To get the high 32-bit stripe, we simply swap the 32-bit logical right shift in the original implementation for an arithmetic right shift. In total, this is slightly more expensive than the unsigned formulation

⁷ We actually replicate the same pattern across all 4 lanes of a ZMM register to avoid further lane crossing in a later step. Since all 128-bit lanes are identical, the visual in Figure 6.2 remains accurate.

(costing one less bitwise operation and two more shifts), but is still much cheaper than the naive widening/narrowing operations the kernel originally used. This change produces a small speedup on both the 8-bit and 16-bit kernels.

6.4.2 MADD filter computation

In an intermediate step to compute the motion score, the motion kernels compute a low-pass-filtered version of the absolute difference between two consecutive image frames. The filter is a 5-wide, 16-bit quantized Gaussian, and is in fact identical to the VIF scale 2 kernel: the taps are [3571, 16004, 26386, 16004, 3571].

The 8-bit kernel uses a similar approach to the VIF *16-bit* baseline kernel to compute the vertical convolution. It extends 8-bit pixels to 16-bit, multiplies the extended values by the filter taps using separate low and high multiplications (`_mm512_mullo_epi16` and `_mm512_mulhi_epi16`), then interleaves the low and high bits with `_mm512_unpacklo_epi16` and `_mm512_unpackhi_epi16` to reassemble the halves into 32-bit lanes. This is 4 instructions per tap, or 20 for the loop overall.

As we have seen in VIF (§6.2), there is a faster way to do this. `_mm512_madd_epi16` combines 16-bit multiplication with horizontal accumulation and 32-bit widening in a single instruction. We can apply this instruction to a vector of interleaved pixel values from different rows, alongside a vector of their corresponding interleaved filter taps, to compute the convolution significantly faster. In total, the MADD approach saves 14 of the 32 vector instructions the original loop body needed for each 32 pixels. The five taps now cost 6 interleaves and 6 MADDs instead of 20 multiplies and interleaves; the horizontal accumulation that MADD performs for free removes four of the eight explicit additions; and the rounding term rides in the spare lane of the center tap, which is the one tap without a symmetric partner, so it no longer costs a separate addition.

The 8-bit motion kernel earns a substantial speedup after this optimization, but its impact on end-to-end performance is quite small because the motion feature is only a small portion of the VMAF computation as a whole. It is no coincidence that this environmental specialization both came relatively easily and brought little end-to-end improvement. The VIF 8-bit kernel is structured very similarly to the motion kernel for the initial convolution step, and it took advantage of the `_mm512_madd_epi16` intrinsic in the baseline while motion did not. This discrepancy is a result of the difference in runtime for the two features: because VIF dominates end-to-end runtime for the v0 model, while motion is a relatively small feature in both v0 and v1, the motion kernels have likely received comparatively less attention from performance engineers. The other features in the patch set that had better-optimized baselines required more creative techniques to attain a speedup, and that is where the specialization framework excels for finding speedups that traditional approaches cannot.

6.5 ADM

The ADM feature is VMAF’s implementation of the Detail Loss Metric (DLM). ADM and motion are the only two features in VMAF that are used by both the v0 and v1 models. Calculating ADM composes a modest portion of the runtime for the v0 model and a relatively large portion for v1. Due to its algebraic complexity, ADM has an AVX-512 codepath with more total kernels and lines of code than any other feature. For the purposes of the patch, I focused on the single kernel taking the most runtime on a typical video: the 8-bit discrete wavelet transform.

6.5.1 Byte-wide multiplies in the DWT

ADM computes a 2D discrete wavelet transform (DWT) of both the reference and distorted video frames at four scales. The operation is structured similarly to a separable convolution, but with separate kernels to compute the high-pass and low-pass outputs in each direction. The 8-bit version of the DWT kernel originally computed its outputs similarly to the 8-bit VIF statistic kernel from §6.2: it loads 256-bit YMMs of pixel values, widens the 8-bit pixels to 16-bit, and uses `_mm512_madd_epi16` to compute 32-bit partial products more efficiently. After accumulating the 32-bit convolution-esque sum, it normalizes the results back to 16-bit with a rounding division (adding 128, then right-shifting by 8). The coefficients for both passes in the ADM kernel were 16-bit, though signed rather than unsigned as in VIF. The DWT also required only convolutions of the original image plane, and not of any transformations.

The original implementation was already quite efficient, and it moved data between registers in such a way that the only cross-lane operations were the original widening operations to convert the 8-bit inputs to 16-bit values. Like the VIF filter, the coefficients for the ADM DWT are fixed at build time, but there are no obvious patterns or symmetries that can be exploited to reduce the needed work.

Seeing that data movement operations composed a large proportion of the kernel runtime relative to the multiplications, I investigated whether it was possible to trade some ALU work for reduced shuffling overhead. I discovered that by exploiting particular properties of the DWT coefficients the kernel used, it became possible to perform the multiplications in 8-bit rather than 16-bit, and accumulation in 16-bit rather than 32-bit. Although my transformation did not reduce the total multiplication work, it avoided the 16-bit widening, simplified the normalization, and skipped several intermediate bit twiddling operations that were needed in the original kernel.

The DWT filter has width 4, so four products must be summed for each accumulated output. The original implementation packed pixels from the first two and last two rows into adjacent 16-bit lanes using the packing intrinsics `_mm512_unpacklo_epi16` and `_mm512_unpackhi_epi16`, then applied MADD to an interleaved vector of the corresponding DWT coefficients to compute two requisite partial products per pass. Specifically, if we denote the pixels as x_0, x_1, x_2, x_3 and the filter taps as f_0, f_1, f_2, f_3 , two MADD operations computed $f_0x_0 + f_1x_1$

and $f_2x_2 + f_3x_3$. Let

$$\begin{aligned} f_i^h &= \left\lfloor \frac{f_i + 128}{256} \right\rfloor \\ f_i^l &= f_i - f_i^h \cdot 256 \end{aligned}$$

Note that $f_i = f_i^h \cdot 256 + f_i^l$ and $f_i^l, f_i^h \in [-128, 127]$ by construction. We can use this decomposition to construct a pair of partial products $p_i^l = f_i^l x_i$ and $p_i^h = f_i^h x_i$ such that $f_i x_i = p_i^h \cdot 256 + p_i^l$.

SSSE3 (and, by extension, AVX-512) includes an MADDUBS instruction that performs a similar operation of element-wise multiplication and horizontal accumulation, but for the product of signed and unsigned 8-bit values instead of two signed 16-bit values. Because our f_i^l and f_i^h are signed 8-bit integers, and the pixel values are unsigned 8-bit, we can use this instruction to compute low and high partial products. The only difficulty is saturation: unlike the `_mm512_madd_epi16` intrinsic, `_mm512_maddubs_epi16` saturates its horizontally summed partial products to the signed 16-bit integer limit if they would otherwise overflow. This is a problem: if we call `_mm512_maddubs_epi16` with interleaved pixel vector of x_i and x_{i+1} , combined with an interleaved low filter tap vector of f_i^l and f_{i+1}^l , the result may saturate if $f_i^l + f_{i+1}^l \notin [-128, 128]$.⁸

To progress, we must make use of our knowledge about what the filter taps actually are rather than speaking purely algebraically. The coefficients to extract the low pass are $[15826, 27411, 7345, -4240]$, while the high pass requires $[-4240, -7345, 27411, -15826]$. Applying the decomposition above to each pass, each pixel pair, and each byte of the coefficient yields the eight interleaved tap vectors required by `_mm512_maddubs_epi16`, listed in Table 6.2.

We see that there are exactly two pairs that could saturate. Since both involve the (x_0, x_1) pixel pair, we can avoid the saturation by using a correction term that requires only a single additional `_mm512_maddubs_epi16` to calculate.⁹ For example, we can use the tap pair $[79, 0]$ as our correction term; then we could replace the $[107, 62]$ pair with $[107 - 79, 62] = [28, 62]$, and $[79, 112]$ with $[79 - 79, 112] = [0, 112]$. None of the new set of pairs saturate, so we can successfully compute all the low and high partial products we need. It then becomes straightforward to compute the final normalized results: since the normalized results in the original implementation are 16-bit and are obtained by dividing the 32-bit accumulator by 256 with rounding, we can treat our high partial products as an approximation for the final result, and add a small corrective term based on right-shifting the low partial products and adding the rounding term.

The single correction term we must calculate for each pair of high and low passes leads to an overhead of $\frac{1}{8} = 12.5\%$ more multiplies than were needed with the MADD-based implementation. In exchange, we dramatically reduce the number of shuffle-type operations

⁸ This differs slightly from the signed 8-bit integer range because we can accept 128, as $128 \cdot 255 = 32640 \leq 32767$.

⁹ One can verify by hand that it is not possible to change the tap grouping such that no overflows occur; this would avoid the need for a correction term entirely.

Table 6.2: The eight interleaved tap pairs required to compute the 8-bit ADM DWT with `_mm512_maddubs_epi16`, one per combination of pass, pixel pair and coefficient byte. A star marks a pair that can saturate, that is, one whose taps sum outside $[-128, 128]$ and whose horizontal sum can therefore exceed the signed 16-bit range for admissible pixel values. Only two of the eight are affected, and both involve the first pixel pair.

Pass	Pixels	Byte	Taps	Saturates?
Low	(x_0, x_1)	Low	[19, -46]	
		High	[107, 62]	*
	(x_2, x_3)	Low	[112, -79]	
		High	[-17, 29]	
High	(x_0, x_1)	Low	[79, 112]	*
		High	[-29, -17]	
	(x_2, x_3)	Low	[46, 19]	
		High	[-62, 107]	

needed to compute the outputs. Normalizing both kernels to the same unit of work (64 output columns), the original executes 8 byte-to-word widenings, 8 16-bit unpacks, 4 saturating packs and 8 masked blends, or 28 data movement instructions in total. The replacement needs 4 byte unpacks and 4 cross-lane permutes, for 8. The load count halves as well, since eight 256-bit loads become four 512-bit ones, and the whole inner loop falls from 84 vector instructions to 60. This change leads to a sizable speedup whenever those shuffles are the bottleneck, and is about neutral if loads or multiplies are the bottleneck. The exact result depends on the microarchitecture, as we will see in Chapter 7.

The ADM DWT optimization is the clearest example of the value of algebraic specialization in the patch series. The transformation from 16-bit to 8-bit multiplication and accumulation was worthwhile due to a particular (and entirely coincidental) property of the specific coefficients used by the DWT calculation: it is possible to create a tap pairing in which only one correction term is needed. With two or three correction terms, the additional multiplication overhead would likely make this transformation cost more than it saves. These kinds of optimizations are the most difficult to find with traditional performance engineering techniques because they require inspection of specific numerical properties of the algorithm, rather than high-level algebraic insights that can be applied to many kinds of code.

6.6 Float Convolution

The final component of VMAF that I optimized is the symmetric, separable floating-point convolution used by the SpEED-QA feature [34]. It is an ordinary two-pass filter: a vertical pass that convolves down a column of source rows into a temporary buffer, then a horizontal pass that convolves along each row of that buffer. The baseline implementation assumes that the filter is symmetric and its width is odd, but neither the width nor the coefficients are known when `libvmaf` is compiled because they are derived from the model configuration at runtime. In practice, when the convolution codepath is invoked by the v1 model, they are Gaussian kernels with widths of either 3 or 9. However, the baseline implementation does not include specialized fast-paths for any size.

This kernel is perhaps the most useful demonstration of the value of specialization within this patch series. I wrote the kernel twice: once by hand with standard AVX-512 intrinsics, and once as a staged BuildIt generator that specializes on the filter width. I spent a similar amount of effort optimizing each: both implementations target the same instruction set and are heavily optimized with traditional techniques to be cache efficient and minimize overhead. Therefore, the difference in performance between them is a product of the additional information embedded into the structure of the BuildIt-generated code. A speedup for the staged implementation over the hand-written implementation is evidence that specialization fundamentally increases the throughput ceiling for a particular workload.

Note that there is a key difference between the BuildIt generator presented in this section, and those discussed in Chapter 5. The VMAF convolution kernel generator runs at *build* time, not at runtime. Meson invokes it as a custom target during compilation, and it emits one kernel for every odd width up to 17, for AVX2 and AVX-512 separately, into a width-indexed dispatch table. The runtime code generation discussed in Chapter 5 is a different binding time and provides a different set of trade-offs. In this case, build-time staging was appropriate because the most useful runtime information to specialize on for a convolution is the filter width, and because filter widths have a small, finite distribution in practice, the runtime dispatch approach from §4.3 materializes the specialization with the least overhead.

6.6.1 The existing vector path

The baseline both replacements are measured against is the upstream AVX2 implementation. Code Block 6.1 shows its vertical pass in full. Although it is vectorized, it is not otherwise well-optimized for the unique structural properties of a convolution. For example, because each output row is computed independently, every source row is reloaded once for each output row that needs it even though consecutive outputs overlap almost entirely. Computing multiple rows at a time could dramatically reduce the total number of loads performed by this kernel across a full image. There are other minor issues too: the multiply and the addition are separate instructions rather than a fused multiply-add, and nothing in this implementation exploits the symmetry of the filter (which is known statically and guaranteed by the API contract).

We begin by accelerating this kernel using traditional techniques, but without making any

```

for (int j = 0; j < j_end; j += AVX_STEP) {
    __m256 sum = _mm256_setzero_ps();

    for (int k = 0; k < filter_width; k++) {
        __m256 g = _mm256_load_ps(src + k * src_stride + j);
        g = _mm256_mul_ps(f[k], g);
        sum = _mm256_add_ps(sum, g);
    }

    _mm256_store_ps(dst + j, sum);
}

```

Code Block 6.1: The upstream AVX2 vertical pass. The tap loop reloads a source row on every iteration, and the filter width is a runtime parameter.

assumptions about filter width. Afterwards, we will investigate how much further width-specialization can improve on that optimized baseline.

6.6.2 Explicit vectorization

The hand-written replacement was designed exclusively using traditional vectorized performance engineering techniques. I implemented improved kernels for both AVX2 and AVX-512. The vertical pass is more expensive than the horizontal pass in the naive implementation because it reloads every source row once per output row it contributes to, which causes the kernel to load each pixel in the image `filter_width` separate times over the course of the full convolution. The filters used by VMAF v1 are generally wide enough that this creates a load bottleneck and dominates overall runtime.

The optimized kernel produces four output rows at a time and keeps a sliding window of source rows live in registers, rotating them by hand as the tap loop walks outward from the center tap. Each source row is loaded once and contributes to all four accumulators it participates in, so the load count falls by roughly a factor of four across the full image. The harness for the kernel is also modified to run the horizontal pass immediately after the vertical pass produces a particular output row to improve temporal locality.

This kernel is a massive improvement: it keeps the FMA units fed with data more consistently and mitigates the load bottlenecking of the original implementation. However, it still suffers from a few performance limitations. First, the tap loop is a genuine runtime loop: because the filter width is not known at compile time, it cannot be unrolled. Therefore, a filter coefficient is broadcast from memory on every iteration, and the window rotation compiles to real register-to-register moves rather than being resolved away. Only if we knew the width of the filter statically could this overhead be eliminated.

Additionally, the number of output rows being produced at once is a fixed parameter of the kernel regardless of kernel filter width. Using more than four output rows causes stack spilling on AVX2 with this style of middle-out accumulation. Although the number of rows could theoretically be increased further on AVX-512 due to the higher total register count, it

```

for (int k = radius - 1; k >= 0; k--) {
    __m512 fk = _mm512_set1_ps(filter[k]);

    __m512 g3_l = g2_l;          /* rotate the upper window by hand */
    g2_l = g1_l;
    g1_l = g0_l;
    g0_l = _mm512_loadu_ps(sl);
    sl += src_stride;

    sum0 = _mm512_fmadd_ps(fk, g0_l, sum0);
    sum1 = _mm512_fmadd_ps(fk, g1_l, sum1);
    sum2 = _mm512_fmadd_ps(fk, g2_l, sum2);
    sum3 = _mm512_fmadd_ps(fk, g3_l, sum3);

    __m512 g0_r = g1_r;          /* rotate the lower window by hand */
    g1_r = g2_r;
    g2_r = g3_r;
    g3_r = _mm512_loadu_ps(sr);
    sr += src_stride;

    sum0 = _mm512_fmadd_ps(fk, g0_r, sum0); /* same tap, same */
    sum1 = _mm512_fmadd_ps(fk, g1_r, sum1); /* accumulators */
    sum2 = _mm512_fmadd_ps(fk, g2_r, sum2);
    sum3 = _mm512_fmadd_ps(fk, g3_r, sum3);
}

_mm512_storeu_ps(&dst[0 * dst_stride + j], sum0); /* four accumulators, */
_mm512_storeu_ps(&dst[1 * dst_stride + j], sum1); /* four output rows */
_mm512_storeu_ps(&dst[2 * dst_stride + j], sum2);
_mm512_storeu_ps(&dst[3 * dst_stride + j], sum3);

```

Code Block 6.2: The hand-written vertical pass, abridged. Two windows are carried, one walking up from the center tap and one walking down, and both are rotated explicitly on every iteration. The filter’s symmetry lets a single broadcast tap `fk` serve both, and both fold into one accumulator per output row, which the epilogue retires to four different rows, and the tap loop bound `radius` is a runtime value.

did not seem to bring significant speedups in testing. With a $4\times$ reduction in load count the kernel was no longer so strongly load-bottlenecked; instead, the benefits of the larger unroll are largely negated by increased issue throughput bottlenecking from the register sliding window and worse instruction-cache behavior. Increasing the unroll factor any further also made the code much more difficult to parse and maintain. Without strong evidence of greater unrolling being better in general, it seemed like the most sensible decision from a maintainability perspective to keep the AVX2 and AVX-512 codepaths aligned with the same unroll factor.

The horizontal pass (Code Block 6.3) has the same runtime tap loop but a different data problem, and it is the one that turns out to constrain the hand-written kernel most severely. Each tap needs the row displaced by a different number of elements, so the kernel needs several differently offset views of the same data. Every tap therefore issues its own unaligned load, eight of them per iteration in the AVX-512 kernel, and the views they produce overlap almost entirely: consecutive taps read the same cache lines shifted by one element. We essentially use the load-store unit as an ad-hoc shuffle engine.

```
for (int k = radius - 1; k >= 0; k--) {
    __m512 fk = _mm512_set1_ps(filter[k]);

    sum0_l = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sl),      sum0_l);
    sum1_l = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sl + 16), sum1_l);
    sum2_l = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sl + 32), sum2_l);
    sum3_l = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sl + 48), sum3_l);
    sl--;
    /* shift one element left */

    sum0_r = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sr),      sum0_r);
    sum1_r = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sr + 16), sum1_r);
    sum2_r = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sr + 32), sum2_r);
    sum3_r = _mm512_fmadd_ps(fk, _mm512_loadu_ps(sr + 48), sum3_r);
    sr++;
    /* shift one element right */
}

/* eight accumulators, four output blocks: the banks are summed at the end */
/* two banks are used so that the dependent multiply-add chains do not stall on one
   ↪ another */
_mm512_storeu_ps(&dst[j + 0], _mm512_add_ps(sum0_l, sum0_r));
_mm512_storeu_ps(&dst[j + 16], _mm512_add_ps(sum1_l, sum1_r));
_mm512_storeu_ps(&dst[j + 32], _mm512_add_ps(sum2_l, sum2_r));
_mm512_storeu_ps(&dst[j + 48], _mm512_add_ps(sum3_l, sum3_r));
```

Code Block 6.3: The hand-written horizontal pass, abridged. Each tap issues a fresh unaligned load at a new offset, and two accumulator banks are carried per output block so that the dependent multiply-add chains do not stall on one another.

The offsets themselves are unavoidable: a convolution needs the data at several displacements and no amount of restructuring can change that. However, we can choose how we generate those offset registers. An alternative approach to issuing a separate load for each offset is to load each row exactly once and derive the offset views by shuffling within a sliding

window of registers. This spends shuffle throughput instead of load throughput, and on some microarchitectures, this can be a massive boost to throughput. AVX-512 provides exactly the operation we need in `_mm512_alignr_epi32`, which concatenates two registers and extracts a window from the pair. However, the amount to shift by is encoded as an immediate in the instruction rather than read from a register. Because the set of needed shifts is dependent upon the filter width, a kernel whose filter width arrives at runtime cannot supply immediates for shift amounts and is therefore incapable of using the window-shuffling strategy at all.¹⁰ This is the case of §4.2 where generality does not merely make an optimization slower, but rather makes it entirely inexpressible. §6.6.3 will use the sliding window approach when the width is known.

6.6.3 Staged code generation with BuildIt

The staged version of the convolution filter makes use of exact knowledge about the filter width to implement the convolution with less overhead and more rigorous optimality conditions than the width-generic version in §6.6.2. I implemented the staged kernel in BuildIt because its type-based multi-staging and support for natural control flow made it very straightforward to port my existing implementation. I was able to design a BuildIt generator that could synthesize both AVX2 and AVX-512 kernels for any filter width automatically, and select the ideal implementation strategy based on the filter width and environmental constraints (e.g. register budget).

Because the generator is a single source file that emits a family of kernels rather than a separate hand-written kernel for each width, it avoids the maintenance burden that typically plagues kernels specialized for a large input space. Any change to the algorithm or instruction selection can be made in one place and will automatically apply to all widths.

The `vconv` generator implements three structures for a vertical convolution: `sequential`, `midout`, and `midout_vartap`. Code Block 6.4 is the generator fragment corresponding to `midout_vartap`, which corresponds most directly to the hand-written loop of Code Block 6.2.

It is recognizably the same algorithm, but `tap` is a `static_var`, so the loop itself never reaches the second-stage code; it runs during generation and emits its body once per tap. Because `bidx` and the modular subscripts are computed from first-stage values, they resolve to particular registers while the code is being generated, which removes the runtime register rotation entirely. In the hand-written kernel, the window must be shifted because the tap index is a runtime value and the correct source row has to be selected dynamically. By contrast, when the index is first-stage, the generator can statically keep track of which register holds which value and name different registers in the generated code for different iterations in the first-stage loop. The bookkeeping does not become cheaper; it ceases to exist entirely. Put another way, the data movement has moved down a layer: instead of software copying values between registers so that the hardware can keep reading the same ones, the first stage

¹⁰ Alternate approaches such as using a dynamically generated set of permutation masks do not work either, as the cost to load the masks will generally counteract any performance benefit from shuffling within registers over loading unaligned from memory.


```

for (builder::static_var<int> tap = 0; tap < radius; ++tap) {
    int k = radius - 1 - tap;
    int bidx = tap % cfg.vert;
    builder::dyn_var<T> fk = ftaps[k];

    sr[0] += src1_stride;
    rows_r[bidx] = T::loadu(sr[0]);    /* overwrite one slot */

    for (builder::static_var<int> i = 0; i < cfg.vert; ++i)
        accums[i] = T::fma(fk, rows_r[(i + tap + 1) % cfg.vert], accums[i]);

    /* ...and symmetrically for the rows above the center tap */
}

```

Code Block 6.4: The middle-out structure in the generator, abridged. Every index that was a runtime quantity in Code Block 6.2 is now first-stage, so the window advances by overwriting a single slot rather than shifting its whole contents.

statically embeds that permutation into the instruction encoding itself between each unrolled tap.

This effect is visible in the code the BuildIt generator emits. Code Block 6.5 shows two consecutive tap blocks of the `midout` kernel at width 9, which is the structure the hand-written vertical pass of Code Block 6.2 implements.

Comparing this against Code Block 6.2 isolates what staging bought. Both compute the same thing in the same order and hold the same window of source rows live. The hand-written version advances that window with a chain of register-to-register moves on every tap, one per row held live, while the generator has no register-to-register moves at all.¹¹ Because the subscript $(i + \text{tap} + 1) \% \text{cfg.vert}$ is first-stage, it resolves during generation into a different register name in each block, so advancing the window is a matter of changing which register each multiply-add names rather than of copying anything. The tap loop itself is also gone: the emitted code contains one block per tap, so the trip count is implicit in the instruction stream instead of being a branch the predictor has to learn.

Note that it is likely the second-stage compiler would have been able to avoid the register-to-register moves even if the permutation had been emitted into the second-stage code as a sequence of variable reassignments. It is the fact that the first stage embeds information about the filter width into the second-stage code (implicitly, via the number of unrolled tap iterations) that makes this optimization possible, rather than clever design of the generator.

Although the above structure corresponds with the original unspecialized code best, it is not what the generator chooses at these widths. In particular, the generator has an internal cost model that tries to minimize the “load blowup”; that is, the ratio between the number of load instructions executed per iteration, and the number of rows that can be stored after each iteration, when all variants use the maximum unroll width that does not overflow the

¹¹ The occasional `__m512` reassignments are an artifact of the generator script and are elided by the compiler.


```

var40 = var40 + var3;
var31 = _mm512_loadu_ps(var40);           /* new row overwrites a dead slot */
var15 = _mm512_fmadd_ps(var41, var32, var15);
var16 = _mm512_fmadd_ps(var41, var33, var16);
var17 = _mm512_fmadd_ps(var41, var34, var17);
/* ... */
var22 = _mm512_fmadd_ps(var41, var31, var22);

__m512 var42 = var11;                     /* next tap */
var40 = var40 + var3;
var32 = _mm512_loadu_ps(var40);           /* the slot consumed first is reused */
var15 = _mm512_fmadd_ps(var42, var33, var15); /* operands renamed, not moved */
var16 = _mm512_fmadd_ps(var42, var34, var16);
var17 = _mm512_fmadd_ps(var42, var35, var17);
/* ... */

```

Code Block 6.5: Two consecutive tap blocks from the generated `midout` vertical kernel at width 9. The first argument to `_mm512_fmadd_ps` is the filter tap, the second argument is the row, and the third is the accumulator. Notice how from one tap to the next, the window of registers used by the accumulators shifts by 1 (from 32, 33, 34, ... to 33, 34, 35, ...). Between the blocks, the window advances without any data moving: the row loaded for the next tap overwrites the slot that was consumed first and is now dead, and every accumulator simply reads a different register than it did in the previous block.

register budget of the target SIMD instruction set (32 for AVX-512, 16 for AVX2). The cost model prefers `sequential` to `midout` for small widths because they perform the same number of loads per element, but `sequential` needs fewer accumulators and can thus unroll further before being at risk for stack spilling; therefore, `sequential` has lower load blowup. Code Block 6.6 shows the generator code for `sequential`, while Code Block 6.7 shows three consecutive blocks of the generated kernel at width 3.

In the generated code for the `sequential`-style kernel there is no broadcast, window shift, or pointer arithmetic between one block and the next. Each source row is still loaded exactly once and shared across every output that needs it, so this kernel attains all the reuse the hand-written kernel worked for without any of the overhead that was necessary to make that reuse possible.

By inspecting the generated second-stage code, we can see why the `sequential` kernel has lower load blowup, and is thus preferred by the generator’s cost model. The `midout` structure needs to keep two registers alive for every output row it wishes to produce; these are the left and right accumulators for each row, which is carried over from the striping in the unspecialized kernel that was used to prevent the second FMA in each loop iteration from stalling on the first one. Because the `sequential` kernel only retains one accumulator per output row, it can compute more output rows concurrently without spilling, and thus achieves the lower load blowup.

In this case, the cost model successfully picks the cheaper option for most microarchitectures. The main upside of the `midout` kernel is that it hides FMA latency better than the `sequential`

```

for (builder::static_var<int> i = -radius; i < radius + cfg.vert; ++i) {
    builder::dyn_var<T> row = T::loadu(&src1[j + i * src1_stride]);

    for (builder::static_var<int> k = 0; k < cfg.filter_width; ++k) {
        int tapk = std::min(+k, 2 * radius - k);
        int a = i - k + radius;
        if (a >= 0 && a < cfg.vert) {
            accums[a] = k == 0 ? T::mul(ftaps[tapk], row)
                               : T::fma(ftaps[tapk], row, accums[a]);
            if (k == cfg.filter_width - 1)
                T::storeu(&dst[j + a * dst_stride], accums[a]);
        }
    }
}
}

```

Code Block 6.6: The sequential structure, abridged. The accumulator index `a` and the tap index `tapk` are ordinary integers derived from first-stage counters, so the bounds test and the store condition are resolved during generation and neither appears in the output.

```

__m512 var42 = _mm512_loadu_ps(&var6[var11 + 2 * var3]);
var15 = _mm512_mul_ps(var9, var42);
var14 = _mm512_fmadd_ps(var10, var42, var14);
var13 = _mm512_fmadd_ps(var9, var42, var13);
_mm512_storeu_ps(&var4[var11 + 1 * var1], var13);
__m512 var43 = _mm512_loadu_ps(&var6[var11 + 3 * var3]);
var16 = _mm512_mul_ps(var9, var43);
var15 = _mm512_fmadd_ps(var10, var43, var15);
var14 = _mm512_fmadd_ps(var9, var43, var14);
_mm512_storeu_ps(&var4[var11 + 2 * var1], var14);
__m512 var44 = _mm512_loadu_ps(&var6[var11 + 4 * var3]);
var17 = _mm512_mul_ps(var9, var44);
var16 = _mm512_fmadd_ps(var10, var44, var16);
var15 = _mm512_fmadd_ps(var9, var44, var15);
_mm512_storeu_ps(&var4[var11 + 3 * var1], var15);

```

Code Block 6.7: Three consecutive blocks of generated code for the 3-wide filter. Each loads one source row, folds it into every accumulator it contributes to, and retires whichever output row that load completed. The two distinct taps of the symmetric filter are broadcast once at the top of the function as `var9` and `var10`.

kernel at the same level of unrolling. For an unroll factor U , the `midout` variant guarantees exactly $U - 1$ FMAs will be scheduled between any two FMAs accumulating to the same register. By contrast, the `sequential` kernel has a ramp-up phase and ramp-down phase at the edges where there are not enough output rows that use a given input row to fully hide FMA latency. In the unspecialized kernel with $U = 4$, the `midout`-style kernel was superior because four FMAs is not enough to fully hide FMA latency between successive accumulations to the same registers on most modern microarchitectures; using the `sequential` kernel would lead to even worse stalling than `midout`. However, because the unroll factor is so large for small sizes ($U = 27$ for width 3), FMA latency is completely hidden for the overwhelming majority of the time in the sequential kernel, and the greater unroll factor brings a bigger speedup than the marginal latency-hiding improvements of `midout`.

We will now compare the generated result for the horizontal staged convolution to the original unspecialized version. Code Block 6.8 shows the high-level structure of the BuildIt generator. The specific filter width and unroll factor requested are used to compute the set of all `_mm512_alignr_epi32` offsets needed for all windows in the horizontal convolution. Code Block 6.9 shows a snippet of generated code for a width-9 horizontal kernel.

This is the alternative approach to creating shifted windows for the horizontal convolution, as described in §6.6.2. The displacements are still required, and the generated kernel does not avoid them any more than the hand-written one did. What changes is which port supplies them: the hand-written kernel spends a load for every displacement, while the generated kernel spends U loads and a $N \cdot U$ shuffles to extract U vectors of output concurrently. The value of that change depends on the microarchitecture.¹²

6.6.4 Results

We will quantify the performance of these and other kernels more thoroughly in Chapter 7, but the key result is that the generated, specialized kernels are faster than the hand-written ones, which are themselves faster than the original unspecialized kernels. Table 6.3 shows the observed speedups on two different machines.

On the EPYC chip (which uses the Zen 5 microarchitecture), conventional tuning of the kernel bought +12% and +54% on the two passes, and specializing for width bought a further +65% and +99% on top of that. The ALIGNR-based staged horizontal convolution sees a speedup over the hand-optimized baseline because the shuffle units support $2\times$ higher throughput for ALIGNR instructions than their load port does for unaligned loads [35]. Likewise, the staged vertical convolution benefits from better cache reuse, plus reduced overhead from the register sliding window and variable loop. Specialization was worth more than the tuning that preceded it, even against a baseline of high-quality hand-written vector intrinsics.

The Xeon chip, based on the older Skylake-X microarchitecture, inverts the ranking. There,

¹² For the AVX2-generated kernels, `_mm256_alignr_epi32` is not available because ALIGNR was introduced with AVX-512. Therefore, the AVX2 kernels use the same unaligned-load logic as does the unspecialized kernel; the only benefit for specialization is slightly lower loop overhead because the filter tap loop is unrolled in the second stage.

```

builder::dyn_arr<T> blocks;
blocks.set_size(aligned_unroll);

/* ... */

/* iterate through filter taps */
for (builder::static_var<int> k = radius - 1; k >= 0; k--) {
    for (builder::static_var<int> i = 0; i < compute_blocks; ++i) {
        /* compute the element offsets we need within the contiguous register window
           ↪ */
        int loff = i * T::width + k;
        int roff = i * T::width + 2 * radius - k;

        int lstart = loff / T::width;
        int lshift = loff % T::width;
        int rstart = roff / T::width;
        int rshift = roff % T::width;

        /* use alignr with the exact registers we need */
        builder::dyn_var<T> ltap = lshift == 0
            ? blocks[lstart]
            : T::alignr(blocks[lstart + 1], blocks[lstart], lshift);
        builder::dyn_var<T> rtap = rshift == 0
            ? blocks[rstart]
            : T::alignr(blocks[rstart + 1], blocks[rstart], rshift);

        outputs[i] = T::fma(ftaps[k], T::add(ltap, rtap), outputs[i]);
    }
}

```

Code Block 6.8: The shuffle-favoring horizontal convolution generator, abridged. Each iteration of the outer loop loads a sequence of blocks on 64-byte boundaries into separate registers. The inner loop computes the exact offsets needed for symmetric taps of the horizontal convolution, evaluates which registers that offset straddles, and uses `_mm512_alignr_epi32` to extract the necessary window.

```

var18 = _mm512_loadu_ps(arg2 + (var7 - 4) + 0);    /* load the row once, */
var19 = _mm512_loadu_ps(arg2 + (var7 - 4) + 16);   /* in aligned blocks */
var20 = _mm512_loadu_ps(arg2 + (var7 - 4) + 32);
/* ... */
/* then derive each displaced view from registers, shift amount an immediate */
var43 = _mm512_alignr_epi32(var42, var41, 4);
var30 = _mm512_mul_ps(var4, var43);
var46 = _mm512_alignr_epi32(var45, var44, 4);
var31 = _mm512_mul_ps(var4, var46);
var49 = _mm512_alignr_epi32(var48, var47, 4);
var32 = _mm512_mul_ps(var4, var49);

```

Code Block 6.9: The generated horizontal pass for a 9-wide filter, abridged and with the float/integer casts elided for legibility. The row is loaded once and each displaced view is extracted from the loaded registers by `_mm512_alignr_epi32`, whose shift amount is an immediate and therefore only available once the width is known.

Table 6.3: Speedups from the two steps, separated on two representative hosts. Chapter 7 contains more information on speedup measurement methodology, and Table 7.1 includes more information about these hosts. Each figure is measured against the implementation named in the row above it: the hand-written AVX-512 kernel against upstream, then the generated kernel against the hand-written one. The second row isolates what specializing for width was worth once conventional tuning had already been applied.

Host	Comparison	Horizontal	Vertical
EPYC 9R05 (Zen 5)	Hand-optimized over upstream	+12%	+54%
	Generated over hand-optimized	+65%	+99%
Xeon 8180 (Skylake-X)	Hand-optimized over upstream	+44%	+97%
	Generated over hand-optimized	+6%	+4%

conventional tuning captures nearly all of the available gain and staging adds only a few points. The horizontal convolution is no faster because unlike on Zen 5, the shuffle port has equal throughput to making unaligned loads; there is no throughput benefit to using the ALIGNR strategy [35]. The vertical convolution is limited because the hand-vectorized implementation already approached the throughput ceiling of the system given the limitations on cache and memory bandwidth. About a third of all cycles on the Xeon host are stalled on memory, and the generated kernel retires fewer instructions in the same number of cycles. The staged kernel does remove work relative to the hand-written kernel, but that work was occupying issue slots the processor already had to spare.

Staging did not fundamentally change the scope of what could be implemented in this kernel. In other words, a sufficiently determined engineer could theoretically write 18 total width-specialized kernels by hand for AVX2 and AVX-512, and hand-tune the row count of each. However, staging achieved the same effect as that hand-optimization with significantly less engineering effort, and made it possible to programmatically construct a cost model that could be optimized with respect to the hyperparameters of the staged code (i.e. the kernel variant for the vertical convolution). This is the same coverage that projects like `dav1d` achieve across block sizes by writing out every case by hand, but generated in a much more maintainable way.

Overall, this combination of environmental and width specialization for the convolution kernels brought a substantial speedup to both the SpEED-QA feature in VMAF, and the end-to-end runtime of VMAF as a whole.

6.7 Composite Result and Upstreaming

We will consider the extent to which the VMAF patch series meets the goals of deployability laid out in §4.8, and the potential for upstreaming these changes to the official VMAF project.

6.7.1 Correctness

The first deployability criterion requires each specialization to produce bit-identical output to its unspecialized reference, or to bound errors within a justified tolerance. All the patches to integer features are bit-identical across the full range of input, as verified algebraically while writing the patches and empirically through the test corpus in Chapter 7. There are two changes in the floating-point features that reassociate floating-point arithmetic and thus may cause small discrepancies versus the baseline: the convolution optimizations and the CAMBI top- k quickselect.

The three iterations of the convolution patch (refactor, AVX-512 optimizations, and BuildIt-staged kernels) all make changes that could have a small numerical impact on the produced floating-point image: the refactor swaps a multiply + add for an FMA, the AVX-512 optimizations reorder the accumulation from top-down to middle-out, and the BuildIt-staged kernels allow for selection of different kernel variants with different association orders, though in practice the deployed variant is top-down. These errors exclusively impact the `speed_chroma_uv` scores in VMAF v1, as SpEED-QA is the only consumer of the floating-point convolution codepath in production versions of VMAF. The error is tightly bounded (on the order of 10^{-6} relative to the value being computed) because none of the discrepancies are behavioral changes to the algorithm in an algebraic sense; the errors appear exclusively because floating-point values are an imperfect approximation for the real numbers. Specific empirical data on these bounds is discussed in Chapter 7.

The other floating-point reassociation occurs with the top- k quickselect. Because the optimized kernel applies vectorized partitioning and does not write values equal to the pivot into either side, it writes elements into the left partition in a materially different order than the original scalar implementation. The values in that partition are later summed by another kernel in order to compute a mean. In practice, the denominator of the mean is so large that the small floating-point reassociation errors disappear, and CAMBI scores are identical between the master branch and the optimized tip for all videos in our test corpus within the precision that VMAF can display. It is nonetheless theoretically possible for floating-point reassociation here to cause a minute score discrepancy between the reference and patched versions of VMAF.

Because the error bounds are so tight on these features, the change in final VMAF score for a content never differs by more than 10^{-6} for scores ranging in $[0, 100]$. We deem this discrepancy acceptable, especially as VMAF already includes vectorized feature kernels that are not bit-identical to their scalar references.¹³ Should maintainers choose to nonetheless decline the small floating-point errors caused by the convolution refactor, it would be possible to drop those patches from the series and adopt the rest to attain a sizable speedup with bit-identical output everywhere. Therefore, we consider the output correctness criterion satisfied.

¹³ This applies specifically to the SpEED-QA covariance kernel, but the errors are so small that they do not affect the score in practice.

6.7.2 Dependencies

The second criterion is freedom from dependencies on new programs, compilers, or toolchains to build (or especially run) the specialized code. This is satisfied immediately for every commit in the series except the BuildIt staging commit. The BuildIt code is specifically designed to be easy to read, since its structure closely mirrors the second-stage code it generates; this alleviates the primary concern with new build dependencies, which is that they would be difficult to onboard or maintain. Nonetheless, should the maintainers refuse to adopt BuildIt as a build dependency, they can still adopt the rest of the patch series and retain the majority of the end-to-end speedup, just as in §6.7.1. Thus the criterion is mostly satisfied.

6.7.3 Incremental adoption

Third, the patch series must be incrementally adoptable such that accepting the speedups does not mean swapping out the internals of dozens of core functions all at once, both to reduce the probability of cascading bugs and to make it easier for maintainers to adapt and review. Because the patch is split up into independent commits, each of which is standalone and can be adopted independently, this criterion is satisfied.¹⁴

6.7.4 Fallback paths

The specializations must support the full range of inputs that the original kernels did, or otherwise must have fallback paths for when inputs fall out of their accepted domains. The only specialization in the series that depends on a property of the input is the staged convolution; all other specializations are bound statically at build time. For the staged convolution, a family of kernels for odd sizes up to `MAX_BUILDIT_AVX_CONV_SIZE == 17` are generated at build-time, and only if the runtime width matches one of the available specialized widths is that kernel dispatched. By default, the existing AVX2/AVX-512 code for general widths applies and ensures correct output. Therefore, the criterion is satisfied.

6.7.5 Maintainability

Finally, the patches must be comprehensible to and reviewable by the maintainers of the project. This is where the patch set is weakest: many of the specializations are difficult to understand without extensive supplementary documentation beyond mere inline comments. The ADM byte-packing patch, for example, looks very obscure upon casual inspection, and the proof that it is indeed a correct algebraic specialization of the original program is subtle. Many optimizations are also fragile to parameter changes, and may need to be nontrivially redesigned or restructured if the values of the weights for a particular kernel were updated in the future. Lastly, the BuildIt convolution generator requires maintainers to use the mental

¹⁴ The three convolution commits depend sequentially on one another, but since it does not make sense to use e.g. the BuildIt-staged convolution without also using the AVX-512 generic fallback path, we deem this acceptable.

model of a higher-order program, which may be difficult without prior experience. These are all barriers to maintainability, and thus potentially to upstreaming.

We argue that although the patch set is more difficult to review and maintain than typical scalar C code, that is a property of high-performance intrinsic code in general. Even within VMAF, many of the existing AVX-512 kernels are sparsely documented and require line-by-line tracing against their scalar counterparts to be understood or verified for correctness. Based on the relative complexity of the SIMD acceleration kernels contributed by open-source developers that have already been merged into VMAF, we believe that the higher-than-usual review and maintenance cost for the patch set will not be a blocker to mergeability. There are at least some commits within the patch set that are easier to review than the others, so it may be possible that these are merged sooner while the larger changes await more thorough review.

Some changes that bring too little speedup for too much complexity, as deemed by the maintainers, may be denied in the process. However, we generally believe that most of the patch set carries only a one-time review cost and relatively little further maintenance cost, especially since most of the features that the patch set targets have had their algorithms remain relatively unchanged since they were introduced. Therefore, we consider the maintainability criterion mostly satisfied, with the caveat that some commits may need to be reworked or justified further to be accepted.

In total, all the criteria are satisfied reasonably well, so we anticipate that at least a large majority of the patches developed in this thesis can indeed be upstreamed into the official open-source VMAF project. Although the upstreaming work will be outside the scope of this thesis, we intend to submit these changes to the VMAF maintainers in the near future.

Chapter 7

Evaluation

This chapter evaluates the end-to-end and component-wise performance improvements that the specialization patch series discussed in Chapter 6 brings to VMAF across a variety of inputs and hardware configurations. Although many of the optimizations are sensitive to microarchitectural details and input format, the patches bring strong improvements in end-to-end throughput for VMAF across the board.

7.1 Experimental Setup

The VMAF patch set exclusively targets AVX2 and AVX-512 codepaths, so the changes were tested on high-end x86 microprocessors. I ran extensive benchmarks on three machines: a consumer-grade desktop PC, a CPU-optimized EC2 instance (m8azn.xlarge), and an older NUMA enterprise server. These machines are as a representative sample of the kinds of systems VMAF typically runs on.

The desktop PC and EC2 instance are both based on the Zen 5 microarchitecture, and therefore produced similar benchmark results in every configuration. The major microarchitectural difference between the two is the additional cache available on the desktop PC: because it is an X3D chip, it includes 96 MiB of total L3 cache, as opposed to the EPYC's 20 MiB. The server is based on the older Skylake-X microarchitecture. Both feature native 512-bit datapaths, but Zen 5 offers much higher throughput for shuffles, permutes, shifts, conversions, and bitwise operations. By comparison, Skylake-X overprovisions multiply and load/store throughput. For instance, both Zen 5 and Skylake-X can execute 2 FMA and 2 512-bit loads per cycle, but Zen 5 can perform 4 in-lane shuffles per cycle while Skylake-X can perform only 1 [35, 36, 37]. These differences change the performance characteristics of many kernels. Many optimizations choose to replace more expensive operations with less expensive ones based on a cost model that tries to capture trends across several modern AVX-512 implementations. Deviations from this idealized cost model can make a particular optimization much less effective on one microarchitecture than another. Note that it is possible to instead write kernels that are specialized for a particular microarchitecture rather than an abstract cost model to achieve further speedups; we discuss this approach in §8.2. However, evaluating

these patches on several microarchitectures allows us to verify that our optimizations indeed reduce work on the ISA level and will thus provide a speedup on future hardware without further engineering effort.

Table 7.1: Measurement hosts. The Skylake-X machine is generally approximately $3\times$ slower than the Zen machines for an equivalent VMAF workload. This section will compare performance figures before and after the patch set was applied independently on each host.

Host	Microarchitecture	Class	Threads	Memory
Ryzen 9 9850X3D	Zen 5 (2024)	Desktop	8C/16T @ 5GHz	32GiB
EPYC 9R05	Zen 5 (2024)	EC2 m8azn.xlarge	4 vCPU @ 5GHz	16GiB
Xeon Platinum 8180	Skylake-X (2017)	4-socket NUMA	112C/224T @ 2.5GHz	3TiB

The corpus is composed of two contents from the SVT Open Content Video Test Suite, which features a variety of natural scenes with details that are generally challenging for encoders to capture, as well as Sintel, an open-source animated movie with visual features typical of computer-generated media [38, 39]. We cut each video to a short clip of between 120 and 240 frames, requantize the clip to both 8-bit and 10-bit (with the exception of Sintel, which is natively 8-bit), then resample each bit-depth to five typical resolutions: 480p, 720p, 1080p, 1440p and 2160p. Each of these variants is encoded once to create the VMAF “reference” clip, and encoded again to create the “distorted” clip. Note that this is not a sound approach to evaluating the perceptual quality of the lower-resolution encodes due to VMAF’s scale sensitivity; however, it properly models the pixel distribution of input sources at each scale and is thus sufficient for performance benchmarking. We chose this suite of resolutions and bit depths to cover the most common resolutions for running VMAF on SDR and HDR video (which are typically 8-bit and 10-bit respectively).¹ The final corpus has 25 unique clips.

We test both VMAF v0 and v1 integer models on the aforementioned test corpus. There are many model variants in both v0 and v1, but all models of the same version family compute the same features and thus have near-identical performance characteristics. Our v0 model (`vmaf_4k_v0.6.1`) computes ADM, motion and VIF at four scales. The current v1 model (`vmaf_v1.0.16_1d5h_2160`) computes ADM, motion, CAMBI, and SpEED-QA chroma features. Performance result from both models are reported throughout because they exercise different parts of the series and, as §7.4 shows, respond differently on different hardware.

Each configuration runs single-threaded, pinned to a fixed core with a warm page cache, with one warmup run and several timed repetitions. Inputs were sized such that they fit entirely in the page cache. This configuration minimizes run-to-run variance and isolates the benchmark from paging effects. Note that VMAF parallelizes strictly across different frames, so multithreading provides a speedup that is largely independent of any kernel-level

¹ Note that VMAF does not officially support HDR, though Netflix has an internal HDR-VMAF model. Regardless, some SDR videos are stored in higher bit depths for archival purposes, and VMAF is often used to evaluate these encodes.

optimizations (each of which apply only to a single thread). We discuss the performance characteristics of a more typical multithreaded workload without these controls in §7.4.2 and Appendix B.

Every binary is statically linked, built once on the workstation with gcc 15.2, and run unmodified on all three hosts, using VMAF’s standard release build configuration: `-O3` for all translation units, plus `-mavx512f` and related flags for those kernels gated behind AVX-512 feature detection. The same code paths are exercised by all hosts: although Zen 5 supports more AVX-512 extensions than Skylake-X, the AVX-512 feature detection in VMAF depends only on AVX512F/DQ/CD/VL/BW, which is common to both microarchitectures. Therefore, any differences in performance between two variants are a result of differences in microarchitecture or input configuration.

Every performance figure reported is a speedup, which measures the relative increase in throughput for a particular task. For instance, a 100% speedup means the throughput of a particular operation is now two times greater; that is, it can process two times as much input in the same amount of time. For a fixed workload, speedup is computed as $t_{\text{base}}/t_{\text{new}} - 1$, where t_{base} is the original runtime and t_{new} is the optimized runtime.

Speedups were measured via a bootstrapped ratio estimator based on the two-sample Hodges-Lehmann statistic, which allows us to construct confidence intervals for end-to-end and kernel-specific speedups despite differences in relative noise levels within different kernels. A consequence of this approach is that reported speedup figures are not ratio-of-minima but rather medians, which absorb runtime overheads and noise external to the program to a small extent. We deemed this acceptable because the difference between the ratio-of-minima and median was generally under 1%. Unless otherwise noted, all presented speedup ranges are 95% confidence intervals excluding the 2.5% upper and lower tails. Within this chapter such an interval is quoted as a single $\pm$ figure taken from the wider of its two arms. This is a weaker statement than the interval itself, since it widens the narrower side, but the intervals are close to symmetric in practice and the symmetric form is far easier to read down a column. Appendix B reports the exact upper and lower bounds for every configuration. Additionally, to avoid misattributing performance changes caused by code layout or intra-translation unit effects to our optimizations, we only consider a kernel-level speedup to be the result of a successful optimization if its CI does not overlap a $[-4\%, +4\%]$ band. There is no such requirement for end-to-end speedups: any CI that does not intersect 0 is considered a true optimization. A full explanation and justification of the speedup estimation procedure is available in Appendix A.

Table 7.1 lists the three machines and their performance specifications. The two Zen 5 chips run VMAF at similar speeds, while the Skylake-X server is roughly three times slower per core on the same work. That ratio is a property of the baseline code rather than of the patches, and it is worth keeping in mind when reading the per-host speedups below: the same relative improvement is a much larger absolute saving on the slowest machine.

7.2 Output Parity

Sacrificing correctness for performance is often unacceptable in production software environments. Therefore, a key constraint on the VMAF patch set is that it must not produce materially different VMAF scores than the original implementation in any configuration. By enforcing this, we ensure that the optimizations strictly fall under the realm of specialization; that is, they reduce the work needed to compute the output by changing the *formulation* of the algorithm rather than its behavior.

All of the optimizations made to integer features (VIF, ADM, and motion) were engineered to produce bit-identical outputs on the latest patch versus on the master branch, as verified across our entire test corpus. Due to the nonassociativity of IEEE 754 addition, as well as precision differences between fused and unfused floating-point multiply-adds, some of the optimizations to floating-point features (CAMBI and SpEED-QA) result in tiny differences in computed feature scores that propagate to the computed VMAF scores.

Because the v0 model uses only integer features, it produces bit-identical values in the original and optimized binaries for all intermediate features and frames. For the v1 model, the floating-point feature optimizations introduce some small changes to the produced scores. The maximum relative errors for intermediate features and per-frame scores were on the order of 10^{-6} and 10^{-7} respectively across our corpus. However, because VMAF aggregates per-frame results into its final scores with a mean and only reports scores to 6 decimal places, the final reported score for the v1 model was identical between the unoptimized and optimized binaries for 13 of the 25 videos in the test corpus. In the other 12 videos, the reported scores differed by exactly 10^{-6} due to rounding in VMAF’s output presentation. Since the VMAF models we tested produce scores in the range $[0, 100]$, this is a typical relative error of approximately 10^{-8} .

The mechanism behind the floating-point differences is described in more detail in §6.7. We deemed the correctness properties of the final patch acceptable because the deviations are bounded, confined to the floating-point paths, and can be traced to conscious choices within the implementations that do not materially change the algorithm.

7.3 Per-Kernel Results

Figure 7.1 and Table 7.2 give the speedup of each specialized kernel, together with the axis of specialization it exercises. Every kernel here is an instance of structural specialization in the sense of Chapter 4, embedding a fact about the problem into the structure of the program rather than passing it as a parameter; the axis records which kind of fact was embedded. Shape and value range are both forms of input specialization, since each fixes a property of the input, and they sit alongside algebraic and environmental specialization as the three classes the series exercises. The axis is more informative than the ordering, though not because one class dominates. Input specialization pays consistently, never below +55% on any host; algebraic specialization pays modestly but on the kernels that carry the most runtime; and environmental specialization is by far the most variable, holding both the single

largest result in the series and several of the smallest.

Table 7.2: Per-kernel speedup by host, with the specialization axis each exercises. *Weight* indicates how much of total runtime the kernel accounts for in the configuration where it matters most: *** is more than 7.5%, ** between 3.5% and 7.5%, * below 3.5%. *Base* records whether the baseline implementation was already vectorized (V) or was scalar C (S). Entries marked † fail the $\pm 4\%$ attribution criterion of §7.1 and cannot be distinguished from incidental recompilation effects on that host. The three kernels below the rule are mechanical widenings of the existing AVX2 logic to 512 bits, exploiting no structural property of the data; they are discussed in §7.3.

Kernel	Axis	Weight	Base	Ryzen	EPYC	Xeon
CAMBI anti-dithering	environment	***	S	+1178%	+1089%	+626%
CAMBI top- <i>k</i> select	value range	**	S	+397%	+381%	+320%
CAMBI boolean DP scan	value range	**	S	+303%	+306%	+415%
Convolution, vertical	shape (staged)	***	V	+149%	+220%	+60%
Convolution, horizontal	shape (staged)	**	V	+89%	+80%	+55%
CAMBI c-values	environment	**	V	+62%	+54%	+82%
ADM DWT, 8-bit	algebraic	**	V	+27%	+38%	-5% [†]
CAMBI mode filter	environment	*	V	+12%	+21%	+77%
VIF statistic, 16-bit	algebraic	***	V	+25%	+25%	+50%
Motion score, 8-bit	environment	*	V	+28%	+11%	+23%
VIF statistic, 8-bit	algebraic	***	V	+4% [†]	+4% [†]	+28%
CAMBI derivative	(naive widening)	*	V	-1% [†]	+15%	+57%
CAMBI decimate	(naive widening)	*	V	-8%	+38%	+34%
CAMBI histogram range	(naive widening)	*	V	-8% [†]	-15%	+19%

The weight column is what connects this table to §7.4. A kernel with one star can improve by a factor of four and be invisible end to end; the +1178% on anti-dithering matters precisely because that kernel is 13% of runtime in the configuration where it runs at all. Conversely the VIF statistic is the single largest consumer in the series, at 66% of runtime under the v0 model at 10-bit, which is why a +25% improvement there significantly reduces v0 runtime, while larger speedups in some of the smaller kernels have a less visible effect.

The three kernels that demonstrated the largest speedups had no vectorized implementation in the baseline at all, which is not surprising. These are the places where “traditional” performance engineering still has room to create large speedups on its own. We will discuss these kernels in §7.3.1 and §7.3.2.

Many of the rows disagree across the hosts due to microarchitectural differences between them. For instance, the ADM discrete wavelet transform gains +27% on the Ryzen and +38% on the EPYC, and nothing on the Skylake-X part. The optimization avoids widening 8-bit values into 16 bits by emulating 16-bit operations with pairs of 8-bit operations, which reduces shuffles, permutes, and conversions at the cost of two additional multiplication instructions. This is effective on the Zen hosts, where 35% fewer total instructions leads to a 27-38% speedup. On the Xeon chip, however, the kernel is much more sensitive to memory latency and nonuniform locality due to the NUMA configuration. Performance counters reveal that

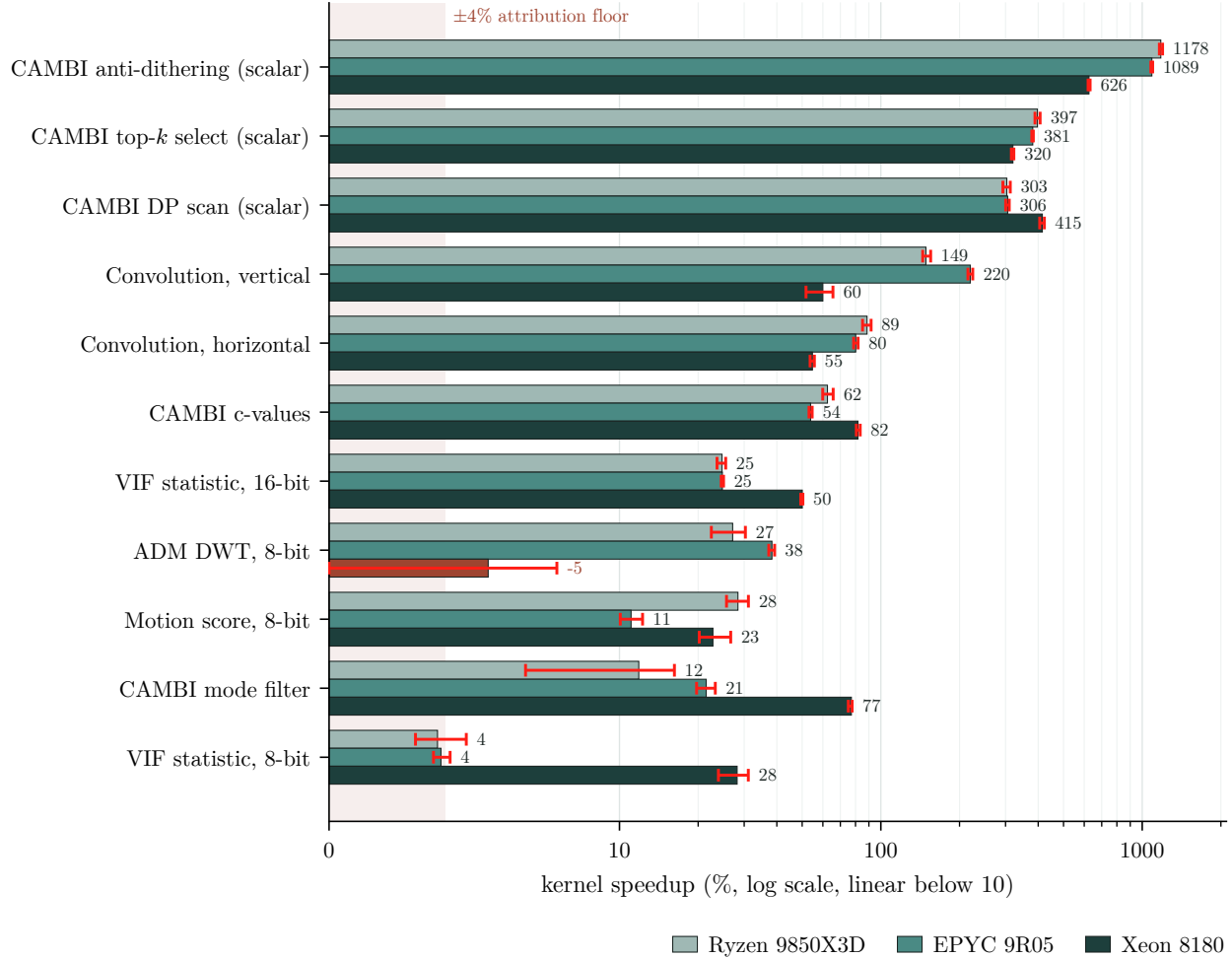


Figure 7.1: Per-kernel speedup, upstream master to branch tip, on all three hosts. It is a log axis, linear within 10% and logarithmic outside. Every bar is drawn by magnitude: a kernel that got slower is shown in the loss color with its negative figure on the label. Whiskers are the 95% bootstrap confidence intervals of §7.1. The shaded band marks the $\pm 4\%$ attribution floor. The only kernel plotted here that exhibited a slowdown on any host, the 8-bit ADM transform, overlaps with the attribution floor; its performance thus cannot be distinguished from recompilation effects. The same is true of the 8-bit VIF kernel on both Zen 5 hosts, although they are positive. Kernels marked (*scalar*) had no vectorized implementation in the baseline (§7.3.2).

although Xeon retires 15% fewer uops in the updated ADM kernel, it performs 50% more cache line-splitting loads, while total cycle count remains effectively unchanged. This pattern is consistent with the wider loads further exposing the critical load-to-store path to cache and NUMA latency, offsetting the saved execution work. This could also explain why the ADM speedup measurement has such a wide confidence interval of $[-7.8\%, +5.8\%]$ on Xeon; the nonuniformity of memory access timings across runs dramatically increases measurement noise.

The 8-bit and 16-bit VIF statistics invert the microarchitectural preference. The 8-bit optimization applies some structural tricks to remove about a quarter of the total number of shuffle operations used by the kernel. Since shuffles have the same throughput as multiplies on Zen 5, those two hosts were bottlenecked by ALU throughput and gained nothing from this optimization. On Skylake-X, shuffles can only run on a single port and have half the throughput of multiplies, so this optimization won a 28% speedup. The 16-bit optimization has a similar story: it reduces some conversion overhead and replaces several `vpmulld` (32-bit multiply low) with `vpmaddwd` (16-bit multiply-accumulate to 32 bits). These instructions have the same throughput on Zen, but Skylake-X has $2\times$ more throughput for `vpmaddwd` [35]. Zen benefits only from the overhead reductions and gains 25%, while Skylake-X also benefits from much higher ALU throughput and gains 50%.

These results are the clearest instance in the series of the idealized cost model of §7.1 departing from a real machine. Each of these kernels was written to replace an expensive operation with a cheaper one under an abstract cost model, and each does *generally* less work: both the optimized DWT and the VIF statistic pack data more intelligently to issue fewer conversions and shuffles than the original implementations. What differs between the hosts is not whether the work was removed but whether the removed work was the constraint. The specialization is doing what it was designed to do in all cases; the idealized cost model may simply rank two operations differently than any one particular machine does.

The decimation pass is unique in that its sign flips between two parts of the *same* microarchitecture, which the cost-model argument above cannot explain. The mechanism is almost certainly cache capacity rather than execution: the desktop Ryzen carries 96 MiB of stacked L3 against the EPYC’s 20 MiB, and a 4K 10-bit plane is roughly 16.6 MB. The decimation working set therefore fits in the Ryzen’s last level and spills to memory on the EPYC. Widening the loads helps only when the loads are actually waiting on memory, so the same change buys bandwidth on one part and pays for wider shuffles for nothing on the other. This is a hypothesis rather than a measurement; what is measured is that the disagreement is reproducible and does not follow the microarchitecture.

7.3.1 Vectorization alone

There are four kernels in Table 7.2 that involved vectorization alone and little to no specialization effort: the anti-dithering filter, and the derivative, histogram and decimate kernels.

The anti-dithering filter brought the biggest speedup of any kernel in the patch set because it was originally a pure scalar loop, as the original pure C implementation. This was the only

low-hanging fruit within VMAF, and it required little specialization-specific insight beyond the observation that a 2×2 filter could be vectorized row-wise without overwriting pixels until the last time they were read.

By contrast, the three kernels below the rule in Table 7.2 show the opposite effect. Those kernels were direct AVX-512 ports of existing AVX2 implementations that did not exploit any structural property of the data. All three mechanically widen the 256-bit logic to 512 bits, doubling the vector width and changing little else. The results are correspondingly incoherent, with no sign that holds across hosts and two outright regressions: the decimation pass loses 8% on the Ryzen while gaining 38% on the EPYC, and the histogram range update loses on both Zen 5 parts while gaining 19% on the Xeon. Every specialized kernel above the rule, by contrast, is positive on all three hosts (or zero in the case of 8-bit VIF on Zen 5 and ADM on Xeon, as discussed in §7.3).

These results illustrate one of our central claims: extracting maximum performance out of an algorithm usually requires more than surface-level analysis of a computational problem and its execution environment. For the anti-dithering filter, naively applying vectorization was an easy performance win over existing scalar code. However, vectorization is fundamentally limited as a performance engineering tool on its own, and as we see in the cases of the decimate and histogram kernels, it cannot improve performance beyond a certain point. Widening a 256-bit kernel to 512 bits is the obvious thing to do and requires no analysis of the data at all; done on its own it produces results that are as likely to be negative as positive. §7.6 demonstrates the same result with an independent experiment, in which the two implementations are dispatched from within a single binary.

7.3.2 Composed vectorization and specialization

Beyond the anti-dithering filter, the other two kernels that were unvectorized prior to the patch were the top- k selection and the boolean scan. These, likewise, started as scalar C code, and part of their improvement is therefore ordinary vectorization: work any competent performance engineer would have done, with or without the framework this thesis proposes. However, the specialization strategies we have discussed largely work orthogonally to vectorization, so applying both techniques together yields a much bigger speedup than either alone.

For the boolean scan, we can see this result directly by comparing to an unspecialized (but vectorized) alternate implementation. The scan computes a running count over a window of derivative values. By inspecting the surrounding code, we can see that these derivatives are always the outputs of equality tests, so they are provably zero or one even though they are declared as 16-bit integers within the C type system. The specialized implementation exploits this by placing each derivative into a separate nibble within a 512-bit ZMM register to run sixteen 8-element prefix sums in parallel, and then combining them into a 128-element prefix sum block in a second stage. To separate that specialization from the vectorization underneath it, I wrote a second AVX-512 implementation of the same kernel that does *not* use the invariant: it widens the derivatives to 32-bit lanes and performs a standard Hillis-Steele

scan using `vpalignr`.² Each implementation is highly efficient given the assumptions it is allowed to make about its input. The two kernels were compiled into otherwise identical binaries, and they produce identical output on every frame of the corpus.

We also compared the performance across two iterations of the vertical and horizontal convolution kernels within the patch set. The first iteration was a straightforward improvement to the cache-efficiency and FPU throughput of the existing vectorized implementations that did not make any assumptions about the size of the kernel. The second used multi-stage compilation to generate width-specialized kernels that further improved cache-efficiency and reduced overhead relative to the first step.

Table 7.3: Vectorization and specialization, separated, at 4K 8-bit under the v1 model. *First step* is the conventional performance engineering that precedes the specialization, and *compounded* is the full series against upstream master; both are measured against the same baseline, and *specialization* is what remains once the first step is divided out. What counts as the first step differs by kernel. For the scan it is a second AVX-512 implementation written for this experiment that does not use the boolean invariant, swapped into an otherwise identical binary. For the convolution passes it is the hand-written AVX-512 implementation from an earlier commit in the patch series before staging was applied. In each row the larger of the two decomposed steps is set in bold.

Kernel	Host	First step	Specialization	Compounded
CAMBI boolean DP scan	Ryzen 9850X3D	+194%	+36%	+300%
	EPYC 9R05	+203%	+34%	+307%
	Xeon 8180	+224%	+55%	+403%
Convolution, vertical	Ryzen 9850X3D	+65%	+50%	+147%
	EPYC 9R05	+54%	+99%	+208%
	Xeon 8180	+97%	+4%	+105%
Convolution, horizontal	Ryzen 9850X3D	+17%	+63%	+90%
	EPYC 9R05	+12%	+65%	+84%
	Xeon 8180	+44%	+6%	+52%

Table 7.3 gives the result for three kernels on all three hosts. Notably, which step dominates depends on the kernel and host.

For the boolean scan, plain vectorization is by far the larger step on every host. It is worth +194% to +224% against the scalar baseline, while the value-range specialization adds a further +34% to +55% on top of that. This is a real effect and a consistent one, holding to within two points across the two Zen 5 parts, but it is a refinement of the vectorization rather than the thing that makes the kernel fast. Anyone who vectorized this loop at all would have captured most of the available gain.

² A potential future extension of this work could shrink the storage type of the derivative values to an 8-bit array or a bit-vector; I opted not to do so here to keep the patch small and avoid touching the scalar fallback paths.

The ceiling is structural and visible in the source. Both implementations write 32-bit results and accumulate into a 32-bit array, so the widening, the base-element addition and the stores are identical work per element in both. That common tail cannot be simplified by exploiting the boolean invariant, and it bounds what the specialization can contribute no matter how efficient the scan itself is.

The convolution inverts this on the Zen 5 parts. There the baseline was not scalar but a mature AVX2 implementation, and the series improves it twice: first with hand-written AVX-512 intrinsics, which is conventional performance engineering, and then with staged generation, which is not. On the horizontal pass the conventional step is worth only +17% and +12% while staging adds +63% and +65%, so the specialization is by a wide margin the larger contributor. The vertical pass splits the difference, with staging worth +50% on the Ryzen and +99% on the EPYC against conventional steps of +65% and +54%.

The Xeon is the exception, and it is the clearest measurement in this thesis of a specialization exhausting its headroom. There the conventional step captures nearly everything, +97% on the vertical pass and +44% on the horizontal, and staging adds only +4% and +6%. As we discuss in §6.6.4, the primary change in the width-specialized horizontal convolution kernel is the use of `vpalignr` instructions instead of unaligned loads; `vpalignr` has $2\times$ greater throughput on Zen 5, but equal throughput on Skylake-X, so performance is flat for Skylake-X on that kernel. For the vertical kernel, the hand-optimized kernel already stalls on memory for most of its runtime, and so the reduced work in the specialized kernel does not help because both kernels must wait for the same total amount of data to pass into L1. There is no way around these memory bottlenecks; they are intrinsic to the microarchitecture, so the specialization is mostly flat despite significantly reducing each kernel’s total work.

We find that vectorization and specialization are orthogonal and compose well with each other: where the baseline is scalar one vectorizes first and specializes afterwards, and where it is already tuned SIMD the specialization goes on top. Which of the two is larger is not a property of the technique but of the kernel and the machine. The key insight is that specialization can bring speedups even *after* conventional performance engineering is complete, and that its benefits do not disappear against a strong baseline.

7.4 End-to-End Results

Table 7.4 and Figures 7.2 and 7.3 give whole-program wall clock for all four configurations.

The headline result is the v1 model at 8-bit, where the series is worth +41.1% on the Ryzen, +43.4% on the EPYC and +23.4% on the Xeon. This is the configuration that matters in practice: v1 is the current VMAF model and 8-bit the more common deployment. In throughput terms, the EPYC goes from 22.8 to 32.7 frames per second on a 4K stream, single-threaded.

The more interesting feature of the table is that the hosts rank in opposite orders depending on the model. Skylake-X gains far more on v0 (+16.7% and +26.0%, against +5.5% and +15.9% on the Ryzen and +5.2% and +14.2% on the EPYC) and less on v1. This follows

Table 7.4: Whole-program wall clock, upstream master to branch tip, for a 4K content. Wall-clock columns are the fastest of twelve timed repetitions; the speedup is the bootstrap median with a 95% confidence interval, computed as described in §7.1. The 8-bit v0 row is the weakest configuration on both Zen 5 parts: it is the one whose result rests almost entirely on a single kernel, the 8-bit VIF path that neither Zen 5 host rewards.

Host	Configuration	Baseline	Optimized	Speedup ($\pm 95\%$ CI)
Ryzen 9850X3D	8-bit, v0	15.92 s	15.08 s	+5.5% ± 0.3
	8-bit, v1	9.15 s	6.48 s	+41.1% ± 0.8
	10-bit, v0	13.74 s	11.82 s	+15.9% ± 0.9
	10-bit, v1	6.17 s	5.13 s	+20.2% ± 1.3
EPYC 9R05	8-bit, v0	16.75 s	15.90 s	+5.2% ± 2.0
	8-bit, v1	10.53 s	7.34 s	+43.4% ± 0.3
	10-bit, v0	11.06 s	9.63 s	+14.2% ± 3.4
	10-bit, v1	5.32 s	4.33 s	+22.4% ± 0.4
Xeon 8180	8-bit, v0	51.68 s	44.27 s	+16.7% ± 0.3
	8-bit, v1	29.80 s	24.14 s	+23.4% ± 0.4
	10-bit, v0	47.22 s	37.46 s	+26.0% ± 0.1
	10-bit, v1	21.62 s	18.92 s	+14.3% ± 0.1

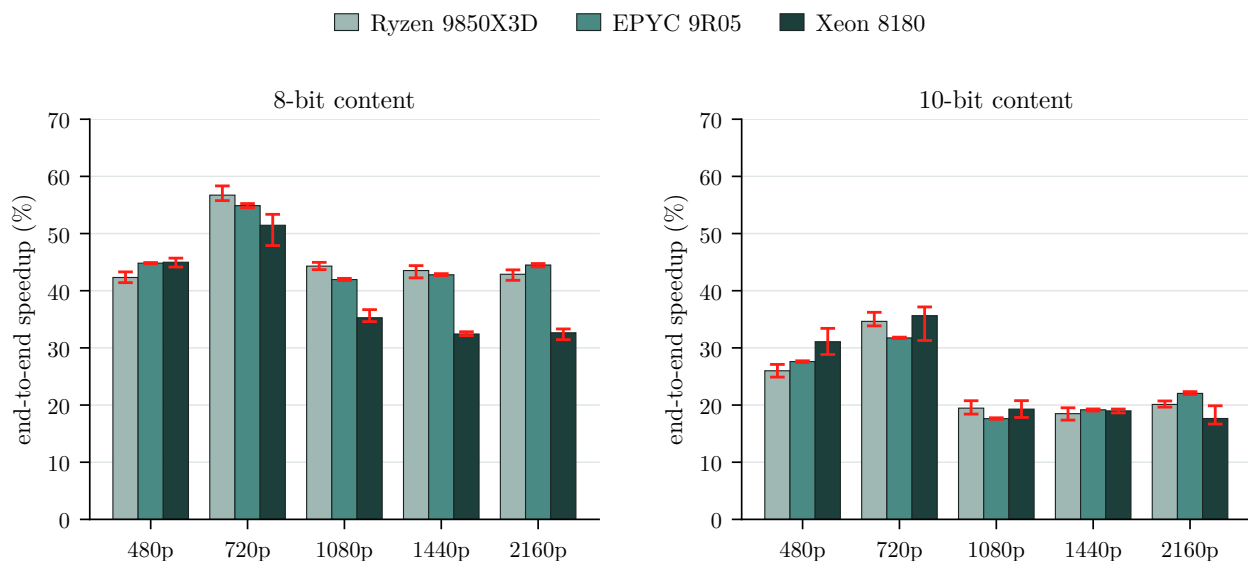


Figure 7.2: End-to-end speedup from upstream master to the branch tip under the v1 model, across five resolutions and all three measured hosts. Bars are the mean over three contents (smithy, waterfall, sintel); whiskers are 95% bootstrap confidence intervals on that mean, resampling the timed runs within each content, and so measure sampling uncertainty rather than content-to-content spread. The per-content values are tabulated in Appendix B. sintel is 8-bit only, so the 10-bit panel averages two contents. Measured with the `vmaf` CLI on raw YUV at 120 frames per resolution; Appendix B gives the per-content values and the decode-inclusive results.

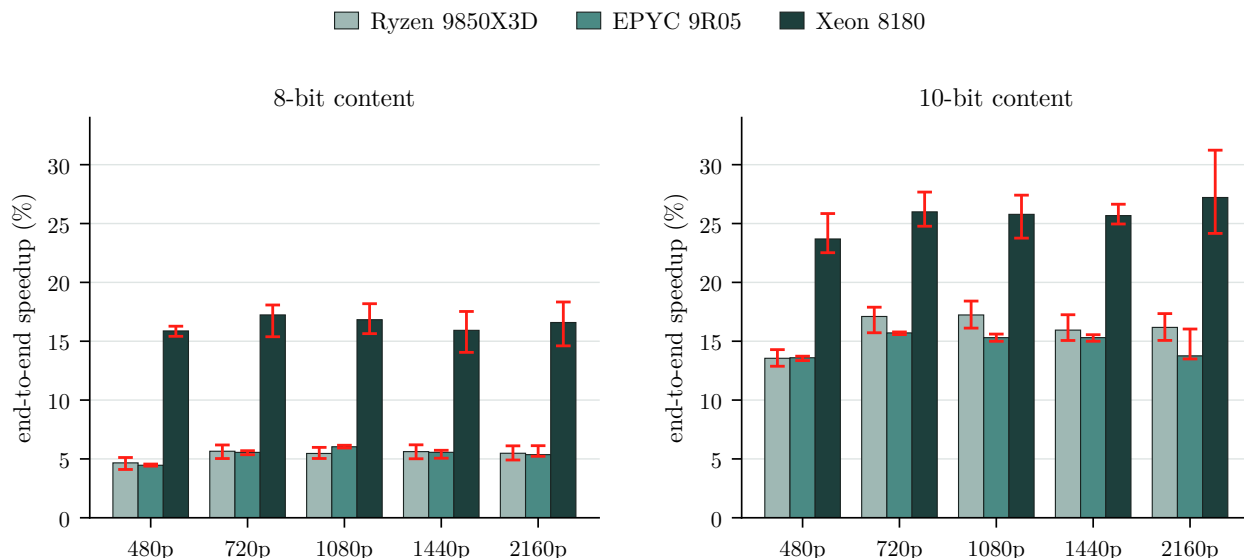


Figure 7.3: End-to-end speedup under the v0 model, otherwise as Figure 7.2. The y -axis runs to 34% rather than 70%: the gain comes almost entirely from VIF and is roughly a third the size. The ordering of the hosts inverts, the Xeon leading here and trailing under v1, for the reason given below.

directly from §7.3. The v0 model is dominated by VIF, whose specialization removes loads and therefore favors the load-limited host, while v1 adds CAMBI and the convolution, whose specializations remove arithmetic and favor the host with the arithmetic bottleneck. The spread across hosts is the cost model expressing itself at whole-program scale. The two Zen 5 parts agree closely in every configuration here (never differing by more than 3 points), which is the expected result and makes the Skylake-X divergence harder to dismiss as scatter.

The weakest configuration, 8-bit v0 at +5.2% on the EPYC and +5.5% on the Ryzen, is weak for a reason visible in the same analysis: it is the one configuration in which the dominant feature is VIF’s 8-bit path, the single specialization that Zen 5 does not reward.

7.4.1 Per-feature results

Figure 7.4 shows which features dominate runtime for each VMAF model.

CAMBI falls from roughly a third of runtime to well under half of that and the convolution shrinks similarly, but ADM occupies about 38% of the baseline and is essentially unchanged, because only its first-level 8-bit transform was specialized and that is a small part of the feature. After the series, ADM is the largest single consumer of runtime on all three hosts, and a further round of work on this codebase should begin there.

The ADM transform also gives the clearest illustration of why end-to-end gains are so much smaller than kernel gains. On the EPYC at 4K 8-bit under v1, the kernel improves by +38%; it occupies about 7% of wall clock; the whole ADM feature therefore improves by only +6.0%,

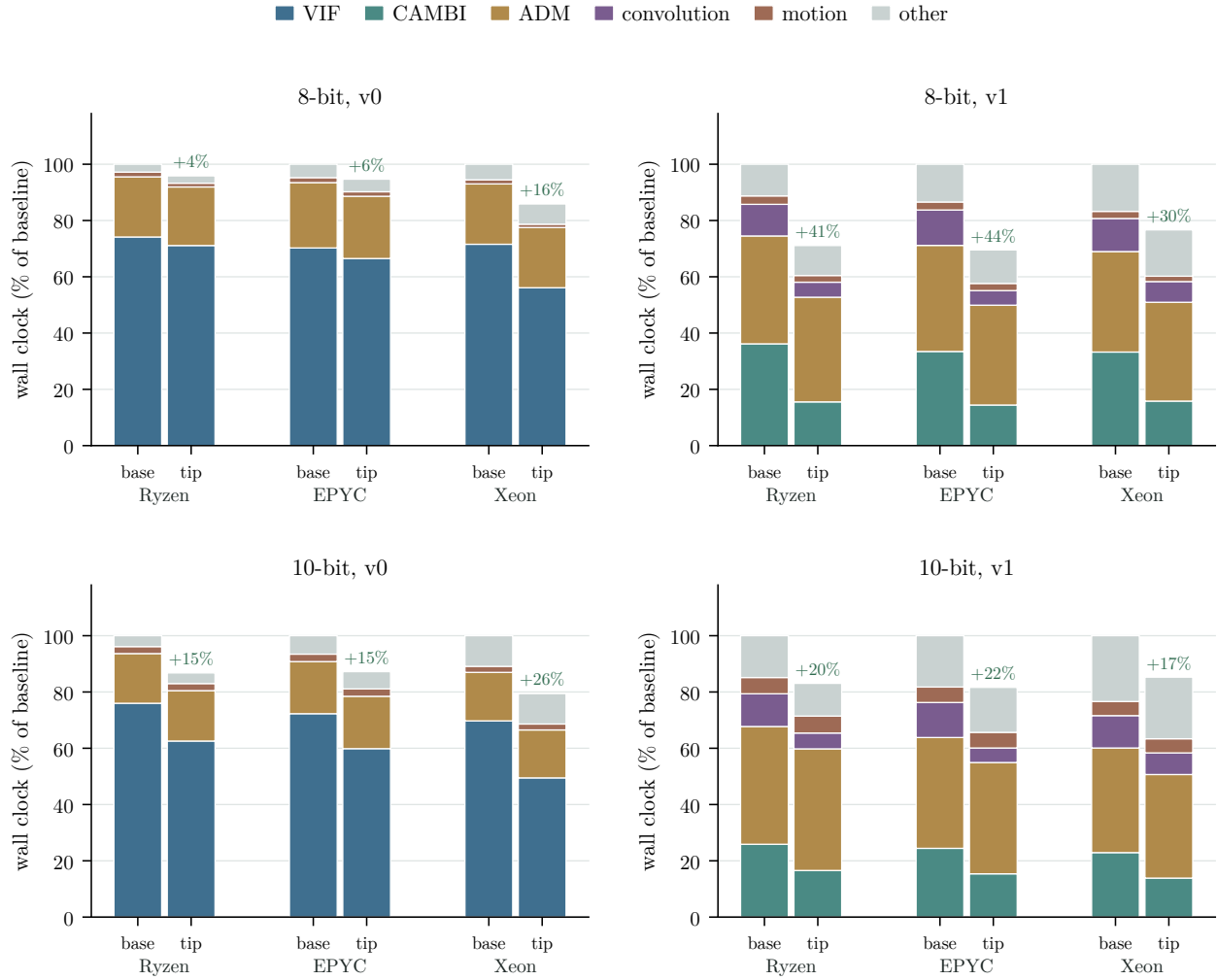


Figure 7.4: Wall-clock composition at 4K across all four configurations, normalized to each host’s own baseline within each panel because the Skylake-X part is roughly three times slower in absolute terms. The four cells decompose almost disjointly, which is why the same series moves them by such different amounts. v0 is roughly 70% VIF and contains no CAMBI or convolution, so only the VIF specialization can act on it; v1 replaces VIF with CAMBI and the convolution and gives the series far more surface. Bit depth then matters within v0: at 10-bit the VIF work runs on the 16-bit path, which the series does optimize, so v0 gains two to four times more there than at 8-bit, where the largely untouched 8-bit path dominates. ADM is barely touched anywhere at this resolution and comes to dominate what remains. Percentages above the tip bars are end-to-end speedups from the same run as the per-feature seconds, which uses a shorter clip than Table 7.4 and so does not match it exactly.

since the rest of it is untouched; and the commit introducing it moves end-to-end wall clock by +2.4%. All three levels of measurement agree, and the attenuation at each step is what Amdahl’s law predicts. Across every configuration measured, the per-feature savings sum to the measured change in wall clock to within one percentage point, which is what admits treating the per-kernel attribution as an explanation of the end-to-end result rather than as an unrelated measurement.

7.4.2 Realistic end-to-end with decoding

We also measured the end-to-end speedup of the series when the `libvmaf` filter is invoked through FFmpeg, which is often how it is used in practice.³ These results therefore include the time it takes for FFmpeg to decode the input video and feed it through VMAF, so they are not directly comparable to the pure VMAF results from above. These run multithreaded for both decoding and execution, pinned to four dedicated physical cores on each host so that the thread count is the only quantity varying between the two columns and so that hosts with very different core counts remain comparable. No NUMA control or page cache warmup is applied. The speedups here are lower than the pure VMAF results because the FFmpeg wrapper adds various overheads, most notably video decoding.

Table 7.5: End-to-end speedup of the whole `ffmpeg` invocation on the Ryzen 9850X3D, with `libvmaf` given one thread and then four. Affinity is held at four dedicated physical cores throughout, so the only quantity changing between the columns is `n_threads`; decoding is free to use those cores in both. Measured on smithy with H.264 inputs, 120 frames. The corresponding figures for the other hosts, contents and codecs are in Appendix B.

Resolution	Depth	v0 model		v1 model	
		1 thread	4 threads	1 thread	4 threads
1080p	8-bit	+6.5%	−0.0%	+38.2%	+12.7%
	10-bit	+15.6%	+10.3%	+17.4%	+4.7%
2160p	8-bit	+6.5%	+3.0%	+36.3%	+14.1%
	10-bit	+12.9%	+5.8%	+18.7%	+6.0%

Table 7.5 shows that the speedup falls in every configuration once `libvmaf` is given more threads, and the reason is Amdahl’s law with a growing denominator rather than anything going wrong in the kernels. Timing the runs themselves shows the pipeline ceasing to scale well before the cores are exhausted: from one thread to four, the v0 runs get 2.0 to 2.3 times faster while the v1 runs manage only 1.3 to 1.4, and v1 gains almost nothing between two threads and four. What does not scale is decoding, which the patch series does not touch. Adding threads therefore shrinks VMAF’s share of the wall clock, and the same kernel

³ NB: these results are distinct from those in Chapter 5, which involved optimizing the `geq` and `convolution` filters within FFmpeg.

improvements move a smaller fraction of the total. The effect is largest exactly where the single-threaded gain was largest, which is why the +38.2% at 1080p 8-bit under v1 falls further than any other entry.

A full list of results for both the pure VMAF binary runs and the FFmpeg runs is available in Appendix B.

7.5 Baseline Strength

The most serious objection to a result of this kind is that the baseline implementation was weak. As we discuss in §7.3.1 and §7.3.2, there are components of VMAF that were relatively underoptimized: three of the eleven kernels targeted were scalar C, and a significant portion of their speedup comes from vectorization alone without infusing any additional knowledge into the implementation.

For the remaining eight, the baseline was already very well-optimized. `libvmaf` ships hand-written AVX2 and AVX-512 intrinsics for all of them, so the comparison is tuned SIMD against differently tuned SIMD. That set includes several of the most expensive kernels in VMAF: the VIF statistic composes 66% of the runtime of the v0 model at 10-bit, and the convolution passes constitute over a tenth of v1. These kernels (especially VIF) have already received years of conventional performance engineering effort, but specialization was able to push throughput significantly higher. The specialized vertical convolution kernel achieves over $3\times$ the throughput of the upstream vectorized implementation, and nearly $2\times$ that of the intermediate implementation which already utilized hand-written AVX-512 intrinsics.

7.6 Assessing the Methodology

All four axes of Chapter 4 are represented in the series. Input specialization accounts for some of the largest and most reliable results, through both of its forms: specializing for the value range of the data produced the CAMBI top- k selection and the boolean scan, both above +300% on every host and both remarkably consistent across microarchitectures, while specializing for shape produced the two convolution passes, likewise large but the most host-dependent of the group. Algebraic specialization produced the ADM transform and both VIF results, the smallest gains in the table but attached to the kernels that dominate runtime, so they move the whole program more than the larger ratios do. Environmental specialization produced the anti-dithering filter, the c-values kernel, the mode filter and the motion pipeline: a spread running from the single largest result in the series down to two of the smallest.

What the largest results share is that exploiting a structural fact made it possible to use an instruction sequence that performs fundamentally less work than would otherwise be possible. The ADM transform can issue two byte-wise multiply-accumulates in place of one 16-bit one because the filter weights are fixed in advance and can be proven not to alias in the partial sums; the VIF statistic can use a signed 16-bit multiply-accumulate on unsigned data

with a cheap correction factor because it computes both first-order and second-order image polynomial convolutions in a single pass; and the convolution can carry a rolling window in registers because the window size is embedded during the first stage, which makes it possible to use efficient alignment instructions that require immediates for shift amounts and thus cannot be used dynamically. That last case is the clearest demonstration of the benefits of staging in the series: it is not that the dynamic version of the rolling-window convolution is slower, but that it cannot be written at all. Chapter 6 develops each of these.

The environment axis is the widest-spread of the four. It is the axis one might expect to be exhausted in a codebase whose authors had already written AVX-512 intrinsics throughout, and yet it contributed the single largest speedup. Still, environmental specialization did not pay off for several of the kernels, particularly the kernels that were naively widened.

The CAMBI mode filter demonstrates the difference between these cases. A prior version of the patch that naively ported the kernel from AVX2 to AVX-512, doubling the vector width and changing nothing else, made it roughly 20% *slower*. Rewriting it to exploit a capability AVX-512 actually adds, using `valignr` to reuse loaded data across the three shifted views a median-of-three requires, made it roughly 20% faster: a swing of forty points in the same function, on the same instruction set. Making use of wider registers is a very blunt form of specialization, and applying it mechanically can produce regressions. The environment axis only paid off after effort was spent understanding what the wider environment actually offered.

7.7 Limitations

We will evaluate various potential limitations and threats to the validity of these results.

7.7.1 Toolchain sensitivity

All results in this chapter come from one gcc 15.2 build, which isolates microarchitecture but says nothing about a user building from source with an older compiler. For one kernel, the answer is not neutral. The VIF specialization is worth +3.8% at 4K 8-bit on the EPYC in a binary compiled with gcc 15.2, but yields -2.1% under gcc 11.5 on the same silicon. This flips the v0 end-to-end result from +5.2% to -0.8%. Hardware counters attribute this to scheduling rather than to the source: the optimized build retires 9% fewer instructions in 4% more cycles, with front-end stalls, branch misses and cache misses all flat. Every other patch in the series is stable across several major versions of gcc.

Specializations may be particularly sensitive to compilation environment because a specialized kernel exploits deeper information about its compilation targets to extract maximal performance. A compiler that does not share the same cost model as the specialization may assemble code in a way that breaks the optimization. In the case of the VIF regression, gcc 11.5 predates the Zen 5 microarchitecture, so the static scheduling and instruction selection it employs does not consider the cost model of Zen 5 whatsoever. The solution is to ensure the compilation environment is well-suited to the desired target.

7.7.2 Abstract cost model

§7.1 sets out the tradeoff the series makes: to avoid the need to design separate codepaths for different microarchitectures, kernels are written against an idealized cost model meant to hold across several modern AVX-512 implementations. Therefore, optimizations are worth less when a real machine ranks operations differently than the model. This is the source of the microarchitectural discrepancies within Table 7.2: performance improves most when the abstract cost model aligns best with the microarchitecture for the operations involved in a particular patch.

The abstract cost model is itself a form of generality, and the slightly worse performance of the patch set on certain microarchitectures is the cost of that generality. An implementation that fully specializes for individual microarchitectures could theoretically achieve even greater speedups; we discuss this further in §8.2.

7.7.3 Maintainability

A specialization embeds a fact about the problem into the structure of the program, so it carries an obligation that the fact remain true. That obligation is the real maintenance cost of this work, and it is not uniform across the series: some of the facts are algorithmic invariants that survive any retuning of the model, and others are specific numeric constants that a future contributor could change without any indication that a kernel now computes the wrong answer.

CAMBI is the benign case. Its scan is specialized on the fact that the derivatives it sums are the outputs of equality tests and are therefore provably zero or one. That property follows from the structure of the algorithm rather than from any tunable parameter, so it holds for any future CAMBI configuration, and a change that violated it would have to be a change to what CAMBI computes. The convolution is similar in kind though not in mechanism: the window size is fixed during the first stage of generation, so changing the filter is a matter of regenerating rather than of rewriting, and a mismatch is caught at build time rather than silently.

The ADM and VIF transforms are the more fragile cases. The optimized kernels here rely on specific properties of the weights within the kernel for correctness, and any change to the weights would require the filter to be rewritten and could invalidate the specialization entirely. In ADM, the use of byte-wise multiply-accumulate is made possible by the specific numeric values of the wavelet filter coefficients, which are small enough that the products cannot overflow the narrower accumulator. Retuning those coefficients would no longer be a matter of changing numbers in a single source file, but would rather require re-engineering the AVX-512 codepath to work with the updated coefficients. Although this would be problematic for weights in general, in practice we find that the existing AVX kernels (including ADM) already embedded various algorithmic parameters into the source code when it is helpful to avoid additional overhead, rather than referencing filter taps from the original source array. This implies that the algorithms are already well-defined enough that a change to algorithmic parameters would never happen, or that the maintainers accept and are willing to change the

vectorized implementations if a retuning should ever be necessary.

The other recurring maintenance cost is that of handling additional dependencies. All of the patches in the series, with the exception of the BuildIt-staged convolution kernels, introduces no additional dependencies to the project and can be used without modification on all standard C compilers. The BuildIt convolution kernels are necessary to extract more performance out of the convolution than the optimized unstaged kernels can achieve (refer to §7.3.2), and compose about 5% of the end-to-end gain of the patch series for Zen 5 machines running the v1 models. Using them requires adding BuildIt as a build-time dependency to `libvmaf` and adds no additional runtime dependencies. Because the patch series is structured such that each optimization can be adopted independently of the others, it is easy to merge all patches except the BuildIt kernels should the project decide that the additional dependency is not worth the performance boost it provides.

7.7.4 Corpus

The per-kernel, per-feature and per-commit results rest on a single source clip at 4K, at two bit depths and under two models; only the end-to-end sweep covers three contents at five resolutions. Content affects the balance between features, and CAMBI’s cost in particular depends on how much banding is present, so a different corpus would shift the shares in Figure 7.4 and therefore the end-to-end totals, though not the per-kernel results.

We believe the use of limited content variety for the benchmarks is acceptable because VMAF’s runtime is not especially content-sensitive, and neither were the optimizations in our patch set. End-to-end runtimes for VMAF on materially different contents (e.g. Sintel, which is computer-generated animation, versus SVT’s “waterfall”, which is real camera footage) were generally similar in testing. To test the extent of the patch set’s content-sensitivity relative to other components in a typical video processing pipeline, we tested our patched VMAF library within an FFmpeg filtergraph. In this setting, VMAF runs on video decoded in real time rather than YUVs warm in the page cache. A performance profile revealed that while the runtime of the `libvmaf` processing was nearly flat at 3.3 ± 0.2 seconds in all tested contents at 4K 8-bit, decoding time varied from 1.1 seconds to 5.4 seconds.

Chapter 8

Conclusion

8.1 Summary

This work develops a specialization-centric patch set that brings substantial end-to-end speedups on real VMAF workloads. The optimizations achieve a >40% speedup on a modern Zen 5 processor running the most important configuration (4K 8-bit with VMAF v1), and produced double-digit performance improvements for almost every other tested configuration. The speedups were not isolated to any single underoptimized component: the patch series discovered and captured performance headroom in every single feature VMAF uses for both its v0 and v1 models.

This result demonstrates that the specialization framework for performance engineering can uncover substantial performance opportunities that were missed by years of traditional effort. Combined with the existing literature, we have validated that the specialization framework is a powerful approach for accelerating software at all levels of optimization, and that it composes with conventional techniques like SIMD vectorization rather than supplanting them.

The discrepancy between the speedups achieved within the FFmpeg case study in [Chapter 5](#) and the speedups achieved within VMAF in [Chapter 6](#) reveals an important, if obvious, qualification for these results: software that is more heavily optimized with traditional techniques stands to gain less from specialization-related optimizations, and often requires additional engineering effort to accelerate. This is not an indictment of the technique but rather a natural consequence of the problem space. Specialization-based optimizations overlap heavily with traditional techniques; for instance, SIMD vectorization is a form of environmental specialization, and fast-paths for common cases are value-range specializations. When we say a program is more heavily optimized by traditional techniques, it is by construction more heavily specialized, so exploiting the remaining specialization headroom naturally becomes more difficult.

The question remains whether the extent to which specialization has succeeded in the literature can extend to real-world performance-sensitive software under the additional constraints

that a production environment imposes. We have reason to believe that it should. As discussed in §6.7, the VMAF patch series largely satisfies the deployability constraints of correctness, maintainability, and dependency-freedom that are often required for a change to be accepted by a major open-source project. The complexity of certain specializations and their fragility to parameter changes is the biggest barrier to adoption for this style of performance engineering. Specializations are unusually sensitive to small changes to the parameters or behavior of an algorithm because they exploit deeper information about those properties than general implementations. If a project frequently updates the behavior of a particular function, specializing that function can introduce more maintenance overhead than it is worth because it may break the specialized invariant entirely, or will at least require every specialized implementation to be updated accordingly.

We can mitigate some of these effects with staging. We used the BuildIt framework to create two-stage kernels for both FFmpeg and VMAF. Especially in VMAF, the generator script is dramatically simpler than the optimized convolution kernels it generates, and any modifications to the first stage will propagate to all generated specializations. Staging makes specialization on high-dimensional information significantly more tractable, while offering a better solution for parameterizing and optimizing over the search space for specialized kernel implementations.

We will end with an analysis of where optimization opportunities remain in today’s software. Most programs that we use on a day-to-day basis are not as thoroughly optimized as VMAF. Even in FFmpeg, which is an otherwise well-written multimedia project with hundreds of x86 acceleration routines, we found that the `vf_convolution` filter was unnecessarily written in a way that impedes autovectorization. The specialization framework is still effective for optimizing code in this state, as we saw in Chapter 5, but traditional approaches will likely prove just as effective when no effort whatsoever has been invested into optimizing a particular codepath. In other words, the standard, well-studied techniques of vectorization, multithreading, and other kinds of environmental specialization will still be effective on a massive swathe of the software we use every day. Once those techniques have been exhausted, however, our software will be in a similar position to VMAF: extremely difficult to optimize further by reasoning about each algorithm purely in the abstract.

We found that although the current version of VMAF makes heavy use of environmental specialization, it rarely exploits specializations that depend upon the structure of the algorithm, its parameters, or its input. The shape, value-range, and algebraic specializations we discovered in VMAF are all examples of a broader trend: even the fastest software we use today does not exploit the unique properties of the computation it performs, or the distribution of inputs it receives, to the extent that it could to achieve maximal performance. By embedding finer-grained insights into the implementation of an algorithm, we can raise its performance ceiling far beyond what would otherwise be possible.

8.2 Future Work

In the near future, we plan to submit the patch set developed in Chapter 6 as a series of merge requests to the official open-source VMAF repository. Successful upstreaming of these patches would validate the deployability analysis in §6.7.

Further extensions of this work could use or design tooling to make the discovery and implementation of certain specializations easier. Many specializations depend upon parameter properties that are visible in the source. Although the implementations of these specializations in the VMAF patch set depended upon manually observing and exploiting these properties with handmade algebraic transformations, a sufficiently advanced compiler could theoretically search for such an algebraic transformation on its own and prove that it is sound. Compilers can already do this to a limited extent; for instance, they regularly replace divisions by a constant integer with a multiplication and shift that emulates the result using the properties of modular arithmetic [27]. Compiler improvements that extend the search space for algebraic specializations could dramatically simplify the process of exploiting these speedups in real code.

In a similar vein, a compiler that extends a language’s type system to capture fine-grained algebraic properties, and propagates this information through arithmetic transformations, could make it easier to discover and exploit value-range specialization and related optimizations in existing codebases. Compilers that propagate bit-width information through a program’s arithmetic have already been developed, but they currently lack the ability to model fine-grained algebraic invariants or additional information about the expected *distribution* of values throughout the program’s lifecycle [26]. Such adaptations could make these tools more applicable to specialization-centric optimization.

Another potential avenue would be extending the work to alternative models of computation, such as a GPGPU scheduling language or a hardware description language. The ideas behind specialization are applicable to more than CPUs, though the exact mechanics of how specialization should best be utilized may differ in different environments. For example, it could be possible to specialize the high-performance GPGPU kernels used in AI applications for common weight patterns to reduce data movement costs. Given the looser correctness requirements for those applications, it may be possible to make use of certain numerical approximations and transformations that would not have been permissible in this work.

Lastly, an extension of this project could adapt the developed kernels to specialize for particular microarchitectures, rather than an abstract cost model that matches no CPU in particular. The ISA is itself a form of generality over multiple hardware implementations, and software pays the cost of this generality because it cannot take full advantage of the execution capabilities of the CPU: it must use the ISA abstractions of registers and instruction operands rather than directly addressing the CPU’s vector pipelines. Although we cannot escape the ISA abstraction, we can still specialize for the execution preferences of a particular microarchitecture. For example, if we have the choice between additional shuffle work and additional ALU work in some portion of a kernel, we can select which work to perform based on the relative throughput of the relevant ports on the CPU in question. These kinds of optimizations are often seen as overly cumbersome because different microarchitectures

can vary dramatically in their performance characteristics on the same instructions, so optimal microarchitecture-specialized implementations can vary dramatically. Moreover, adding microarchitecture as an additional dimension to a specialization only worsens the combinatorial explosion. In other words, specialization improves performance while generality saves engineering effort.

Staging can help bridge the gap between specialization and generality, providing much of the benefit of a microarchitecture-specialized implementation with less engineering effort. For example, the staged vertical convolution kernel in §6.6.3 selects the unroll factor for a particular kernel width based on a load-centric cost model, based on the empirical observation that the convolution was load-bottlenecked for the sizes we were interested in on Zen 5. By extending the first-stage kernel generation logic, it is straightforward to generate several implementations of the same kernel optimized for alternate cost models, such as one that minimizes ALU port usage, and to then select the kernel whose heuristic best matches the current microarchitecture at runtime.

Appendix A

Speedup Estimation

We apply the two-sample Hodges-Lehmann estimator in measuring speedups of an optimized program relative to an unoptimized one because it is simple, requires no distributional assumptions about program runtimes, and is more robust to outliers than other methods.

A.1 Derivation

Formally, let X be a random variable denoting the original runtime of a program or kernel, and let Y be a random variable denoting the optimized runtime. X and Y are independent but may be differently distributed. Per the formula in §7.1, speedup between the unoptimized and optimized runtime is

$$S = \frac{X}{Y} - 1$$

Consider the random variable

$$D = \ln(X) - \ln(Y)$$

Note that $S = g(D)$ where $g(d) = \exp(d) - 1$ is a monotonic function. Let $X_1, \dots, X_n \stackrel{\text{iid}}{\sim} X$ and $Y_1, \dots, Y_m \stackrel{\text{iid}}{\sim} Y$ be random variables denoting our observed samples of X and Y , i.e. our measured runtimes. The two-sample Hodges-Lehmann estimator for $\text{median}(D)$ is

$$\text{HL}_D = \text{median} \{ \ln(X_i) - \ln(Y_j) \mid i \in 1 \dots n, j \in 1 \dots m \}$$

Since the median is equivariant to monotonic operations, we can estimate the median speedup of our optimizations as $\text{median}(S) = \text{median}(g(D))$:

$$\text{HL}_S = \text{median} \{ g(\ln(X_i) - \ln(Y_j)) \mid i \in 1 \dots n, j \in 1 \dots m \}$$

$$\text{HL}_S = \text{median} \left\{ \frac{X_i}{Y_j} - 1 \mid i \in 1 \dots n, j \in 1 \dots m \right\}$$

Therefore we can compute HL_S , the median speedup over the Cartesian product of our pre- and post-optimization runtimes, as an estimator of the true median speedup $\text{median}(S)$.

Note that HL_S inherits the asymptotic efficiency and robustness of the standard two-sample Hodges-Lehmann estimator because it is equivalent to the Hodges-Lehmann estimator on log-runtimes.

Taking the median over all nm pairs, rather than over n ratios of runs matched up one-to-one, is what makes the estimator a function of the data alone. There is no natural pairing between a baseline run and an optimized run, so any paired estimator depends on an arbitrary matching. The Cartesian product averages over all such matchings and is invariant to the choice.

We also prefer the median of ratios to the ratio of fastest observed runtimes, the more common convention in software benchmarking. A minimum drifts downward as more repetitions are collected, so configurations measured a different number of times are not on the same scale, whereas the median is n -consistent. In practice, the two agree closely: across the 24 baseline-to-tip comparisons of Chapter 7 they differ by at most 0.63 percentage points, and across the 132 per-commit comparisons of Appendix B by at most 3.04. Nor is either systematically larger, with HL_S falling below the ratio of minima in 58% and 51% of those comparisons respectively, so this is a choice of a better-defined statistic rather than of a more conservative one.

Although we do not make any assumptions about the distributions of the runtimes X and Y , intuitively we might expect them to be approximately log-normal: this models measured runtime as a product of several i.i.d. multiplicative noise factors (e.g. context switches) relative to some base runtime. If this holds, our estimator is even stronger, as we are effectively applying Hodges-Lehmann to normally distributed data and will achieve 95.5% asymptotic relative efficiency versus a two-sample t -test (as opposed to $\geq 86.4\%$ for an unconstrained continuous distribution) [40].

A.2 Confidence Intervals

In order to generate confidence intervals for our median speedups, we bootstrap the above estimator; that is, we sample n values with replacement from our runtime measurements for the unoptimized variant running a given configuration, sample m runtimes from the optimized variant in the same manner, and then apply HL_S to the bootstrapped sample. We use 10^5 resamples with a fixed seed, so re-running the analysis reproduces the reported intervals exactly. The distribution of the bootstrapped realizations of the estimator provides an empirical CDF of the median speedup, from which we report the 2.5th and 97.5th percentiles as a 95% interval.

Both the estimator and the bootstrap rest on the i.i.d. assumption stated above, which is an idealization: repeated runs on one machine share thermal state, frequency residency and page placement, and are not truly independent draws. Drift over a measurement session is the most consequential way this fails, because it correlates with the order in which builds are measured, which is the axis under comparison. The harness therefore interleaves every build within each repetition, so that any drift is shared across builds rather than accumulating in

whichever was measured last, and pins each run to a fixed core with a warm page cache to suppress the largest sources of run-to-run variation. We do not claim the assumption holds exactly, only that the experiment is arranged so that its failure modes are shared rather than confounded with the comparison.

We report percentile intervals rather than basic (pivotal) ones. The percentile method is exact whenever some monotone transformation renders the estimator’s sampling distribution symmetric, and $g^{-1}(s) = \ln(s + 1)$ is the natural candidate here: it carries the speedup onto the log-runtime scale, where the estimator is the ordinary two-sample Hodges-Lehmann location estimator and where the lognormal model of §7.1 would make the underlying differences normal. The percentile interval is also equivariant under that transformation because g is monotone, so the choice of scale cannot by itself change coverage. We avoid the basic bootstrap interval, which reflects the bootstrap distribution through the point estimate and so treats any asymmetry as evidence of bias rather than as a property of the quantity being estimated.

A.3 Per-Kernel Speedup Measurements

For individual kernel speedup measurements we combine two Hodges-Lehmann estimators, because a kernel’s cost is not measured directly. It is the product of the kernel’s share of sampled cycles, obtained by profiling with `perf`, and a wall-clock measurement of the same build running without profiling attached. Writing r_f for the Hodges-Lehmann ratio of self-time fractions between the two builds and r_w for the corresponding ratio of wall clocks, the reported kernel speedup is

$$r_f r_w - 1$$

Composing the two median ratios multiplicatively is an approximation: strictly, the estimator analogous to HL_S would take the median over the full four-way Cartesian product of fraction and wall-clock samples. However, the approximation is very good in practice because the wall-clock factor has a coefficient of variation well under 1%.

A.4 Limitations

The intervals above bound sampling error, the component that shrinks as more repetitions are collected. They do not bound attribution error, which concerns the extent to which changes in a routine’s performance can actually be attributed to changes within that routine.

The same source code compiled into two different binaries need not run at the same speed, because the surrounding code changes its layout, alignment, instruction-cache behavior, and other microarchitecturally relevant properties. Moreover, compilers make different decisions about how to optimize functions within a single translation unit together, so modifying one function’s source code can affect the performance of all functions within the same TU. We can estimate these effects with a null experiment. Several of the kernels reported in Chapter 7 have identical source code between the upstream master and the optimized branch, so while

measured deviations may be true speedups, calling any such results a successful optimization to that kernel would be attribution error. Across 52 such control measurements the deviations have a standard deviation of 1.2 points on the desktop Ryzen and 1.2 on the EPYC, once each control’s own sampling variance is subtracted out. The Xeon is one-sided: every one of its control deviations is positive, averaging +1.3 points among those precise enough to be informative, so on that host the unmodified kernels are uniformly faster in the optimized binary. We have no mechanism to offer for the asymmetry; it is a property of that host and that pair of binaries.

We can verify directly that the deviations are real and systematic rather than a result of mismeasurement. Disassembling the control kernels shows that none of them compiles to the same machine code in the two binaries: all differ in instruction selection, not merely in register allocation. `adm_dwt2_16_avx512`, for instance, has byte-identical source but compiles to 3563 bytes and 728 instructions in the baseline against 3569 and 726 at the tip, differing in 50 mnemonics, because the sibling function `adm_dwt2_8_avx512` in the same translation unit was modified. Nor is the deviation a sampling artifact: raising the profiler’s sampling rate from 4999 to 12983 Hz leaves a representative control’s deviation effectively unchanged, while the run-to-run standard deviation of the sampled self-time fraction falls as $1/\sqrt{\text{rate}}$, as expected. Sampling error therefore behaves exactly as theory predicts and the deviation does not, which is what separates the two.

The practical consequence is that a per-kernel result carrying a ± 0.3 point confidence interval does not necessarily mean that the speedup is definitively a result of optimizations made within the kernel source code. The interval describes how precisely a speedup can be measured, not how much of the speedup is attributable to modifying that particular kernel.

We evaluated the magnitude of attribution error by measuring speedup deviations on several kernels with unchanged source code from the original VMAF master branch to the optimized tip, excluding those kernels whose CI spread more than 5% to avoid skewing our measurements with too much sampling noise. The central 95% of these control deviations spans $[-2.2, +2.4]$ on the Ryzen host, $[-4.0, +1.4]$ on EPYC, and $[+0.1, +3.2]$ on Xeon.

Rather than carry three host-specific bands, we adopt the single flat criterion stated in §7.1: a kernel-level speedup is treated as a successful optimization only if its confidence interval does not overlap $[-4\%, +4\%]$. This is at least as conservative as every per-host band above. It is deliberately blunt: some small but real effects will fail it, and we prefer that to over-claiming.

Attribution effects can affect whole-program measurements too in the sense that different compiler versions and configurations may cause certain optimizations to perform differently on the end-to-end program. We do not have a good way to consistently quantify this effect. However, such variations should be interpreted more as a limitation on the generality of a particular source-code optimization to different compilers than as an error in the same sense as kernel-level performance shifts. Within a single compilation environment, any end-to-end change in performance after applying a patch should be attributed to that patch, regardless of the exact mechanism by which the patch affects the compiled code. Therefore, any end-to-end speedup whose CI does not intersect 0 is treated as a true optimization.

Appendix B

Complete Measurement Data

Chapter 7 reports a deliberate subset of the measurements, chiefly a single comparison between upstream master and the branch tip. This appendix reports the full set, so that the selection made in the body can be checked against the data it was drawn from.

B.1 Scope

Three machines were used, all executing the same statically linked binary per commit. Each was compiled once with gcc 15.2 and copied unchanged to every host, so that compiler, C library and link mode are constant and any difference between columns is attributable to hardware. Two of the three are Zen 5 parts: a desktop Ryzen 9850X3D and the EPYC 9R05 of Table 7.1. The third is the Skylake-X Xeon 8180.

The two Zen 5 parts are the same microarchitecture, both being full Zen 5 cores rather than the density-optimized Zen 5c variant, and Table B.1 shows that their results track each other closely in every configuration. Where they disagree, the cause is the memory hierarchy rather than the core, with the desktop part carrying 96 MiB of stacked L3 against the EPYC's 20 MiB.

Each host was additionally tested on a VMAF build using its own toolchain rather than gcc 15.2 to test for the sensitivity of each optimization to compilation environment. Those runs are not tabulated as most of the results were very similar to the gcc 15.2 build; however, there were a few exceptions. Notably, the VIF specialization is worth +3.8% under gcc 15.2 and -2.1% under gcc 11.5 on identical silicon, which is enough to flip the 4K 8-bit v0 end-to-end result from +5.2% to -0.8%. Every other kernel in the series moves by at most a few points across a four-major-version compiler gap. We discuss this configuration further in §7.7.

B.2 Reading the Tables

Every figure in this appendix is a speedup in percent, estimated as described in Appendix A. Where a table carries confidence intervals they are 95% bootstrap intervals, given in brackets after the estimate rather than as a $\pm$ figure, so that the two bounds can be read separately. Chapter 7 quotes the same intervals in the symmetric $\pm$ form.

Three conventions apply throughout. Entries marked † have source code identical to upstream in that configuration and are reported as controls: any deviation they show is attribution error rather than optimization, which is what calibrates the $\pm 4\%$ criterion of §7.1. An entry marked with a dash did not execute in that configuration, which is not the same as measuring zero. Symbol width is matched to content depth, so 8-bit kernels are reported on 8-bit content and 16-bit kernels on 10-bit.

The per-commit tables add a fourth convention, because most individual commits move the whole program by less than their own interval and the eye needs help separating the ones that do not. An entry is set in bold where its entire interval lies at least 0.3 points from zero, plain where the interval clears zero by less than that, and grey where the interval contains zero and so supports no directional claim at all. The first row of each is the baseline wall clock in seconds and the last is the cumulative result; the rows between give each commit against its immediate parent rather than against the baseline, so that the series reads as a running account.

Four commits account for essentially the whole result. The VIF speedup carries the v0 model, and the CAMBI, convolution and staged-convolution commits carry v1. The remainder are refactors, cleanups, or changes whose effect is real at kernel level but too small to resolve end to end. The ADM transform is the clearest case: on EPYC, it improves its own kernel by +38% yet moves the whole program by at most +2.4%, which is the Amdahl relationship of §7.4 seen one commit at a time.

B.3 Tables

Table B.1: End-to-end speedup from upstream master (78e11b52) to the branch tip (289cbcf3), over twelve timed repetitions per side.

Configuration	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
4K 8-bit, v0	+5.5 [+5.2, +5.8]	+5.2 [+3.2, +5.4]	+16.7 [+16.5, +17.0]
4K 8-bit, v1	+41.1 [+40.3, +41.6]	+43.4 [+43.1, +43.7]	+23.4 [+23.0, +23.6]
4K 10-bit, v0	+15.9 [+15.1, +16.8]	+14.2 [+13.8, +17.6]	+26.0 [+26.0, +26.1]
4K 10-bit, v1	+20.2 [+19.1, +21.5]	+22.4 [+22.1, +22.8]	+14.3 [+14.2, +14.4]

Table B.2: Baseline wall clock in seconds, upstream master. Included so that the relative figures elsewhere can be converted back to absolute time.

Configuration	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
4K 8-bit, v0	15.92	16.75	51.68
4K 8-bit, v1	9.15	10.53	29.80
4K 10-bit, v0	13.74	11.06	47.22
4K 10-bit, v1	6.17	5.32	21.62

Table B.3: Per-kernel speedup on 8-bit content. Conventions as in §B.2.

Kernel	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
VIF statistic	+3.7	+3.9	+28.2
VIF subsample_rd [†]	−0.1	−0.6	+7.1
Motion score pipeline	+28.4	+11.1	+22.8
Motion x_conv_row_sad	+9.0	+8.8	+29.3
ADM DWT level 1	+27.1	+38.3	−5.5
ADM DWT scales 1–3 [†]	−0.7	+0.1	+1.1
ADM csf / decouple / cm [†]	+0.5	−0.0	−0.6
CAMBI boolean DP scan	+303.4	+305.9	+414.9
CAMBI top- <i>k</i> select	+397.2	+381.3	+320.1
CAMBI c-values + histogram	+62.5	+53.9	+81.7
CAMBI mode filter	+11.9	+21.5	+77.1
CAMBI decimate	−7.7	+38.5	+33.9
CAMBI derivative	−1.2	+15.0	+57.4
CAMBI histogram range	−8.0	−15.3	+18.7
CAMBI anti-dithering	+1178.0	+1088.7	+625.7
Convolution, vertical	+148.8	+220.0	+60.0
Convolution, horizontal	+88.5	+80.1	+54.7

Table B.4: Per-kernel speedup on 10-bit content. Conventions as in §B.2.

Kernel	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
VIF statistic	+24.7	+24.8	+50.1
VIF subsample_rd [†]	+1.7	−1.2	+0.6
Motion score pipeline	−1.6	+0.1	+2.0
Motion x_conv_row_sad	+11.9	+13.5	+18.2
ADM DWT level 1 [†]	−3.5	−4.7	+0.9
ADM DWT scales 1–3 [†]	−0.6	−0.4	+2.2
ADM csf / decouple / cm [†]	−0.5	−0.5	+0.9
CAMBI boolean DP scan	+278.4	+249.7	+423.4
CAMBI top- <i>k</i> select	+391.1	+373.6	+345.9
CAMBI c-values + histogram	+113.0	+77.2	+110.2
CAMBI mode filter	+15.3	+15.8	+55.8
CAMBI decimate	+19.7	+52.8	+19.3
CAMBI derivative	+6.7	+37.9	+12.3
CAMBI histogram range	−29.8	−47.4	+15.8
CAMBI anti-dithering	—	—	—
Convolution, vertical	+144.7	+216.8	+37.5
Convolution, horizontal	+87.5	+75.3	+53.7

Table B.5: Per-feature speedup on 8-bit content. ADM and motion run under both models, so each is given a row per model; VIF runs only under v0, and CAMBI and the convolution only under v1. Where a feature has two rows they measure the same kernels on the same clip, and the two agree to within the attribution error of Appendix A rather than differing for any reason internal to the feature. Motion is not: its 16-bit path is modified too, but by a change the measurement cannot resolve. Conventions as in §B.2.

Feature	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
vif	+5.8	+5.7	+26.9
motion (v0)	+28.4	+11.1	+22.8
motion (v1)	+24.0	+11.5	+33.1
adm (v0)	+4.3	+4.9	−2.8
adm (v1)	+2.4	+6.0	+1.9
cambi	+131.5	+131.7	+110.5
conv	+110.0	+141.1	+61.3

Table B.6: Per-feature speedup on 10-bit content. ADM and motion run under both models, so each is given a row per model; VIF runs only under v0, and CAMBI and the convolution only under v1. Where a feature has two rows they measure the same kernels on the same clip, and the two agree to within the attribution error of Appendix A rather than differing for any reason internal to the feature. The series changes ADM only in the 8-bit wavelet transform, so on 10-bit content every kernel ADM runs is upstream code and the entry is a control. Motion is not: its 16-bit path is modified too, but by a change the measurement cannot resolve. Conventions as in §B.2.

Feature	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
vif	+20.9	+19.9	+40.7
motion (v0)	−1.6	+0.1	+2.0
motion (v1)	−3.2	−0.7	−1.2
adm (v0) [†]	−1.3	−1.5	+1.0
adm (v1) [†]	−2.2	−0.6	+0.2
cambi	+56.9	+59.8	+64.0
conv	+106.5	+138.6	+48.2

Table B.7: Per-commit speedup at 4K 8-bit, v0, each commit against its immediate parent. Conventions as in §B.2.

Commit	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
78e11b52 base (upstream master) (s)	15.92	16.75	51.68
d0f65d15 Compiler flags	−0.9 [−1.2, −0.6]	+0.0 [−1.9, +0.3]	+0.1 [+0.0, +0.4]
c6dd2e0f VIF speedup	+4.1 [+3.7, +4.6]	+3.8 [+3.7, +4.0]	+16.0 [+15.8, +16.2]
9b558ab5 CAMBI AVX-512	−0.1 [−0.8, +0.3]	+0.1 [−1.0, +0.2]	−0.0 [−0.2, +0.2]
0a4ca4ef Motion optimization	+0.3 [−0.6, +0.9]	+0.5 [+0.2, +1.7]	+0.4 [+0.2, +0.6]
8ccbb975 AVX-512 quickselect	−0.0 [−0.5, +0.9]	−0.1 [−1.3, +0.1]	−0.1 [−0.3, +0.2]
dc7db2ff General cleanup	−0.1 [−0.8, +0.6]	−0.2 [−0.3, +0.2]	+0.1 [−0.2, +0.3]
1ef2aa57 Float convolution refactor	−0.3 [−1.2, +0.6]	+0.4 [−0.0, +0.5]	−0.1 [−0.3, +0.2]
d699f89a Convolution AVX-512	+0.1 [−0.8, +1.1]	−0.4 [−0.6, −0.2]	−0.0 [−0.2, +0.3]
34364be1 ADM DWT	+1.6 [+1.1, +2.9]	+1.1 [+1.0, +3.2]	+0.1 [−0.1, +0.3]
289cbcf3 BuildIt convolution	+0.3 [−0.1, +0.8]	−0.1 [−0.5, +0.1]	+0.0 [−0.2, +0.2]
cumulative	+5.5 [+5.2, +5.8]	+5.2 [+3.2, +5.4]	+16.7 [+16.5, +17.0]

Table B.8: Per-commit speedup at 4K 8-bit, v1, each commit against its immediate parent. Conventions as in §B.2.

Commit	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
78e11b52 base (upstream master) (s)	9.15	10.53	29.80
d0f65d15 Compiler flags	−1.5 [−1.9, −1.2]	+0.3 [+0.1, +0.6]	+0.2 [−0.0, +0.4]
c6dd2e0f VIF speedup	+0.2 [−0.4, +0.8]	−0.1 [−0.5, +0.2]	+0.0 [−0.2, +0.4]
9b558ab5 CAMBI AVX-512	+21.6 [+20.4, +22.3]	+21.0 [+20.6, +21.3]	+10.8 [+10.2, +11.1]
0a4ca4ef Motion optimization	+1.0 [+0.3, +2.0]	+0.7 [+0.3, +1.0]	+0.7 [+0.2, +1.1]
8ccbb975 AVX-512 quickselect	+4.6 [+3.9, +5.2]	+3.8 [+3.5, +4.2]	+3.0 [+2.6, +3.5]
dc7db2ff General cleanup	−0.5 [−1.6, +0.6]	−0.0 [−0.2, +0.3]	+0.0 [−0.4, +0.5]
1ef2aa57 Float convolution refactor	+0.1 [−0.9, +1.2]	+0.9 [+0.7, +1.3]	+0.5 [+0.1, +1.0]
d699f89a Convolution AVX-512	+4.2 [+2.4, +5.1]	+4.5 [+4.2, +4.9]	+5.7 [+5.3, +6.1]
34364be1 ADM DWT	+2.9 [+2.0, +4.9]	+2.4 [+2.0, +2.7]	+0.3 [−0.1, +0.8]
289cbcf3 BuildIt convolution	+4.1 [+3.6, +4.9]	+4.7 [+4.4, +5.1]	+0.4 [+0.2, +0.7]
cumulative	+41.1 [+40.3, +41.6]	+43.4 [+43.1, +43.7]	+23.4 [+23.0, +23.6]

Table B.9: Per-commit speedup at 4K 10-bit, v0, each commit against its immediate parent. Conventions as in §B.2.

Commit	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
78e11b52 base (upstream master) (s)	13.74	11.06	47.22
d0f65d15 Compiler flags	+0.2 [−0.5, +1.0]	+0.1 [−0.1, +0.5]	+0.2 [−0.0, +0.3]
c6dd2e0f VIF speedup	+15.2 [+14.1, +16.1]	+13.5 [+13.1, +13.9]	+25.7 [+25.6, +26.0]
9b558ab5 CAMBI AVX-512	−0.1 [−0.6, +0.9]	+0.2 [−0.1, +3.4]	−0.0 [−0.2, +0.1]
0a4ca4ef Motion optimization	−0.1 [−1.3, +0.5]	+0.1 [−0.2, +3.3]	+0.1 [+0.0, +0.3]
8ccbb975 AVX-512 quickselect	−0.3 [−1.1, +0.9]	−0.3 [−3.4, +0.1]	+0.1 [−0.1, +0.2]
dc7db2ff General cleanup	+0.2 [−0.5, +1.1]	−0.1 [−0.3, +0.1]	−0.1 [−0.3, +0.1]
1ef2aa57 Float convolution refactor	+0.0 [−0.8, +0.7]	+0.2 [+0.1, +1.8]	+0.0 [−0.2, +0.2]
d699f89a Convolution AVX-512	+0.0 [−0.9, +0.8]	−0.2 [−1.8, −0.1]	+0.1 [−0.1, +0.3]
34364be1 ADM DWT	+0.3 [−0.5, +1.1]	+0.0 [−0.2, +3.5]	−0.3 [−0.4, −0.1]
289cbcf3 BuildIt convolution	+0.4 [−0.4, +1.3]	+0.4 [+0.1, +3.4]	+0.1 [+0.1, +0.2]
cumulative	+15.9 [+15.1, +16.8]	+14.2 [+13.8, +17.6]	+26.0 [+26.0, +26.1]

Table B.10: Per-commit speedup at 4K 10-bit, v1, each commit against its immediate parent. Conventions as in §B.2.

Commit	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
78e11b52 base (upstream master) (s)	6.17	5.32	21.62
d0f65d15 Compiler flags	+0.1 [−0.8, +1.1]	+0.1 [−0.2, +0.3]	+0.7 [+0.5, +0.9]
c6dd2e0f VIF speedup	+0.3 [−0.6, +1.2]	−0.2 [−0.5, +0.1]	−0.0 [−0.2, +0.2]
9b558ab5 CAMBI AVX-512	+7.3 [+6.4, +8.3]	+8.1 [+7.7, +8.4]	+4.5 [+3.9, +4.8]
0a4ca4ef Motion optimization	+0.0 [−0.8, +0.8]	+0.3 [−0.1, +0.7]	+0.5 [+0.3, +1.1]
8ccbb975 AVX-512 quickselect	+3.1 [+2.2, +4.2]	+3.4 [+3.0, +3.7]	+2.7 [+2.5, +2.9]
dc7db2ff General cleanup	+0.2 [−0.9, +1.3]	+0.1 [−0.2, +0.4]	−0.0 [−0.3, +0.1]
1ef2aa57 Float convolution refactor	+0.5 [−0.3, +1.5]	+0.8 [+0.3, +1.1]	+0.4 [+0.2, +0.6]
d699f89a Convolution AVX-512	+3.6 [+2.8, +4.5]	+4.0 [+3.5, +4.4]	+5.0 [+4.8, +5.3]
34364be1 ADM DWT	−0.5 [−1.5, +1.2]	+0.2 [−0.2, +0.5]	−0.6 [−0.7, −0.2]
289cbcf3 BuildIt convolution	+4.2 [+2.7, +5.5]	+4.2 [+3.9, +4.6]	+0.4 [+0.3, +0.5]
cumulative	+20.2 [+19.1, +21.5]	+22.4 [+22.1, +22.8]	+14.3 [+14.2, +14.4]

B.4 Input Breadth: Resolution, Content and Codec

The tables in this section come from two experiments run only between upstream master and the branch tip, to establish how the end-to-end result varies with the input rather than with the code.

The first is a resolution sweep: the `vmaf` CLI on raw YUV, three contents at five resolutions, a uniform 120 frames throughout so the curve is not also a frame-count curve. All three hosts run the same byte-identical corpus. Figures 7.2 and 7.3 plot the means; the tables give every per-content value.

The second includes decode. It drives `ffmpeg`'s `libvmaf` filter on two encoded videos, the common real-world invocation of scoring the quality lost to a re-encode, so H.264 or AV1 decode of both inputs sits inside the timed region. Reference and distorted are at matched resolution, the distorted side being the reference re-encoded, so no scaling occurs and the measurement is decode plus VMAF and nothing else. Because `ffmpeg` loads `libvmaf` as a shared object, this arm uses per-host shared builds rather than the single static binary used everywhere else; the base-to-tip comparison within each host is still one-variable, but absolute times should not be compared between hosts.

Table B.11: Resolution sweep, 8-bit content under the v1 model: end-to-end speedup of the `vmaf` CLI on raw YUV, 120 frames at every resolution. `sintel` is 8-bit only and is letterboxed to 16:9, so roughly a quarter of its frame is black bars.

Resolution	Content	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
854 × 480	smithy	+39.8	+42.9	+45.6
	waterfall	+42.6	+43.1	+41.4
	sintel	+46.2	+48.8	+48.2
1280 × 720	smithy	+53.6	+53.8	+50.1
	waterfall	+54.2	+52.7	+48.6
	sintel	+62.2	+60.6	+57.5
1920 × 1080	smithy	+42.9	+40.1	+34.3
	waterfall	+41.8	+40.1	+32.0
	sintel	+48.1	+45.6	+39.6
2560 × 1440	smithy	+40.6	+40.9	+32.1
	waterfall	+45.5	+41.6	+34.3
	sintel	+47.8	+46.2	+30.4
3840 × 2160	smithy	+40.7	+42.2	+30.2
	waterfall	+43.4	+44.5	+26.6
	sintel	+45.2	+47.5	+41.1

Table B.12: Resolution sweep, 10-bit content under the v1 model: end-to-end speedup of the vmaf CLI on raw YUV, 120 frames at every resolution. sintel is 8-bit only and is letterboxed to 16:9, so roughly a quarter of its frame is black bars.

Resolution	Content	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
854×480	smithy	+26.8	+28.0	+57.5
	waterfall	+26.4	+27.6	+30.8
1280×720	smithy	+34.7	+32.1	+26.3
	waterfall	+34.6	+31.4	+36.4
1920×1080	smithy	+20.2	+17.7	+16.6
	waterfall	+19.1	+17.7	+22.6
2560×1440	smithy	+20.2	+19.0	+18.4
	waterfall	+17.7	+18.9	+18.9
3840×2160	smithy	+20.3	+21.6	+17.8
	waterfall	+20.4	+23.2	+17.6

Table B.13: Resolution sweep, 8-bit content under the v0 model: end-to-end speedup of the vmaf CLI on raw YUV, 120 frames at every resolution. sintel is 8-bit only and is letterboxed to 16:9, so roughly a quarter of its frame is black bars.

Resolution	Content	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
854×480	smithy	+4.7	+4.6	+38.9
	waterfall	+4.2	+4.4	+15.6
	sintel	+4.8	+4.4	+15.9
1280×720	smithy	+5.6	+5.6	+15.0
	waterfall	+5.6	+5.5	+17.5
	sintel	+5.4	+5.7	+17.5
1920×1080	smithy	+4.8	+6.1	+17.7
	waterfall	+5.5	+5.9	+17.0
	sintel	+5.2	+5.9	+17.0
2560×1440	smithy	+5.2	+5.8	+16.4
	waterfall	+6.5	+5.8	+16.4
	sintel	+5.3	+3.3	+15.7
3840×2160	smithy	+5.6	+4.5	+16.9
	waterfall	+5.4	+5.3	+5.0
	sintel	+5.4	+5.0	+16.6

Table B.14: Resolution sweep, 10-bit content under the v0 model: end-to-end speedup of the `vmaf` CLI on raw YUV, 120 frames at every resolution. `sintel` is 8-bit only and is letterboxed to 16:9, so roughly a quarter of its frame is black bars.

Resolution	Content	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
854 × 480	smithy	+14.0	+13.7	+21.4
	waterfall	+13.5	+13.7	+23.9
1280 × 720	smithy	+17.3	+15.9	+26.9
	waterfall	+17.0	+15.7	+26.2
1920 × 1080	smithy	+16.6	+15.1	+31.0
	waterfall	+16.4	+15.4	+22.8
2560 × 1440	smithy	+15.7	+15.1	+25.6
	waterfall	+15.8	+14.9	+25.7
3840 × 2160	smithy	+16.3	+14.3	+21.5
	waterfall	+15.6	+14.7	+33.9

Table B.15: Decode-inclusive speedup under the v1 model: `ffmpeg`’s `libvmaf` filter on two encoded videos, so H.264 or AV1 decode of both inputs is inside the timed region. Reference and distorted are at matched resolution (the distorted side is the reference re-encoded), so no scaling occurs. `x264` uses CRF 28 and SVT-AV1 CRF 35, chosen for comparable quality and decode complexity.

Content	Source	Ryzen H.264	Ryzen AV1	EPYC H.264	EPYC AV1	Xeon H.264	Xeon AV1
smithy	3840, 8-bit	+22.4	+20.1	+22.0	+21.6	+16.9	+16.0
smithy	3840, 10-bit	+10.3	+10.1	+12.6	+15.1	+8.9	+7.6
smithy	1920, 8-bit	+18.8	+17.4	+17.3	+19.3	+19.1	+16.2
smithy	1920, 10-bit	+9.3	+9.7	+9.3	+10.6	+8.8	+9.5
waterfall	3840, 8-bit	+13.9	+16.1	+15.1	+16.0	+13.7	+12.3
waterfall	3840, 10-bit	+7.1	+9.6	+9.1	+12.0	+8.9	+8.4
waterfall	1920, 8-bit	+13.4	+10.8	+11.9	+11.0	+13.3	+11.7
waterfall	1920, 10-bit	+7.5	+6.6	+6.2	+7.3	+7.4	+7.5
sintel	3840, 8-bit	+26.5	+28.4	+25.1	+29.7	+21.2	+20.6
sintel	1920, 8-bit	+29.8	+33.8	+25.4	+31.1	+31.1	+33.7

Table B.16: Decode-inclusive speedup under the v0 model: `ffmpeg`’s `libvmaf` filter on two encoded videos, so H.264 or AV1 decode of both inputs is inside the timed region. Reference and distorted are at matched resolution (the distorted side is the reference re-encoded), so no scaling occurs. `x264` uses CRF 28 and SVT-AV1 CRF 35, chosen for comparable quality and decode complexity.

Content	Source	Ryzen H.264	Ryzen AV1	EPYC H.264	EPYC AV1	Xeon H.264	Xeon AV1
smithy	3840, 8-bit	+5.4	+3.7	−0.5	+0.4	+12.1	+10.9
smithy	3840, 10-bit	+11.7	+11.5	+4.5	+4.5	+18.4	+18.5
smithy	1920, 8-bit	+3.9	+5.3	+0.4	+0.1	+13.1	+11.3
smithy	1920, 10-bit	+12.0	+11.2	+6.2	+6.7	+19.4	+19.4
waterfall	3840, 8-bit	+3.2	+3.0	−0.2	+0.5	+10.1	+9.5
waterfall	3840, 10-bit	+10.7	+10.4	+2.6	+4.1	+17.2	+16.5
waterfall	1920, 8-bit	+4.4	+2.0	−0.2	+0.7	+10.3	+8.5
waterfall	1920, 10-bit	+8.9	+10.4	+5.1	+5.4	+17.8	+16.4
sintel	3840, 8-bit	+6.3	+5.1	−2.6	−3.5	+12.7	+13.1
sintel	1920, 8-bit	+6.4	+4.2	−0.5	−0.7	+14.2	+13.5

Table B.17: Decode-inclusive speedup under the v1 model with `libvmaf` given four threads on four dedicated physical cores. The single-threaded counterpart is Table B.15, which pins one thread to one core; the two arms therefore differ in core count as well as thread count and their difference is not a clean measure of threading. Table 7.5 in Chapter 7 holds the cores fixed and varies only the thread count. Speedups are consistently smaller here because threading shrinks VMAF’s share of the wall clock while leaving decoding, which this work does not touch, unchanged.

Content	Source	Ryzen H.264	Ryzen AV1	EPYC H.264	EPYC AV1	Xeon H.264	Xeon AV1
smithy	3840, 8-bit	+14.4	+11.8	+15.5	+15.2	+12.8	+15.0
smithy	3840, 10-bit	+6.7	+3.3	+9.8	+9.8	+7.6	+8.2
smithy	1920, 8-bit	+14.8	+9.0	+15.3	+12.9	+11.9	+14.2
smithy	1920, 10-bit	+5.2	+4.4	+9.1	+7.9	+8.7	+7.5
waterfall	3840, 8-bit	+14.9	+9.9	+12.5	+10.5	+11.3	+10.5
waterfall	3840, 10-bit	+7.1	+10.7	+8.4	+8.6	+7.9	+7.6
waterfall	1920, 8-bit	+9.5	+5.9	+10.6	+6.4	+12.3	+8.0
waterfall	1920, 10-bit	+4.6	+5.7	+6.1	+5.1	+6.1	+6.3
sintel	3840, 8-bit	+15.0	+17.1	+18.8	+19.6	+17.5	+18.5
sintel	1920, 8-bit	+21.4	+23.1	+22.0	+25.6	+25.9	+24.9

Table B.18: Decode-inclusive speedup under the v0 model with `libvmaf` given four threads on four dedicated physical cores. The single-threaded counterpart is Table B.16, which pins one thread to one core; the two arms therefore differ in core count as well as thread count and their difference is not a clean measure of threading. Table 7.5 in Chapter 7 holds the cores fixed and varies only the thread count. Speedups are consistently smaller here because threading shrinks VMAF’s share of the wall clock while leaving decoding, which this work does not touch, unchanged.

Content	Source	Ryzen H.264	Ryzen AV1	EPYC H.264	EPYC AV1	Xeon H.264	Xeon AV1
smithy	3840, 8-bit	+1.8	+3.2	−1.2	−0.1	+9.9	+9.9
smithy	3840, 10-bit	+8.4	+10.5	+3.2	+4.0	+17.8	+17.8
smithy	1920, 8-bit	+3.5	+1.6	+0.1	−1.3	+8.1	+10.3
smithy	1920, 10-bit	+8.0	+5.1	+4.4	+4.6	+16.7	+16.5
waterfall	3840, 8-bit	+3.5	+3.6	−0.7	−0.6	+8.9	+9.1
waterfall	3840, 10-bit	+5.9	+10.4	+3.4	+2.9	+14.1	+15.3
waterfall	1920, 8-bit	+2.6	+1.5	+0.6	−0.8	+12.9	+9.1
waterfall	1920, 10-bit	+4.5	+7.8	+4.6	+4.1	+17.4	+15.2
sintel	3840, 8-bit	+5.8	+3.8	−0.8	−1.0	+11.0	+13.7
sintel	1920, 8-bit	+0.5	+5.3	+0.6	−1.4	+14.0	+12.2

B.5 FFmpeg Filter Specialization

These tables support Chapter 5. They cover the two replacement filters of §5.2 and §5.3 across seven filter configurations, two resolutions and all three hosts, where the chapter reports one host for legibility.

Each measurement is the wall time of a complete `ffmpeg` invocation writing to `-f hash`, so the encoder is outside the timed region but demuxing, decoding and hashing are inside it. Both implementations pay those identically, which makes a raw ratio a whole-pipeline figure rather than a statement about the filter. Timing a null filter at the same resolutions and durations measures that shared cost directly, and subtracting its fitted rate leaves the filter's own; Table B.19 gives the corrected figures, Table B.20 the uncorrected ones, and Table B.21 the decode share and generation cost that connect them.

Running time is linear in clip length for both implementations, so each configuration is summarized by an intercept and a slope. Two passes were run: a short one over durations of one to ten seconds covering every filter, and a longer one over five to forty seconds covering the convolutions, added because a cheap filter's slope is poorly determined by short clips. Each estimate is taken entirely from one pass, and is compared against the null baseline measured in that same pass. Combining the two by taking the faster measurement at each duration was tried and abandoned: repetitions within a pass are interchangeable, so the fastest of them is the least disturbed, but separate passes differ systematically in cache and clock state, and taking a minimum across them selects whichever pass happened to be favored at each duration, which distorts the fitted slope rather than refining it.

These tables give point estimates without confidence intervals, which is a departure from the rest of this appendix. The estimator of Appendix A takes a median over the Cartesian product of measured baseline and optimized runtimes, and bootstraps that median. It does not apply here, because a filter-only rate is never measured directly: it exists only as a difference between two fitted regression slopes, so there are no paired runtimes to take a product over. An interval can still be approximated, by propagating the standard errors of the three slopes involved and retaining the covariance induced by the shared null term, but that construction assumes normality on a handful of residual degrees of freedom, which we prefer to avoid. However, we do use these estimated intervals to determine noise levels. A † marks any entry whose approximate interval reaches zero or spans more than a factor of three; such entries should not be read as results.

Table B.19: Filter-only speedup of the BuildIt filters over their stock counterparts. Decoding, demuxing and hashing are removed by subtracting a null-filter baseline measured at the same resolutions and durations, so these describe the filter alone. These are point estimates only. Unlike every other measurement in this thesis they are not accompanied by an interval, because the quantity is a ratio of differences of fitted regression slopes rather than a ratio of measured runtimes, and the estimator of Appendix A does not apply to it; the approximate intervals that can be computed instead rest on assumptions that appendix deliberately avoids, and are not reported as though they were equivalent. A † marks an entry that such an approximation leaves unconstrained, spanning zero or more than a factor of three. That flag is robust even though the interval behind it is not, and those entries should not be read as results.

Filter	Source	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
identity	1080p	2.6	2.8	1.3
	2160p	2.7	2.7	1.8
vignette	1080p	7.4	11.2	6.1
	2160p	8.5	11.1	9.9
polyheavy	1080p	5.1	5.9	5.6
	2160p	5.9	6.9	6.9
planeosc	1080p	7.7	10.4	7.6
	2160p	8.5	11.6	10.7
sobel 3×3	1080p	2.3	2.8 [†]	9.0 [†]
	2160p	2.8	3.4	1.2
Gaussian, 21 taps	1080p	7.2	7.8	12.6 [†]
	2160p	11.8	7.8	16.8 [†]
LoG 7×7	1080p	8.9	10.6	1.8
	2160p	13.2	9.5	10.0

Table B.20: End-to-end speedup of a complete `ffmpeg` invocation, the quantity actually measured. Decoding and hashing are inside both sides of every ratio here, so each entry is a whole-pipeline figure and a lower bound on the improvement to the filter itself; comparing against Table B.19 shows how large that effect is. It is negligible for `geq`, whose per-pixel cost far exceeds the decoder’s, and dominant for the smallest convolution.

Filter	Source	Ryzen 9850X3D	EPYC 9R05	Xeon 8180
identity	1080p	2.4	2.4	1.3
	2160p	2.4	2.4	1.6
vignette	1080p	6.4	9.1	5.1
	2160p	7.3	9.1	7.6
polyheavy	1080p	4.9	5.7	5.2
	2160p	5.6	6.6	6.4
planeosc	1080p	7.1	9.5	6.7
	2160p	7.8	10.5	9.1
sobel 3×3	1080p	1.3	1.2	1.4
	2160p	1.3	1.3	1.0
Gaussian, 21 taps	1080p	3.2	3.1	3.7
	2160p	3.8	3.2	2.9
LoG 7×7	1080p	5.0	5.3	1.6
	2160p	5.7	4.9	3.8

Table B.21: The two components behind the corrections. *decode* is the share of the stock filter’s measured per-second cost that is demuxing, decoding and hashing rather than filtering, given as a percentage; it is what Table B.19 removes, and the entries above roughly 50% are the ones whose end-to-end figures are most misleading. *gen.* is the one-time cost of generating and compiling the kernel in seconds, measured as the difference between the two fitted intercepts so that `ffmpeg`’s own startup, which both implementations pay, is excluded.

Filter	Source	Ryzen decode	Ryzen gen.	EPYC decode	EPYC gen.	Xeon decode	Xeon gen.
identity	1080p	7	0.28	9	0.10	14	0.52
	2160p	7	0.27	9	0.13	15	0.25
vignette	1080p	2	0.27	2	0.10	4	0.45
	2160p	2	0.28	2	0.10	3	0.40
polyheavy	1080p	1	0.45	1	0.32	2	0.84
	2160p	1	0.48	1	0.37	1	0.76
planeosc	1080p	1	0.31	1	0.31	2	0.64
	2160p	1	0.25	1	0.63	2	0.54
sobel 3×3	1080p	58	0.13	70	0.07	70	0.27
	2160p	67	0.17	70	0.06	87	0.03
Gaussian, 21 taps	1080p	20	0.19	23	0.10	21	−0.01
	2160p	19	0.19	22	0.09	30	−0.34
LoG 7×7	1080p	10	0.31	10	0.18	17	0.23
	2160p	11	0.31	11	0.16	18	0.72

References

- [1] N. D. Jones, C. K. Gomard, and P. Sestoft, *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993, ISBN: 0-13-020249-5.
- [2] C. Consel and F. Noël, “A general approach for run-time specialization and its application to C,” in *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1996, pp. 145–156. DOI: [10.1145/237721.237767](https://doi.org/10.1145/237721.237767)
- [3] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, “Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, pp. 519–530. DOI: [10.1145/2491956.2462176](https://doi.org/10.1145/2491956.2462176)
- [4] T. Chen et al., “TVM: An automated end-to-end optimizing compiler for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.
- [5] J. L. Hennessy and D. A. Patterson, “A new golden age for computer architecture,” *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019. DOI: [10.1145/3282307](https://doi.org/10.1145/3282307)
- [6] Apple Inc. “Apple introduces M4 chip.” Enumerates the CPU, GPU, Neural Engine, media engine with hardware AV1 decode, and display engine on one die, Apple Newsroom. URL: <https://www.apple.com/newsroom/2024/05/apple-introduces-m4-chip/>
- [7] Y. Futamura, “Partial evaluation of computation process—an approach to a compiler-compiler,” *Higher-Order and Symbolic Computation*, vol. 12, no. 4, pp. 381–391, 1999, Reprint of the 1971 original. DOI: [10.1023/A:1010095604496](https://doi.org/10.1023/A:1010095604496)
- [8] W. Taha and T. Sheard, “MetaML and multi-stage programming with explicit annotations,” *Theoretical Computer Science*, vol. 248, no. 1–2, pp. 211–242, 2000. DOI: [10.1016/S0304-3975\(00\)00053-0](https://doi.org/10.1016/S0304-3975(00)00053-0)
- [9] A. Brahmakshatriya and S. Amarasinghe, “BuildIt: A type-based multi-stage programming framework for code generation in C++,” in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 39–51. DOI: [10.1109/CGO51591.2021.9370333](https://doi.org/10.1109/CGO51591.2021.9370333)
- [10] A. Brahmakshatriya and S. Amarasinghe. “BuildIt: Type-based multi-stage programming for C++.” URL: <https://buildit.so/>
- [11] Intel Corporation. “Intel intrinsics guide.” URL: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
- [12] VideoLAN. “dav1d: A small and fast AV1 decoder.” URL: <https://code.videolan.org/videolan/dav1d>

- [13] T. Sheard and S. Peyton Jones, “Template meta-programming for Haskell,” in *Proceedings of the ACM SIGPLAN Workshop on Haskell*, 2002, pp. 1–16. DOI: [10.1145/581690.581691](https://doi.org/10.1145/581690.581691)
- [14] T. Rompf and M. Odersky, “Lightweight modular staging: A pragmatic approach to runtime code generation and compiled DSLs,” in *Proceedings of the Ninth International Conference on Generative Programming and Component Engineering (GPCE)*, 2010, pp. 127–136. DOI: [10.1145/1868294.1868314](https://doi.org/10.1145/1868294.1868314)
- [15] Z. DeVito, J. Hegarty, A. Aiken, P. Hanrahan, and J. Vitek, “Terra: A multi-stage language for high-performance computing,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, pp. 105–116. DOI: [10.1145/2491956.2462166](https://doi.org/10.1145/2491956.2462166)
- [16] Y. Ikarashi, G. L. Bernstein, A. Reinking, H. Genc, and J. Ragan-Kelley, “Exocompilation for productive programming of hardware accelerators,” in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2022, pp. 703–718. DOI: [10.1145/3519939.3523446](https://doi.org/10.1145/3519939.3523446)
- [17] M. Frigo and S. G. Johnson, “The design and implementation of FFTW3,” *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005, Canonical example of shape specialization via generated codelets. DOI: [10.1109/JPROC.2004.840301](https://doi.org/10.1109/JPROC.2004.840301)
- [18] VideoLAN. “x264: A free software library for encoding H.264/AVC video streams.” URL: <https://www.videolan.org/developers/x264.html>
- [19] MulticoreWare. “x265: An open-source HEVC encoder.” URL: <https://www.x265.org/>
- [20] F. Kossentini, H. Guermazi, N. Mahdi, C. Nour, A. Naghdinezhad, H. Tmar, O. Khelif, P. Worth, and F. Ben Amara, “The SVT-AV1 encoder: Overview, features and speed-quality tradeoffs,” in *Applications of Digital Image Processing XLIII*, vol. 11510, SPIE, 2020. DOI: [10.1117/12.2569270](https://doi.org/10.1117/12.2569270)
- [21] Alliance for Open Media. “SVT-AV1: Scalable video technology for AV1.” URL: <https://gitlab.com/AOMediaCodec/SVT-AV1>
- [22] Netflix, Inc. “VMAF: Video multi-method assessment fusion.” Reference implementation, `libvmaf`. URL: <https://github.com/Netflix/vmaf>
- [23] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara. “Toward a practical perceptual video quality metric,” Netflix Technology Blog. URL: <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>
- [24] FFmpeg Project. “FFmpeg: A complete, cross-platform solution to record, convert and stream audio and video.” URL: <https://ffmpeg.org/>
- [25] L. Gritz et al. “Open shading language.” LLVM-backed just-in-time compilation of user shaders. URL: <https://github.com/AcademySoftwareFoundation/OpenShadingLanguage>
- [26] M. Stephenson, J. Babb, and S. Amarasinghe, “Bitwidth analysis with application to silicon compilation,” in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2000, pp. 108–120. DOI: [10.1145/349299.349317](https://doi.org/10.1145/349299.349317)
- [27] Compiler Explorer. “Constant integer division compiled to a multiply and shift.” gcc lowers a division by 5 to a magic-constant multiply, and vectorizes the loop, where an `idiv` would not vectorize. URL: <https://godbolt.org/z/MorG6xaGc>

- [28] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, “Overview of the high efficiency video coding (HEVC) standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012. DOI: [10.1109/TCSVT.2012.2221191](https://doi.org/10.1109/TCSVT.2012.2221191)
- [29] H. R. Sheikh and A. C. Bovik, “Image information and visual quality,” *IEEE Transactions on Image Processing*, vol. 15, no. 2, pp. 430–444, 2006. DOI: [10.1109/TIP.2005.859378](https://doi.org/10.1109/TIP.2005.859378)
- [30] International Telecommunication Union, *H.264: Advanced video coding for generic audiovisual services*, ITU-T Recommendation H.264, 2021. URL: <https://www.itu.int/rec/T-REC-H.264>
- [31] Alliance for Open Media. “AV1 bitstream and decoding process specification.” version 1.0.0 with Errata 1. URL: <https://aomediacodec.github.io/av1-spec/>
- [32] P. Tandon, M. Afonso, J. Sole, and L. Krasula, “CAMBI: Contrast-aware multiscale banding index,” in *Picture Coding Symposium (PCS)*, 2021. DOI: [10.1109/PCS50896.2021.9477464](https://doi.org/10.1109/PCS50896.2021.9477464) arXiv: [2102.00079](https://arxiv.org/abs/2102.00079).
- [33] G. E. Blelloch, “Prefix sums and their applications,” School of Computer Science, Carnegie Mellon University, Technical Report CMU-CS-90-190, 1990.
- [34] C. G. Bampis, P. Gupta, R. Soundararajan, and A. C. Bovik, “SpEED-QA: Spatial efficient entropic differencing for image and video quality,” *IEEE Signal Processing Letters*, vol. 24, no. 9, pp. 1333–1337, 2017. DOI: [10.1109/LSP.2017.2726542](https://doi.org/10.1109/LSP.2017.2726542)
- [35] A. Abel and J. Reineke. “Uops.info: Instruction latency, throughput and port usage tables. ” URL: <https://uops.info/>
- [36] A. Abel and J. Reineke, “Uops.info: Characterizing latency, throughput, and port usage of instructions on intel microarchitectures,” in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 673–686. DOI: [10.1145/3297858.3304062](https://doi.org/10.1145/3297858.3304062)
- [37] Intel Corporation. “Intel 64 and IA-32 architectures optimization reference manual. ” URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [38] Sveriges Television AB. “SVT open content video test suite 2022: Natural complexity.” CC BY 4.0. URL: <https://svt.github.io/en/content/>
- [39] Blender Foundation. “Sintel. ” URL: <https://durian.blender.org/>
- [40] J. L. Hodges and E. L. Lehmann, “The efficiency of some nonparametric competitors of the t -test,” *The Annals of Mathematical Statistics*, vol. 27, no. 2, pp. 324–335, 1956, ISSN: 0003-4851. DOI: [10.1214/aoms/1177728261](https://doi.org/10.1214/aoms/1177728261)